

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

VĂN CÔNG GIA LUẬT
LÊ MINH HÙNG

KHÓA LUẬN TỐT NGHIỆP
THIẾT KẾ BỘ ĐIỀU KHIỂN ĐỒNG BỘ CACHE CHO
CÁC HỆ THỐNG 2 LỖI DỰA TRÊN KIẾN TRÚC TẬP
LỆNH RISC-V RV32IA

**Designing a cache coherence controller for dual-core systems based
on RISC-V RV32IA instruction set architecture**

CỬ NHÂN NGÀNH KỸ THUẬT MÁY TÍNH

TP. HỒ CHÍ MINH, 2026

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

VĂN CÔNG GIA LUẬT – 22520830
LÊ MINH HÙNG – 22520506

KHÓA LUẬN TỐT NGHIỆP
THIẾT KẾ BỘ ĐIỀU KHIỂN ĐỒNG BỘ CACHE CHO
CÁC HỆ THỐNG 2 LỖI DỰA TRÊN KIẾN TRÚC TẬP
LỆNH RISC-V RV32IA

**Designing a cache coherence controller for dual-core systems based
on RISC-V RV32IA instruction set architecture**

CỬ NHÂN NGÀNH KỸ THUẬT MÁY TÍNH

GIẢNG VIÊN HƯỚNG DẪN
ThS. THÂN THẾ TÙNG
KS. TRẦN ĐẠI DƯƠNG

TP. HỒ CHÍ MINH, 2026

THÔNG TIN HỘI ĐỒNG CHẤM KHÓA LUẬN TỐT NGHIỆP

Hội đồng chấm khóa luận tốt nghiệp, thành lập theo Quyết định số 710/QĐ-ĐHCNTT ngày 19 tháng 06 năm 2026 của Hiệu trưởng Trường Đại học Công nghệ Thông tin.

LỜI CẢM ƠN

Nhóm thực hiện đề tài, gồm Lê Minh Hùng và Văn Công Gia Luật, xin được gửi lời tri ân sâu sắc nhất đến ThS. Thân Thế Tùng và KS. Trần Đại Dương. Trong suốt thời gian thực hiện khóa luận, các thầy đã luôn tận tâm hướng dẫn, đưa ra những định hướng chuyên môn quý báu và hỗ trợ chúng em nhiệt tình để hoàn thiện khóa luận này.

Chúng em cũng xin gửi lời cảm ơn chân thành đến tập thể quý thầy cô Khoa Kỹ thuật Máy tính cùng toàn thể cán bộ, giảng viên Trường Đại học Công nghệ Thông tin – Đại học Quốc gia Thành phố Hồ Chí Minh. Những kiến thức chuyên ngành vững chắc mà quý thầy cô truyền đạt chính là nền tảng vô giá giúp chúng em thực hiện thành công nghiên cứu này. Đồng thời, nhóm xin trân trọng cảm ơn quý thầy cô trong Hội đồng phản biện đã dành thời gian xem xét và đánh giá đề tài. Cuối cùng, chúng em xin biết ơn gia đình và bạn bè – những người đã luôn là chỗ dựa tinh thần, động viên và sát cánh cùng chúng em vượt qua mọi thử thách.

Mặc dù đã nỗ lực tìm hiểu và nghiên cứu hết sức, song do những giới hạn về mặt thời gian cũng như kinh nghiệm thực tiễn, khóa luận chắc chắn khó tránh khỏi những thiếu sót nhất định. Nhóm kính mong nhận được sự cảm thông cũng như những ý kiến đóng góp quý báu từ quý thầy cô để đề tài có thể tiếp tục được hoàn thiện và phát triển hơn trong tương lai.

Một lần nữa, chúng em xin chân thành cảm ơn!

TP. Hồ Chí Minh, ngày 10 tháng 07 năm 2026
Sinh viên thực hiện

Văn Công Gia Luật

Lê Minh Hùng

MỤC LỤC

CHƯƠNG 1.	GIỚI THIỆU TỔNG QUAN ĐỀ TÀI.....	3
1.1.	Tổng quan đề tài	3
1.2.	Lý do thực hiện đề tài.....	4
1.3.	Khảo sát các đề tài đã thực hiện	4
1.4.	Mục tiêu của đề tài	5
1.5.	Thách thức của đề tài.....	6
1.6.	Giới hạn của đề tài.....	6
CHƯƠNG 2.	CƠ SỞ LÝ THUYẾT	7
2.1.	Khái niệm và tổng quan về bộ nhớ đệm (Cache).....	7
2.2.	Phân loại bộ nhớ đệm và phương pháp định vị dữ liệu.....	7
2.2.1.	Cấu trúc ánh xạ trực tiếp (Direct Mapped Cache)	8
2.2.2.	Cấu trúc ánh xạ tập hợp liên kết (N-Way Set Associative Cache)	8
2.2.3.	Cấu trúc liên kết đầy đủ (Fully Associative Cache)	9
2.3.	Các chính sách vận hành bộ nhớ đệm (Cache Policies).....	9
2.3.1.	Chính sách thay thế dòng dữ liệu (Cache Replacement Policy).....	9
2.3.2.	Chính sách ghi dữ liệu (Cache Write Policies).....	10
2.4.	Tổng quan về tính nhất quán bộ nhớ đệm (Cache Coherence)	10
2.4.1.	Bài toán mất nhất quán dữ liệu (Cache Incoherence)	10
2.4.2.	Các trường phái kiến trúc giải quyết Cache Coherence	11
2.5.	Các biến thể giao thức Snooping.....	13
2.5.1.	Giao thức MSI (Cấu hình cơ bản).....	13
2.5.2.	Giao thức MESI (Cải tiến quyền độc quyền)	14
2.5.3.	Giao thức MOESI (Tối ưu hóa chia sẻ khối dữ liệu bản).....	14
2.6.	Sơ lược về kiến trúc tập lệnh RISC-V RV32IA.....	15
2.6.1.	Tổng quan	15
2.6.2.	Kiến trúc tập lệnh cơ sở RV32I	16
2.7.	Tổng quan về giao thức AXI.....	19
2.7.1.	Giới thiệu	19

2.7.2.	Kiến trúc giao thức AXI.....	19
2.7.3.	Mô tả các quá trình giao dịch.....	19
CHƯƠNG 3.	THIẾT KẾ HỆ THỐNG.....	21
3.1.	Tổng quan hệ thống.....	21
3.2.	Thiết kế hệ thống phần cứng.....	22
3.2.1.	Thiết kế phần cứng lõi xử lý RISC-V RV32I.....	22
3.2.2.	Thiết kế phần cứng bộ nhớ đệm.....	24
3.2.3.	Thiết kế mạng lưới phân xử và đồng bộ đa lõi.....	30
CHƯƠNG 4.	THIẾT KẾ CHI TIẾT.....	33
4.1.	Thiết kế chi tiết phần cứng.....	33
4.2.	Thiết kế chi tiết lõi vi xử lý RV32IA.....	33
4.2.1.	Vi kiến trúc Pipeline 6 tầng và Cơ chế đồng bộ.....	33
4.2.2.	Hiện thực mạch logic Khối xử lý rủi ro (Hazard Unit).....	34
4.2.3.	Hiện thực khối dự đoán rẽ nhánh.....	34
4.3.	Hiện thực giải thuật thay thế PLRUt.....	36
4.3.1.	Vi kiến trúc tổng quát của mô-đun PLRUt.....	37
4.3.2.	Cấu trúc định tuyến cây nhị phân (Binary Tree Routing).....	37
4.3.3.	Mạch tổ hợp phân rã tín hiệu và cập nhật trạng thái đối ngẫu.....	38
4.4.	Thiết kế chi tiết bộ nhớ đệm phân cấp.....	39
4.4.1.	Vi kiến trúc nền tảng (Common Cache Architecture).....	39
4.4.2.	Kiến trúc D-Cache.....	40
4.4.3.	Thiết kế phần cứng L1 I-Cache (Instruction Cache).....	44
4.4.4.	Thiết kế phần cứng L2 Cache và giao tiếp AXI4 Full.....	47
4.5.	Hiện thực SoC để nạp xuống kit Virtex 7.....	49
CHƯƠNG 5.	MÔ PHỎNG VÀ ĐÁNH GIÁ THIẾT KẾ.....	50
5.1.	Kế hoạch kiểm thử tổng quan.....	50
5.1.1.	Giai đoạn 1: Xác minh mức lõi đơn (Single-Core Verification).....	50
5.1.2.	Giai đoạn 2: Xác minh hệ thống đa lõi.....	50
5.1.3.	Hệ thống kiểm thử tự động tập lệnh RV32IA.....	50

5.1.4.	Kế hoạch kiểm thử với lỗi kép.....	52
5.1.5.	Kết quả kiểm thử lỗi đơn.	53
5.1.6.	Kết quả kiểm thử lỗi kép.....	56
5.1.7.	Kết quả tổng hợp và thực thi thiết kế.....	57
CHƯƠNG 6.	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	63
6.1.	Kết luận	63
6.2.	Ưu điểm và hạn chế của thiết kế	63
6.2.1.	Ưu điểm.....	63
6.2.2.	Hạn chế	64
6.3.	Hướng phát triển.....	64
TÀI LIỆU THAM KHẢO.....		66
PHỤ LỤC I. NGUỒN KỊCH BẢN KIỂM THỬ		69
PHỤ LỤC II. KẾT QUẢ MÔ PHỎNG		74
PHỤ LỤC III. BẢNG MÔ TẢ TÍN HIỆU CÁC MODULE		79

DANH MỤC HÌNH

Hình 2.1: Sơ đồ phân cấp bộ nhớ trong hệ thống	7
Hình 2.2: Kiến trúc phần cứng bộ nhớ đệm 2-Way	8
Hình 2.3: Sơ đồ tổng quát của giao thức Thư mục	11
Hình 2.4: Biểu đồ chuyển đổi trạng thái của giao thức MSI	13
Hình 2.5: Sơ đồ trạng thái giao thức MESI.....	14
Hình 2.6: Biểu đồ chuyển đổi trạng thái của giao thức MOESI	15
Hình 2.7: Định dạng mã hóa các nhóm lệnh trong kiến trúc RISC-V	17
Hình 2.8: Cấu trúc tập lệnh mở rộng RV32A	18
Hình 2.9: Sơ đồ luồng tín hiệu giao dịch ghi trên chuẩn AXI	20
Hình 2.10: Sơ đồ luồng tín hiệu giao dịch đọc trên chuẩn AXI	20
Hình 3.1: Sơ đồ hệ thống 2 lõi sử dụng Cache Coherence	21
Hình 3.2: Kiến trúc tổng quát lõi vi xử lý RISC-V RV32I.....	24
Hình 3.3: Sơ đồ khối kiến trúc Cache 4-Way Set Associative	25
Hình 3.4: Sơ đồ khối vi kiến trúc L1 D-Cache	26
Hình 3.5: Sơ đồ khối vi kiến trúc L1 I-Cache.....	28
Hình 3.6: Vi kiến trúc L2 Cache và giao tiếp AXI4 Full.....	29
Hình 3.7: Vi kiến trúc khối phân xử lệnh.....	31
Hình 3.8: Sơ đồ khối và luồng dữ liệu của bộ phân xử luồng dữ liệu	31
Hình 3.9: Sơ đồ vi kiến trúc và luồng dữ liệu của Khối phân xử trung tâm.....	32
Hình 4.1: Sơ đồ vi kiến trúc tổng thể khối dự đoán rẽ nhánh động BPU	35
Hình 4.2: Sơ đồ thiết kế mạch phần cứng khối BTB	35
Hình 4.3: Sơ đồ mạch băm logic GShare và máy trạng thái bộ đếm 2-bit	36
Hình 4.4: Sơ đồ khối phân hệ quản lý chính sách thay thế.....	37
Hình 4.5: Cấu trúc cây nhị phân định tuyến của giải thuật PLRUt	38
Hình 4.6: Lưu đồ logic mạch cập nhật trạng thái nhánh PLRUt	38
Hình 4.7: Sơ đồ khối vi kiến trúc của bộ nhớ đệm L1 D-Cache.....	40
Hình 4.8: Sơ đồ vi kiến trúc mạch tổ hợp giải mã trạng thái MOESI	41
Hình 4.9: Sơ đồ vi kiến trúc mức cổng logic của Khối giám sát Bus.....	42

Hình 4.10: Sơ đồ vi kiến trúc mạch logic của Đơn vị tính toán nguyên tử	43
Hình 4.11: Tổng quan interface của D-cache	44
Hình 4.12: Sơ đồ khối vi kiến trúc của L1 I-Cache	45
Hình 4.13: Tổng quan giao diện tín hiệu (Interface) của L1 I-Cache.....	46
Hình 4.14: Sơ đồ khối vi kiến trúc L2 Cache	47
Hình 4.15: Tổng quan giao diện tín hiệu (Interface) của L2 Cache	48
Hình 4.16: Block design trên Vivado.....	49
Hình 5.1: Kết quả tổng hợp timing toàn hệ thống.....	57
Hình 5.2: Tổng hợp tài nguyên sử dụng của toàn hệ thống	58
Hình 5.3: Phân tách tài nguyên theo khối thành phần	58
Hình 5.4: Báo cáo tổng hợp năng lượng tiêu thụ toàn hệ thống	59

DANH MỤC BẢNG

Bảng 1.1: Minh họa hiện tượng mất nhất quán dữ liệu trong hệ thống đa lõi	3
Bảng 2.1: So sánh đặc tính phần cứng các giải thuật thay thế bộ nhớ đệm	10
Bảng 2.2: So sánh đặc tính kỹ thuật giữa kiến trúc Snooping và Directory	12
Bảng 2.3: Phân loại các tập lệnh cơ sở của kiến trúc RISC-V	16
Bảng 3.1: Thông số cấu hình phân cấp bộ nhớ đệm	22
Bảng 3.2: Mô tả 6 tầng trong kiến trúc pipeline lõi RV32IA	23
Bảng 3.3: Thông số cấu hình hệ thống phân cấp bộ nhớ đệm	24
Bảng 3.4: Mô tả các khối chức năng vi kiến trúc L1 D-Cache	27
Bảng 5.1: Kết quả thực thi bộ kiểm thử chuẩn RISC-V Test	54
Bảng 5.2: Tổng hợp kết quả kiểm thử chức năng tập lệnh cơ sở (RV32I)	55
Bảng 5.3: Tổng hợp thông số hiệu năng xử lý và bộ nhớ đệm L1, L2	59
Bảng 5.4: So sánh kết quả đạt được với các công trình khác	60

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Viết đầy đủ	Ý nghĩa
ALU	Arithmetic Logic Unit	Đơn vị tính toán số học và logic
AMBA	Advanced Microcontroller Bus Architecture	Kiến trúc bus vi điều khiển tiên tiến chuẩn giao tiếp của ARM
AMO	Atomic Memory Operations	Các phép toán bộ nhớ nguyên tử
AXI	Advanced eXtensible Interface	Giao diện mở rộng tiên tiến với giao thức bus trong chuẩn AMBA
BPU	Branch Prediction Unit	Khối dự đoán rẽ nhánh
BTB	Branch Target Buffer	Bộ đệm địa chỉ đích rẽ nhánh
CPU	Central Processing Unit	Đơn vị xử lý trung tâm
CSR	Control and Status Register	Thanh ghi điều khiển và trạng thái
D-Cache	Data Cache	Bộ nhớ đệm dữ liệu để lưu tạm dữ liệu đọc/ghi bởi lõi CPU
DMA	Direct Memory Access	Truy cập bộ nhớ trực tiếp
FF	Flip-Flop	Phần tử nhớ 1-bit trong mạch số đồng bộ, đơn vị đo thanh ghi trên FPGA
FPGA	Field-Programmable Gate Array	Mảng cổng logic lập trình được tại hiện trường
FSM	Finite State Machine	Máy trạng thái hữu hạn

GHSR	Global History Shift Register	Thanh ghi dịch lịch sử toàn cục để lưu kết quả rẽ nhánh cho GShare
HDL	Hardware Description Language	Ngôn ngữ mô tả phần cứng
I-Cache	Instruction Cache	Bộ nhớ đệm lệnh
LR/SC	Load-Reserved / Store-Conditional	Cấp lệnh đặt trước / lưu có điều kiện
LRU	Least Recently Used	Thuật toán thay thế ưu tiên loại bỏ khối ít được dùng nhất gần đây
LUT	Look-Up Table	Bảng tra cứu
LUTRAM	LUT-based Random-Access Memory	Bộ nhớ RAM được xây dựng từ LUT trên FPGA (khi cần dung lượng nhỏ)
MESI	Modified, Exclusive, Shared, Invalid	Giao thức đồng bộ 4 trạng thái: Đã sửa, Độc quyền, Chia sẻ, Không hợp lệ
MMU	Memory Management Unit	Đơn vị quản lý bộ nhớ (ánh xạ địa chỉ ảo sang địa chỉ vật lý)
MOESI	Modified, Owned, Exclusive, Shared, Invalid	Giao thức đồng bộ 5 trạng thái: Đã sửa, Sở hữu, Độc quyền, Chia sẻ, Không hợp lệ
MSI	Modified, Shared, Invalid	Giao thức đồng bộ 3 trạng thái cơ bản: Đã sửa, Chia sẻ, Không hợp lệ
MUX	Multiplexer	Bộ ghép kênh (mạch chọn một trong nhiều tín hiệu đầu vào để cho ra đầu ra)

PC	Program Counter	Bộ đếm chương trình
PHT	Pattern History Table	Bảng lịch sử mẫu (lưu trạng thái bộ đếm bão hòa 2-bit để dự đoán rẽ nhánh)
PIM	Policy Information Memory	Khối/mảng nhớ lưu trữ vector lịch sử truy cập (history bits) phục vụ giải thuật thay thế PLRUt
PLRUt	Tree-based Pseudo Least Recently Used	Thuật toán thay thế xấp xỉ LRU cây nhị phân, tiết kiệm tài nguyên phần cứng
RAM	Random-Access Memory	Bộ nhớ truy cập ngẫu nhiên
RTL	Register Transfer Level	Mức chuyển thanh ghi
SoC	System on Chip	Hệ thống trên chip
SRAM	Static Random-Access Memory	Bộ nhớ truy cập ngẫu nhiên tĩnh dùng Flip-Flop để làm Cache
SRRIP	Static Re-Reference Interval Prediction	Thuật toán thay thế dự đoán, truy cập lại, thường áp dụng cho cache L2/L3
WNS	Worst Negative Slack	Độ dư thời gian âm tệ nhất ($WNS > 0$ nghĩa là thiết kế đạt yêu cầu tần số)

TÓM TẮT KHÓA LUẬN

Nội dung chính của khóa luận tập trung vào việc thiết kế và hiện thực hóa hệ thống vi xử lý lõi kép dựa trên kiến trúc tập lệnh RISC-V RV32IA. Thiết kế được xây dựng với vi kiến trúc pipeline 6 tầng, tích hợp bộ dự đoán rẽ nhánh động và cơ chế xử lý xung đột nhằm nâng cao hiệu quả thực thi lệnh. Về tổ chức bộ nhớ, hệ thống sử dụng phân cấp bộ nhớ đệm theo cấu trúc 4-Way Set Associative, bao gồm L1 I/D-Cache 16-Set hoạt động độc lập trên từng lõi và L2 Cache 64-Set chia sẻ chung giữa hai lõi. Các thành phần bộ nhớ được kết nối thông qua chuẩn bus AXI4 Full, giúp chuẩn hóa quá trình truyền nhận dữ liệu và hỗ trợ tích hợp hệ thống ở mức phần cứng. Khóa luận tập trung xây dựng và cấu hình kiến trúc bộ nhớ đệm cho cấu trúc đa lõi với giải thuật thay thế PLRUt 3-bit, chính sách Write-back và Read/Write Allocate. Trên nền tảng đó, bộ điều khiển đồng bộ cache được triển khai tại tầng L1 D-Cache bằng giao thức MOESI Snoopy, cho phép theo dõi và chuyển đổi dữ liệu qua năm trạng thái Modified, Owned, Exclusive, Shared và Invalid nhằm đảm bảo tính nhất quán khi hai lõi cùng truy cập vùng nhớ chia sẻ. Đồng thời, thiết kế tích hợp bộ hỗ trợ phần cứng cho tập lệnh Atomic thuộc chuẩn RV32A; trong đó các lệnh AMO được xử lý bởi khối AMO ALU theo chu trình đọc–sửa–ghi, còn lệnh LR/SC được kiểm soát trực tiếp bởi D-Cache Controller thông qua cơ chế reservation, giúp duy trì toàn vẹn dữ liệu trong các tình huống truy cập đồng thời.

Sau khi hoàn thiện thiết kế phần cứng ở mức RTL bằng ngôn ngữ Verilog HDL, một môi trường kiểm thử và đồng xác minh (co-verification) đã được xây dựng nhằm đánh giá toàn diện chức năng từ mức khối đến toàn hệ thống. Kết quả mô phỏng cho thấy thiết kế hoạt động chính xác theo đúng các kịch bản kiểm thử thiết lập. Đồng thời, hệ thống đã được tổng hợp (synthesis) thành công trên nền tảng AMD/Xilinx Vivado, chứng minh tính ổn định ở mức logic và triển khai thực nghiệm trên kit Virtex-7 VC707.

ABSTRACT

This thesis presents the design and implementation of a dual-core system based on the RISC-V RV32IA instruction set architecture. The processor is built upon a 6-stage pipeline microarchitecture, incorporating a dynamic branch predictor and hazard handling mechanisms to improve instruction execution efficiency. The memory hierarchy employs a 4-Way Set Associative cache structure, comprising a 16-Set L1 I/D-Cache operating independently on each core and a shared 64-Set L2 Cache. All memory components are interconnected via the AXI4 Full bus standard, ensuring standardized data transfer and facilitating hardware-level system integration. The thesis focuses on constructing and configuring the cache memory architecture for a multi-core environment, utilizing a 3-bit PLRUt replacement algorithm with Write-back and Read/Write Allocate policies. A cache coherence controller is implemented at the L1 D-Cache level using the MOESI Snoopy protocol, enabling state transitions across five states – Modified, Owned, Exclusive, Shared, and Invalid – to ensure coherency when both cores access shared memory regions. The design also integrates hardware support for the RV32A Atomic extension; AMO instructions are handled by a dedicated AMO ALU unit following a read-modify-write cycle, while LR/SC instructions are managed by the D-Cache Controller through a reservation mechanism, preserving data integrity under concurrent access scenarios.

Upon completing the RTL-level hardware design in Verilog HDL, a verification and co-verification environment was developed to evaluate functional correctness from the block level to the full system level. Simulation results confirm that the design operates correctly across all defined test scenarios. The system was also synthesized on Vivado and validated on the Virtex-7 VC707 FPGA, confirming logical stability and deployability.

CHƯƠNG 1. GIỚI THIỆU TỔNG QUAN ĐỀ TÀI

1.1. Tổng quan đề tài

Nâng cao hiệu năng vi xử lý hiện nay chủ yếu dựa vào kiến trúc đa lõi nhằm khai thác tối đa khả năng xử lý song song. Để giảm độ trễ truy cập bộ nhớ, hệ thống phân cấp bộ nhớ đệm (L1/L2 Cache) được triển khai. Tuy nhiên, việc mỗi lõi sở hữu bộ nhớ đệm riêng biệt dẫn đến thách thức nghiêm trọng về tính nhất quán dữ liệu (Cache Coherence).

Khi nhiều lõi cùng truy cập và chỉnh sửa một vùng nhớ chia sẻ, dữ liệu có thể tồn tại dưới nhiều bản sao. Nếu một lõi cập nhật mà không đồng bộ, các lõi khác sẽ đọc nhầm dữ liệu cũ, dẫn đến sai lệch logic chương trình. Hiện tượng này được minh họa chi tiết qua trình tự truy xuất tại Bảng 1.1.

Bảng 1.1: Minh họa hiện tượng mất nhất quán dữ liệu trong hệ thống đa lõi

Bước	Thao Tác	Cache Core 0	Cache Core 1	Bộ nhớ chính (Địa chỉ X)
0	Khởi tạo	—	—	0x00 = 10
1	Core 0 đọc địa chỉ 0x00	0x00 = 10	—	0x00 = 10
2	Core 1 đọc địa chỉ 0x00	0x00 = 10	0x00 = 10	0x00 = 10
3	Core 0 ghi 99 vào địa chỉ 0x00	0x00 = 99	0x00 = 10 (cũ)	0x00 = 10 (chưa cập nhật)
4	Core 1 đọc lại X	0x00 = 99	0x00 = 10 (stale)	0x00 = 10

Từ Bảng 1.1, nếu Core 1 xử lý dựa trên bản sao lỗi thời, kết quả toàn hệ thống sẽ sai lệch. Để giải quyết triệt để ở mức phần cứng, khóa luận tập trung thiết kế RTL hệ thống vi xử lý lõi kép RISC-V, tích hợp L1/L2 Cache và ứng dụng giao thức theo dõi bus (snooping) nhằm đảm bảo tính nhất quán dữ liệu.

1.2. Lý do thực hiện đề tài

Phát triển vi xử lý đa lõi dựa trên kiến trúc mở RISC-V là nhu cầu thực tiễn lớn trong thiết kế vi mạch. Tuy nhiên, thách thức lớn nhất khi chuyển từ lý thuyết sang hiện thực hóa (implement) phần cứng là giải quyết bài toán xung đột bộ nhớ và kiểm soát tương tranh dữ liệu.

Thay vì nghiên cứu lý thuyết đơn thuần, khóa luận tập trung hoàn toàn vào thiết kế RTL hệ thống lõi kép RV32IA. Việc chọn tích hợp giao thức MOESI Snoopy và bộ hỗ trợ tập lệnh Atomic tại tầng D-Cache là yêu cầu kỹ thuật bắt buộc để phần cứng đa lõi có thể vận hành thực tế. Đề tài sẽ đi sâu giải quyết trực tiếp các vấn đề vi kiến trúc, xử lý tín hiệu và kiểm chứng phần cứng.

1.3. Khảo sát các đề tài đã thực hiện

Năm 2022, tác giả M. Majiid đã thiết kế bộ vi xử lý đơn lõi pipeline 5 tầng, tích hợp bộ nhớ cache, bộ dự đoán rẽ nhánh và hiện thực thành công trên nền tảng FPGA [15]. Tuy nhiên, bộ vi xử lý sử dụng kiến trúc 64-bit và được xây dựng lõi đơn, nên chưa giải quyết bài toán đa lõi và cơ chế đồng bộ Cache.

Năm 2023, nhóm tác giả D. Emil, M. Hamdy và G. Nagib đã thiết kế hệ thống vi xử lý 2 lõi sử dụng kiến trúc liên kết chuẩn AXI [8]. Công trình này có giá trị tham khảo về tổ chức giao tiếp trong hệ thống đa lõi. Tuy nhiên, nghiên cứu chủ yếu tập trung vào kiến trúc liên kết giao tiếp, chưa đi sâu vào tối ưu vi kiến trúc lõi CPU, bộ điều khiển đồng bộ cache và chưa hỗ trợ tập lệnh nguyên tử.

Năm 2022, nhóm sinh viên Trần Quốc Trường và Lê Phúc Nhật Nam đã thiết kế bộ vi xử lý RISC-V 32-bit sử dụng kiến trúc siêu vô hướng 2 luồng, có hỗ trợ cache [5]. Bộ vi xử lý đạt tần số hoạt động 110 MHz. Tuy nhiên, thiết kế chưa hỗ trợ cơ chế đồng bộ cache giữa các lõi, do đó chưa giải quyết được vấn đề nhất quán dữ liệu trong hệ thống đa xử lý.

Năm 2023, nhóm sinh viên Cao Thanh Bình và Đỗ Phi Hùng đã thiết kế bộ vi xử lý đơn lõi RISC-V RV32IF pipeline 5 tầng, hỗ trợ 4-Way Set Associative Cache [2]. Bộ vi xử lý đạt tần số hoạt động 140 MHz. Tuy nhiên, thiết kế vẫn thuộc hướng đơn lõi, chưa hỗ trợ tính năng đa xử lý hoặc cơ chế đồng bộ cache.

Năm 2024, nhóm sinh viên Trương Trần Hoài Nam và Nguyễn Đức Duy Khang đã hiện thực hệ thống NoC với 8 lõi, tích hợp Branch Prediction [3]. Bộ vi xử lý đạt tần số hoạt động 100 MHz. Tuy nhiên, trọng tâm của đề tài là tổ chức kết nối NoC, trong khi cache chưa được tối ưu và cơ chế đồng bộ cache cho hệ thống đa lõi chưa được phát triển đầy đủ.

Năm 2025, nhóm sinh viên Khuru Thành Tài và Phạm Trí Đức đã thiết kế bộ vi xử lý 2 lõi pipeline 5 tầng [4]. Bộ vi xử lý đạt tần số hoạt động 75,4 MHz. Tuy nhiên, thiết kế chưa được hiện thực trên FPGA và chỉ xét trong trường hợp lý tưởng với bộ nhớ đơn port, nên chưa xử lý đầy đủ các vấn đề phát sinh khi nhiều lõi cùng truy cập bộ nhớ.

Năm 2025, nhóm sinh viên Nguyễn Quốc Trường An và Dương Hoàng Tuấn đã thiết kế bộ vi xử lý 2 lõi hỗ trợ đồng bộ cache bằng giao thức MOESI Snoopy [1]. Bộ vi xử lý đạt tần số hoạt động 124 MHz. Tuy nhiên, thiết kế vẫn dừng lại ở kiến trúc RV32I, chưa hỗ trợ tập lệnh nguyên tử A, nên chưa đáp ứng cho các bài toán đồng bộ hóa dữ liệu trong môi trường đa lõi.

Kế thừa kết quả trên, đề tài định hướng thiết kế hệ thống lõi kép hoàn chỉnh RISC-V RV32IA, giải quyết bài toán hiệu năng và nhất quán bằng việc kết hợp L1/L2 Cache, giao thức MOESI Snoopy và hỗ trợ phần cứng cho tập lệnh Atomic.

1.4. Mục tiêu của đề tài

Mục tiêu cốt lõi là thiết kế, kiểm chứng và tổng hợp thành công hệ thống vi xử lý lõi kép RV32IA, hoạt động ổn định với cơ chế đồng bộ phần cứng MOESI. Các mục tiêu cụ thể gồm:

- Lõi RV32IA: Pipeline 6 tầng, dự đoán rẽ nhánh động, xử lý xung đột
- Thiết kế hệ thống 4-Way Set Associative Cache: L1 I/D-Cache 16-Set (cục bộ) và L2 Cache 64-Set (chia sẻ).
- Đồng bộ dữ liệu: Cài đặt giao thức MOESI Snoopy tại L1 D-Cache.
- Tập lệnh Atomic (RV32A): Lệnh AMO xử lý qua khối AMO ALU; lệnh LR/SC kiểm soát bởi D-Cache Controller.

- Tích hợp & Kiểm chứng: Chuẩn AXI4 Full, kiểm chứng UVM, tổng hợp trên AMD/Xilinx Vivado (FPGA Virtex-7).

1.5. Thách thức của đề tài

Quá trình hiện thực hóa RTL đối mặt với các bài toán phức tạp:

- Phân xử tài nguyên: Điều phối bus (arbiter) và xử lý stall/flush chéo giữa 2 lõi 6 tầng xuống L2.
- Máy trạng thái MOESI: Phải luân chuyển chính xác từng chu kỳ và phối hợp khắt khe với khối Atomic nhằm quản lý không gian đặt trước (reservation slot).
- Kiểm chứng (Verification): Phải bao phủ các tình huống góc (cả 2 lõi cùng miss cache, cùng kích hoạt MOESI và lệnh Atomic).

1.6. Giới hạn của đề tài

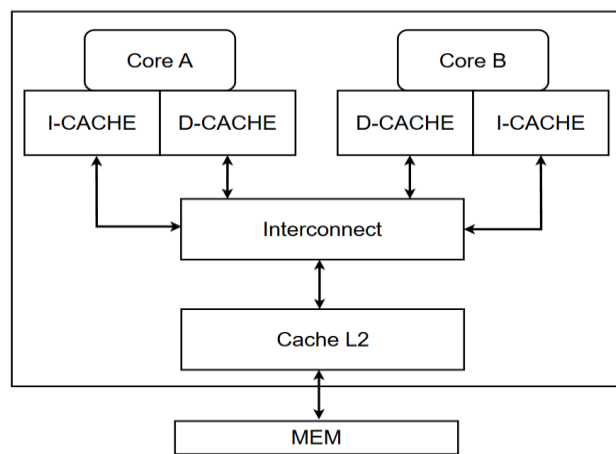
Trong khuôn khổ của hướng nghiên cứu hiện tại, đề tài được giới hạn:

- Chỉ giới hạn ở cấu trúc 2 lõi RV32IA.
- Giao thức MOESI chỉ áp dụng ở L1 D-Cache; L2 Cache chưa tích hợp đồng bộ phân cấp phức tạp.
- Chưa có MMU/DMA/Ngắt ngoại vi để chạy hệ điều hành; kết quả chủ yếu dựa trên mô phỏng và tổng hợp RTL.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm và tổng quan về bộ nhớ đệm (Cache)

Bộ nhớ đệm (Cache) được đặt gần lõi xử lý nhằm mục đích thu hẹp khoảng cách về tốc độ giữa CPU và bộ nhớ chính. L1 Cache thường sử dụng kiến trúc Harvard (tách biệt I-Cache và D-Cache) để cho phép nạp lệnh và xuất/nhập dữ liệu song song [7]. Trạng thái truy cập được phân loại thành Cache Hit (dữ liệu sẵn có) và Cache Miss (trượt, buộc pipeline phải tạm dừng để nạp dữ liệu từ tầng nhớ dưới) [9]. Mô hình phân cấp hệ thống nhớ được khái quát hóa tại Hình 2.1.



Hình 2.1: Sơ đồ phân cấp bộ nhớ trong hệ thống

Hệ thống bộ nhớ đệm được thiết kế với kích thước dòng 64-byte nhằm cân bằng giữa tỷ lệ trượt và độ trễ bus [10]. Địa chỉ 32-bit được phân rã thành ba trường cơ bản: Byte Offset, Set Index, và Tag [9]. Việc quản lý bit trạng thái được tinh chỉnh riêng cho từng cấp độ: L1 I-Cache (chỉ đọc) chỉ dùng bit Valid (V); L1 D-Cache sử dụng 3-bit mã hóa theo giao thức MOESI để bao hàm toàn diện các trạng thái mà không cần tách rời bit V/D [13]; trong khi đó, L2 Cache dùng chung tích hợp cả bit Valid (V) và Dirty (D) để phục vụ chính sách ghi lại (Write-back).

2.2. Phân loại bộ nhớ đệm và phương pháp định vị dữ liệu

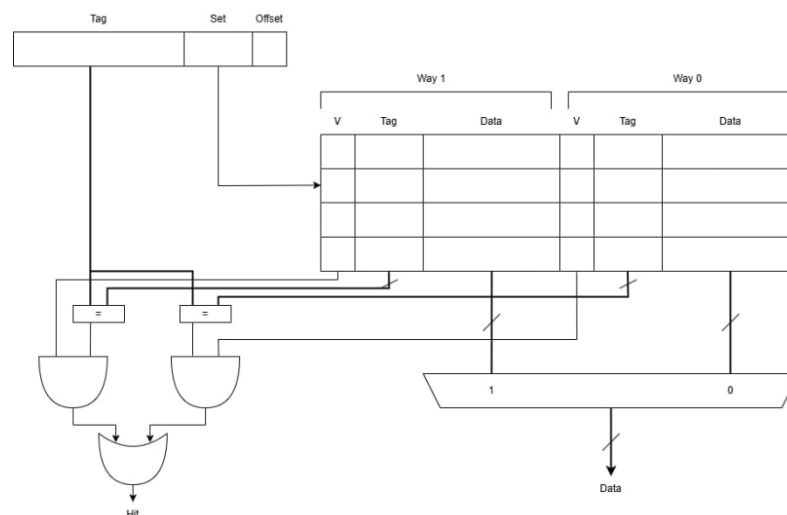
Để định vị dữ liệu giữa bộ nhớ chính và bộ nhớ đệm, kiến trúc hệ thống thường áp dụng ba phương pháp ánh xạ cơ bản [7].

2.2.1. Cấu trúc ánh xạ trực tiếp (Direct Mapped Cache)

Kiến trúc này định nghĩa mỗi nhóm (Set) chỉ chứa một khối dữ liệu ($N = 1$), do đó tổng số nhóm (S) sẽ bằng đúng tổng số khối (B) có trong bộ nhớ đệm. Về cơ chế định vị, một địa chỉ bộ nhớ chỉ ánh xạ vào một vị trí Set duy nhất cố định theo công thức $\text{Set Index} = (\text{Địa chỉ khối ở bộ nhớ chính}) \bmod S$ (1.1) Đặc điểm phần cứng của kiến trúc này là tối ưu về mặt tài nguyên vì chỉ yêu cầu một bộ so sánh thẻ (Single Tag Comparator) cho mỗi chu kỳ truy xuất, tuy nhiên, nó lại có nhược điểm là tỷ lệ trượt do xung đột (Conflict Miss) rất cao khi các địa chỉ bộ nhớ khác nhau liên tục tranh chấp cùng một chỉ số Set [7].

2.2.2. Cấu trúc ánh xạ tập hợp liên kết (N-Way Set Associative Cache)

Để giảm thiểu xung đột vị trí, kiến trúc N-Way Set Associative phân chia cache thành S nhóm, mỗi nhóm gồm N đường (Way) độc lập [9]. Trường Set Index trở vào một nhóm cố định, nhưng dữ liệu có thể lưu linh hoạt tại bất kỳ đường nào trong N đường thuộc nhóm. Do đó, phần cứng cần N bộ so sánh Tag hoạt động song song, một Multiplexer để xuất dữ liệu khi Cache Hit, và khối logic xử lý thuật toán thay thế (như PLRUt 3-bit) để quyết định trục xuất dòng khi nhóm đầy [9]. Cấu trúc phần cứng của mô hình 2-Way được mô tả chi tiết và trực quan tại Hình 2.2 (Lưu ý: Khóa luận này triển khai cấu hình 4-Way Set Associative).



Hình 2.2: Kiến trúc phần cứng bộ nhớ đệm 2-Way

2.2.3. Cấu trúc liên kết đầy đủ (Fully Associative Cache)

Trong cấu trúc này, bộ nhớ đệm được tổ chức thành một nhóm duy nhất ($S = 1$). Do không có trường Set Index, dữ liệu từ bộ nhớ chính có thể được ánh xạ vào bất kỳ vị trí trống nào trong cache, giúp triệt tiêu hoàn toàn lỗi trượt do xung đột địa chỉ (Conflict Miss). Tuy nhiên, kiến trúc này tiêu tốn diện tích mạch và công suất rất lớn do đòi hỏi số lượng bộ so sánh Tag bằng đúng tổng số khối để kiểm tra song song, nên thường chỉ áp dụng cho bộ đệm nhỏ như TLB.

Vì giới hạn này, khóa luận lựa chọn mô hình 4-Way Set-Associative nhằm khắc phục tỷ lệ trượt cao của kiến trúc ánh xạ trực tiếp (Direct Mapped), đồng thời tiết kiệm tài nguyên khi chỉ cần duy trì 4 bộ so sánh Tag. Giải pháp này giúp hạn chế tối đa độ trễ tín hiệu định tuyến trên mạch FPGA so với các cấu hình 8-Way hay Fully Associative.

2.3. Các chính sách vận hành bộ nhớ đệm (Cache Policies)

2.3.1. Chính sách thay thế dòng dữ liệu (Cache Replacement Policy)

Khi một nhóm (Set) đã lấp đầy cả 4 đường (4-Way), bộ điều khiển cache phải kích hoạt thuật toán để chọn ra một khối dữ liệu cũ (victim block) và trục xuất nó, nhường chỗ cho dữ liệu mới nạp vào. Có 3 thuật toán thay thế phổ biến trong thiết kế kiến trúc máy tính [9] và Đặc tính phân cứng của các giải thuật này được so sánh tổng quát tại Bảng 2.1:

- LRU (Least Recently Used): Trục xuất dòng dữ liệu lâu nhất chưa được truy cập. Hiệu năng lý thuyết tốt nhất nhưng tiêu tốn nhiều tài nguyên logic [7].
- SRRIP (Static Re-Reference Interval Prediction): Dự đoán khoảng thời gian dữ liệu sẽ được truy cập lại. Cấu trúc phức tạp, thường chỉ áp dụng cho bộ đệm dùng chung dung lượng lớn (L2/L3) [9].
- Pseudo-LRU (PLRUt - Cây nhị phân): Giải thuật xấp xỉ LRU. Sử dụng cấu trúc cây nhị phân để giảm thiểu số lượng bit trạng thái điều khiển và tối giản mạch logic [9].

Bảng 2.1: So sánh đặc tính phần cứng các giải thuật thay thế bộ nhớ đệm

Tiêu chí so sánh	Giải thuật LRU	Giải thuật SRRIP	Giải thuật PLRUt
Số bit trạng thái / Set	5 - 6 bit	8 bit (2 bit x 4 đường)	3 bit
Độ phức tạp mạch RTL	Rất cao	Cao	Rất thấp
Tỷ lệ trượt (Miss Rate)	Tối ưu nhất	Thấp	Tiệm cận LRU
Độ trễ (Latency)	Lớn	Trung bình	Rất nhỏ (1 chu kỳ)

Quyết định thiết kế: Khóa luận áp dụng giải thuật PLRUt cho khối cache 4-Way Set-Associative. Lựa chọn này giúp mạch đạt hiệu suất ép tỷ lệ trượt xấp xỉ thuật toán LRU, nhưng chi phí thiết kế phần cứng cực thấp (chỉ tốn 3 bit trạng thái cho mỗi Set), đảm bảo mạch RTL gọn gàng và đáp ứng tín hiệu nhanh.

2.3.2. Chính sách ghi dữ liệu (Cache Write Policies)

Chính sách ghi quy định thời điểm và cách thức đồng bộ dữ liệu từ Cache xuống các tầng bộ nhớ thấp hơn nhằm đảm bảo tính nhất quán trên toàn hệ thống [7]. Hai cơ chế phổ biến nhất bao gồm Write-through và Write-back [9]. Trong đó, cơ chế Write-through cập nhật đồng thời dữ liệu vào Cache và bộ nhớ cấp thấp hơn giúp tối giản mạch điều khiển nhưng lại gây áp lực lớn lên băng thông bus [10]. Ngược lại, cơ chế Write-back chỉ ghi cục bộ tại Cache (đánh dấu bằng bit Dirty - D) và mới đồng bộ xuống bộ nhớ chính khi khối dữ liệu đó bị trục xuất [7], giúp giảm thiểu đáng kể lưu lượng bus ngoại vi và đặc biệt tối ưu cho các thiết kế bộ xử lý đa lõi [13].

2.4. Tổng quan về tính nhất quán bộ nhớ đệm (Cache Coherence)

2.4.1. Bài toán mất nhất quán dữ liệu (Cache Incoherence)

Trong các hệ thống đa vi xử lý, việc mỗi lõi sở hữu Cache L1 riêng dẫn đến nguy cơ dữ liệu tại các lõi trở nên lỗi thời (stale) khi một lõi thay đổi giá trị cục bộ, từ đó gây sai lệch kết quả thực thi toàn hệ thống [7]. Để giải quyết bài toán này, phần

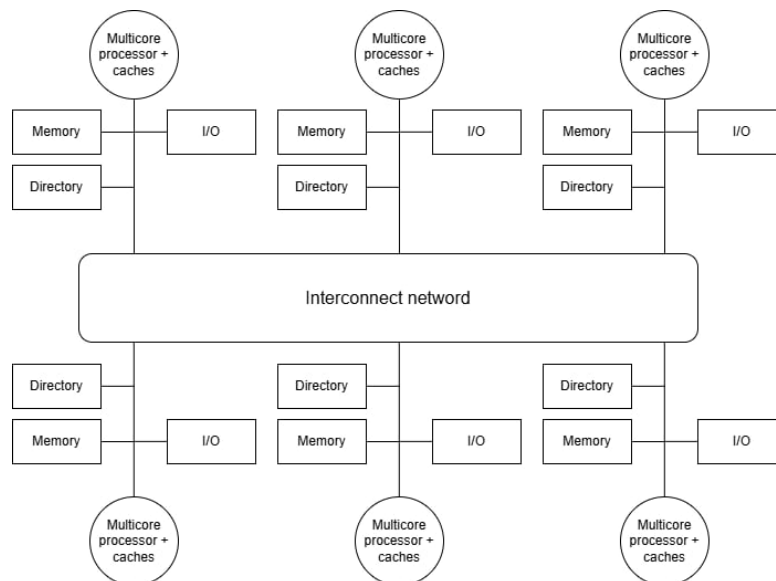
cứng điều khiển bộ nhớ bắt buộc phải tuân thủ hai điều kiện bất biến cốt lõi [10]. Thứ nhất là bất biến SWMR (Single-Writer, Multiple-Reader), quy định tại một thời điểm chỉ cho phép duy nhất một lõi ghi (trạng thái Modified/Exclusive) hoặc nhiều lõi đọc đồng thời (trạng thái Shared). Thứ hai là Tính toàn vẹn giá trị (Data Value Invariant), yêu cầu lệnh đọc phải luôn trả về giá trị mới nhất từ lệnh ghi liền kề trước đó, đảm bảo tính nhất quán dữ liệu xuyên suốt các tầng bộ nhớ.

2.4.2. Các trường phái kiến trúc giải quyết Cache Coherence

Để duy trì tính nhất quán dữ liệu cần sử dụng hai nhóm giao thức phần cứng chính: Directory-based (Dựa trên thư mục) và Snooping-based (Giám sát Bus) [7].

a. Giao thức dựa trên Thư mục (Directory-based Protocol)

Trong giao thức dựa trên Thư mục (Directory-based Protocol), hệ thống sử dụng một cấu trúc dữ liệu (Directory) tại bộ nhớ chung để lưu trữ trạng thái và định danh các lõi đang giữ bản sao dữ liệu. Về cơ chế vận hành, khi có yêu cầu truy cập, Thư mục đóng vai trò trung gian, trực tiếp gửi các lệnh điểm-đến-điểm (point-to-point) để thu hồi hoặc vô hiệu hóa dữ liệu đến đích danh các lõi liên quan [10]. Về đặc tính phần cứng, kiến trúc này không phát tán thông điệp dư thừa, có khả năng mở rộng cực tốt cho hệ thống lớn, tuy nhiên lại yêu cầu tài nguyên lưu trữ lớn và có độ trễ định tuyến cao [10]. Sơ đồ vận hành tổng quát của giao thức này được mô tả tại Hình 2.3.



Hình 2.3: Sơ đồ tổng quát của giao thức Thư mục

b. Giao thức Giám sát Bus (Snooping-based Protocol)

Trong các hệ thống sử dụng mạng kết nối chung có tính phát thanh (Broadcast Medium) như Bus dùng chung, Cache Controller vận hành bằng cách giám sát (snoop) mọi giao dịch trên Bus; khi phát hiện lệnh ghi từ lõi khác, các mạch snoop sẽ tự động cập nhật hoặc vô hiệu hóa bản sao cục bộ để đảm bảo tính nhất quán [7]. Cơ chế này sở hữu logic đơn giản và độ trễ thấp, song lại bộc lộ hạn chế về khả năng mở rộng do dễ gây nghẽn băng thông Bus khi số lượng lõi tăng lên [9]. Các đặc tính kỹ thuật cơ bản của phương thức này được tổng quát hóa tại Bảng 2.2.

Bảng 2.2: So sánh đặc tính kỹ thuật giữa kiến trúc Snooping và Directory

Tiêu chí	Kiến trúc Giám sát (Snooping)	Kiến trúc Thư mục (Directory-based)
Mạng kết nối	Mạng phát thanh (Shared Bus)	Mạng định tuyến điểm-đến-điểm
Băng thông nội bộ	Tiêu tốn cao (do cơ chế Broadcast)	Thấp (chỉ gửi thông điệp chọn lọc)
Khả năng mở rộng	Hạn chế (Tối ưu cho số lõi nhỏ)	Rất cao (Hệ thống hàng chục/trăm lõi)
Độ trễ (Latency)	Tối ưu (Xử lý trực tiếp trên Bus chung)	Cao (Phải qua cấu trúc trung gian)

Quyết định thiết kế của khóa luận: Khóa luận lựa chọn giao thức Snooping-based để giải quyết bài toán Cache Coherence. Đối với quy mô hệ thống lõi kép (Dual-Core) kết nối L2 Cache dùng chung, mạng Bus phát thanh là cấu hình tối ưu nhất. Lựa chọn này giúp hệ thống đạt độ trễ giao dịch thấp, đồng thời tối giản logic điều khiển và tiết kiệm tài nguyên phần cứng trên FPGA [13].

2.5. Các biến thể giao thức Snooping

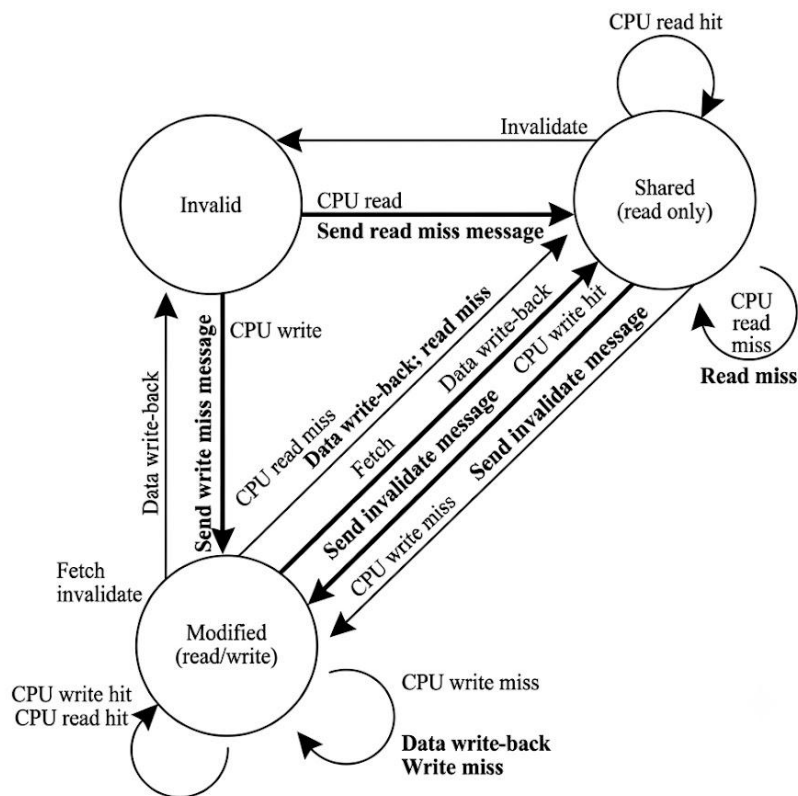
Để hiện thực hóa cơ chế giám sát Bus, trạng thái các dòng dữ liệu được quản lý qua các máy trạng thái hữu hạn (FSM). Các cấu hình phổ biến từ cơ bản đến nâng cao bao gồm [9], [12].

2.5.1. Giao thức MSI (Cấu hình cơ bản)

Giao thức MSI duy trì 3 trạng thái cơ bản cho mỗi dòng bộ đệm:

- **Modified (M):** Dữ liệu là duy nhất trong toàn hệ thống và đã bị chỉnh sửa so với bộ nhớ chính.
- **Shared (S):** Dữ liệu sạch (Clean), có thể tồn tại đồng thời trên nhiều lõi, chỉ cấp quyền đọc.
- **Invalid (I):** Dữ liệu không hợp lệ hoặc dòng cache đang trống.

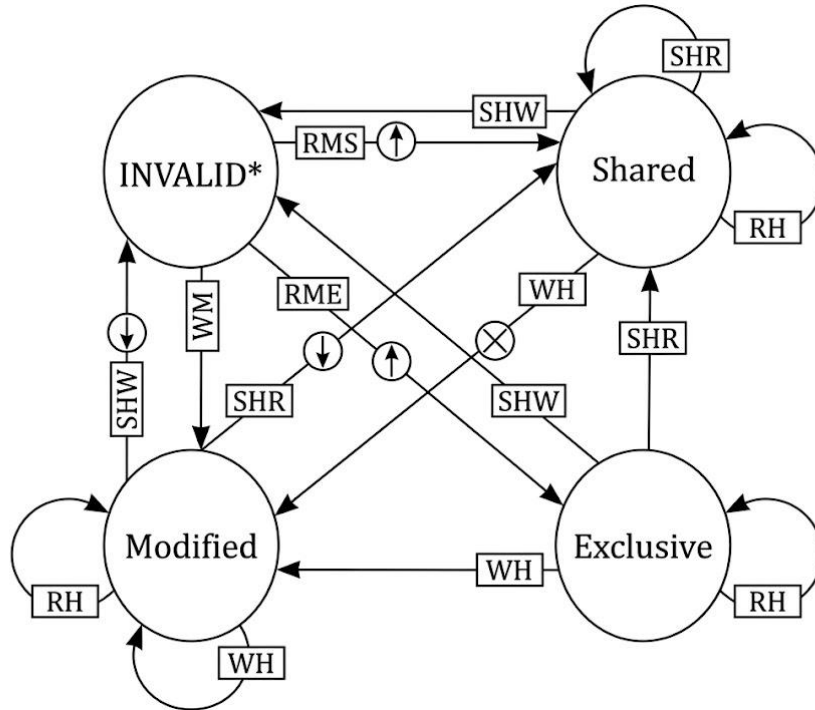
Nguyên lý chuyển đổi trạng thái của giao thức MSI tại Hình 2.4.



Hình 2.4: Biểu đồ chuyển đổi trạng thái của giao thức MSI

2.5.2. Giao thức MESI (Cải tiến quyền độc quyền)

Giao thức MESI bổ sung trạng thái thứ tư nhằm khắc phục sự lãng phí băng thông của MSI: Exclusive (E) – Dữ liệu sạch (đồng bộ với bộ nhớ chính) và chỉ tồn tại duy nhất tại bộ nhớ đệm của lõi hiện tại [13], [16]. Ưu điểm của kiến trúc này là khi ghi vào dòng dữ liệu ở trạng thái E, hệ thống chuyển thẳng sang trạng thái M thông qua "Silent transition" mà không cần phát thông điệp lên Bus, giúp tối ưu hóa năng lượng và băng thông. Tuy nhiên, nó tồn tại nhược điểm (Write-back Overhead) là khi có lõi khác yêu cầu đọc dữ liệu đang ở trạng thái M, hệ thống bắt buộc phải ghi trả (Write-back) xuống bộ nhớ chính trước khi chuyển về S, gây trễ và lãng phí băng thông [9]. Sơ đồ vận hành trạng thái của giao thức MESI được minh họa tại Hình 2.5.

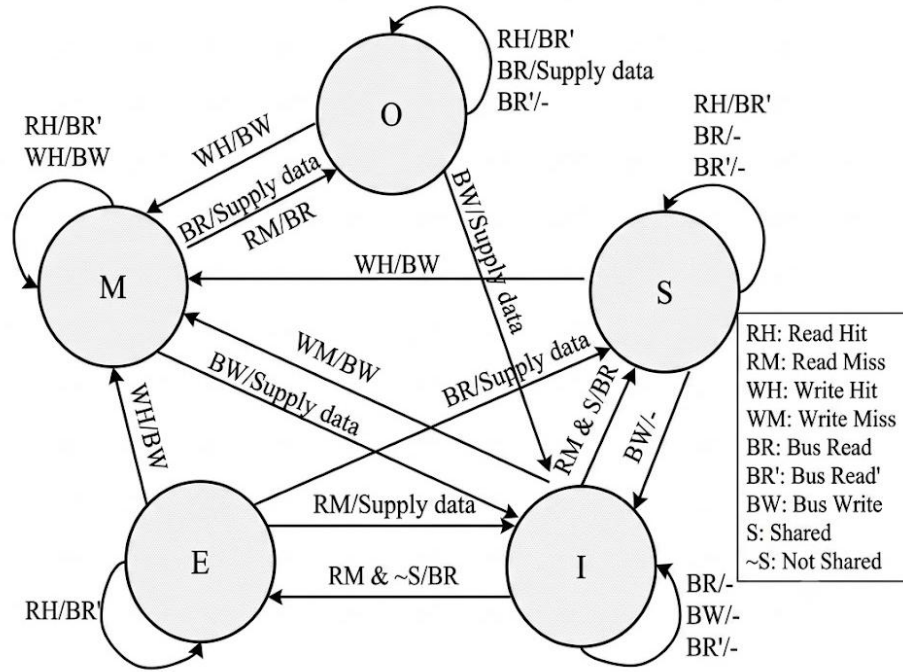


Hình 2.5: Sơ đồ trạng thái giao thức MESI

2.5.3. Giao thức MOESI (Tối ưu hóa chia sẻ khối dữ liệu bẩn)

Giao thức MOESI giải quyết triệt để bài toán lãng phí băng thông (Overhead Write-back) của MESI bằng cách tích hợp trạng thái thứ năm: Owned (O) [13]. Trạng thái Owned đánh dấu khối dữ liệu đã bị chỉnh sửa (Dirty) nhưng vẫn được chia sẻ cho các lõi khác, trong đó lõi nắm giữ trạng thái O chịu trách nhiệm chính cho việc ghi trả (Write-back) dữ liệu xuống bộ nhớ chính khi dòng cache bị trục xuất. Cơ chế này

tối ưu phần cứng bằng cách hỗ trợ truyền dữ liệu trực tiếp giữa các L1 Cache (Cache-to-Cache transfer), nghĩa là lõi giữ trạng thái O sẽ cung cấp dữ liệu trực tiếp cho yêu cầu đọc mà không cần chu kỳ Write-back xuống bộ nhớ chính, giúp tối ưu hóa tối đa băng thông Bus hệ thống [13]. Nguyên lý chuyển đổi trạng thái của giao thức này được mô tả tại Hình 2.6.



Hình 2.6: Biểu đồ chuyển đổi trạng thái của giao thức MOESI

2.6. Sơ lược về kiến trúc tập lệnh RISC-V RV32IA

2.6.1. Tổng quan

Kiến trúc mã nguồn mở RISC-V cho phép tùy biến linh hoạt cấu hình vi xử lý, trong đó cấu trúc RV32IA được lựa chọn để đáp ứng yêu cầu của hệ thống đa lõi [18]. Việc tích hợp tập lệnh này không chỉ giúp tối ưu hóa khả năng xử lý song song các tác vụ phức tạp mà còn đảm bảo tính đồng bộ hóa cao giữa các đơn vị xử lý trong hệ thống. Cấu trúc này bao gồm tập lệnh cơ sở RV32I vận hành theo mô hình Load-Store [7] kết hợp cùng tập lệnh mở rộng nguyên tử "A" giúp đồng bộ hóa, tránh xung đột dữ liệu [18] thông qua cơ chế đặt chỗ LR/SC và thao tác đọc-sửa-ghi độc quyền AMO. Chi tiết phân cấp các tập lệnh tiêu chuẩn của RISC-V tại Bảng 2.3 [18].

Bảng 2.3: Phân loại các tập lệnh cơ sở của kiến trúc RISC-V

Phân loại	Ký hiệu	Tên đầy đủ tập lệnh	Ý nghĩa chức năng
Tập lệnh cơ sở	RV32I	Base Integer Instruction Set, 32-bit	Nền tảng số nguyên cơ bản, 32 thanh ghi
	RV32E	Base Integer Instruction Set (Embedded)	Phiên bản rút gọn cho hệ nhúng (16 thanh ghi)
	RV64I	Base Integer Instruction Set, 64-bit	Nền tảng số nguyên không gian địa chỉ 64-bit
Tập lệnh mở rộng	A	Standard Extension for Atomic Instructions	Các lệnh nguyên tử hỗ trợ đồng bộ đa lõi
	M	Standard Extension for Integer Multiplication/Division	Hỗ trợ tính toán nhân/chia số nguyên bằng phần cứng
	F	Standard Extension for Single-Precision Floating-Point	Hỗ trợ số thực dấu phẩy động đơn (32-bit)
	C	Standard Extension for Compressed Instructions	Tập lệnh nén kích thước 16-bit tối ưu bộ nhớ

2.6.2. Kiến trúc tập lệnh cơ sở RV32I

Tập lệnh RV32I có độ rộng cố định 32-bit và được căn lề bộ nhớ, giúp tối giản hóa vi kiến trúc của mạch giải mã [7]. Tập lệnh được chia thành 6 định dạng tiêu chuẩn, với cấu trúc bit được minh họa chi tiết tại Hình 2.7 [18].

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Hình 2.7: Định dạng mã hóa các nhóm lệnh trong kiến trúc RISC-V

Để tối ưu hóa quá trình giải mã lệnh trong phần cứng, các lệnh trong kiến trúc RISC-V được chia thành 6 loại định dạng chính, cụ thể bao gồm:

- R-type (Register): Thực hiện các phép toán số học/logic cơ bản giữa các thanh ghi.
- I-type (Immediate): Phép toán với hằng số tức thời, dùng cho lệnh nạp (Load) hoặc nhảy gián tiếp.
- S-type (Store): Lệnh lưu trữ. Trạng thái hằng số được chia tách để cố định vị trí thanh ghi nguồn, giúp tối ưu mạch dồn kênh phần cứng.
- B-type (Branch): Rẽ nhánh có điều kiện (địa chỉ tương đối theo giá trị của thanh ghi PC).
- U-type (Upper Immediate): Nạp hằng số lớn (20-bit) vào dải bit cao của thanh ghi đích.
- J-type (Jump): Nhảy không điều kiện, hỗ trợ tính toán địa chỉ nhảy xa.

a. Tổ chức tài nguyên và Mô hình thực thi

Tài nguyên: Lỗi vi xử lý tích hợp trực tiếp thanh ghi PC và tập 32 thanh ghi 32-bit (x_0 – x_{31}). Đặc biệt, thanh ghi x_0 được nối cứng mức 0 (Hardwired Zero), hỗ trợ tối ưu các lệnh giả (pseudo-instructions) mà không làm tăng độ phức tạp mạch giải mã phần cứng [7]. Mô hình thực thi: Hệ thống tuân thủ chặt chẽ theo cơ chế của kiến trúc Load-Store để tối ưu hiệu suất hoạt động. Bộ ALU chỉ thực hiện các phép tính toán trên tập thanh ghi nội bộ, đảm bảo tính nhất quán và tốc độ xử lý dữ liệu cao; mọi thao tác truy xuất dữ liệu từ bộ nhớ ngoài bắt buộc phải dùng lệnh Load/Store riêng biệt. Kiến trúc này hỗ trợ định địa chỉ theo byte và lưu trữ dữ liệu theo định dạng Little-endian [18].

b. Tập lệnh mở rộng Nguyên tử "A" (Atomic Extension)

Tập lệnh "A" hỗ trợ phần cứng cho các thao tác bộ nhớ nguyên tử (indivisible), là cơ sở kiến trúc bắt buộc để đồng bộ hóa và ngăn chặn xung đột dữ liệu (Race Condition) trong hệ thống đa lõi [7]. Tập lệnh vận hành qua hai cơ chế cốt lõi [18].

c. Cơ chế (Load - Reserved / Store - Conditional - LR/SC)

Để thực thi an toàn chuỗi tác vụ Đọc - Sửa - Ghi trên vùng nhớ chia sẻ, hệ thống sử dụng cặp lệnh lr.w và sc.w. Trong đó, lệnh lr.w (Load-Reserved) thực hiện nạp dữ liệu từ bộ nhớ vào thanh ghi, đồng thời kích hoạt trạng thái giám sát địa chỉ (Reservation Set) trên phần cứng. Tiếp theo, lệnh sc.w (Store-Conditional) tiến hành ghi giá trị mới vào địa chỉ đã đặt trước. Lệnh ghi này chỉ thành công (trả về giá trị 0) nếu trạng thái đặt trước chưa bị vô hiệu hóa bởi tác vụ ghi của một lõi khác; trường hợp thất bại (trả về giá trị khác 0), phần mềm buộc phải lặp lại toàn bộ chu kỳ [18].

d. Tác vụ bộ nhớ nguyên tử (AMO)

Nhóm lệnh AMO thực thi chuỗi thao tác liên hoàn: Đọc dữ liệu → Xử lý tính toán (ALU) → Ghi kết quả mới về bộ nhớ. Toàn bộ tiến trình này được mạch điều khiển khóa độc quyền trên hệ thống Bus, đảm bảo tính nguyên tử tuyệt đối và không bị ngắt quãng [7]. Cấu trúc định dạng của nhóm tập lệnh mở rộng này được minh họa chi tiết tại Hình 2.8 [18].

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct5		aq	rl	rs2		rs1		funct3		rd			opcode	
5		1	1	5		5		3		5			7	

Inst	Name	FMT	Opcode	funct3	funct5	Description (C)
lr.w	Load Reserved	R	0101111	0x2	0x02	rd = M[rs1]; reserve M[rs1]
sc.w	Store Conditional	R	0101111	0x2	0x03	if (reserved) { M[rs1] = rs2; rd = 0 } else { rd = 1 }
amoswap.w	Atomic Swap	R	0101111	0x2	0x01	rd = M[rs1]; swap(rd, rs2); M[rs1] = rd
amoadd.w	Atomic ADD	R	0101111	0x2	0x00	rd = M[rs1] + rs2; M[rs1] = rd
amoand.w	Atomic AND	R	0101111	0x2	0x0C	rd = M[rs1] & rs2; M[rs1] = rd
amoor.w	Atomic OR	R	0101111	0x2	0x0A	rd = M[rs1] rs2; M[rs1] = rd
amoxor.w	Atomix XOR	R	0101111	0x2	0x04	rd = M[rs1] ^ rs2; M[rs1] = rd
amomax.w	Atomic MAX	R	0101111	0x2	0x14	rd = max(M[rs1], rs2); M[rs1] = rd
amomin.w	Atomic MIN	R	0101111	0x2	0x10	rd = min(M[rs1], rs2); M[rs1] = rd

Hình 2.8: Cấu trúc tập lệnh mở rộng RV32A

2.7. Tổng quan về giao thức AXI

2.7.1. Giới thiệu

Giao thức AXI thuộc tiêu chuẩn AMBA do ARM phát triển là giao thức giao tiếp hiệu năng cao, sử dụng các kênh đọc/ghi độc lập để loại bỏ điểm nghẽn bus, tối ưu băng thông và giảm độ trễ [6]. Giao thức này được phân thành ba biến thể chính: AXI-Full hỗ trợ truyền dữ liệu dạng mảng (burst-based) với băng thông cao [6], AXI-Lite dành cho giao tiếp thanh ghi nhỏ không hỗ trợ burst [6], và AXI-Stream chuyên truyền dữ liệu luồng tốc độ cao không định tuyến qua địa chỉ. Trong phạm vi khóa luận, hệ thống lựa chọn tích hợp chuẩn AXI4-Full nhằm khai thác các giao dịch burst, giúp tối ưu hóa tốc độ truyền tải các khối dữ liệu lớn giữa bộ nhớ đệm và bộ nhớ chính, từ đó giảm thiểu tối đa độ trễ.

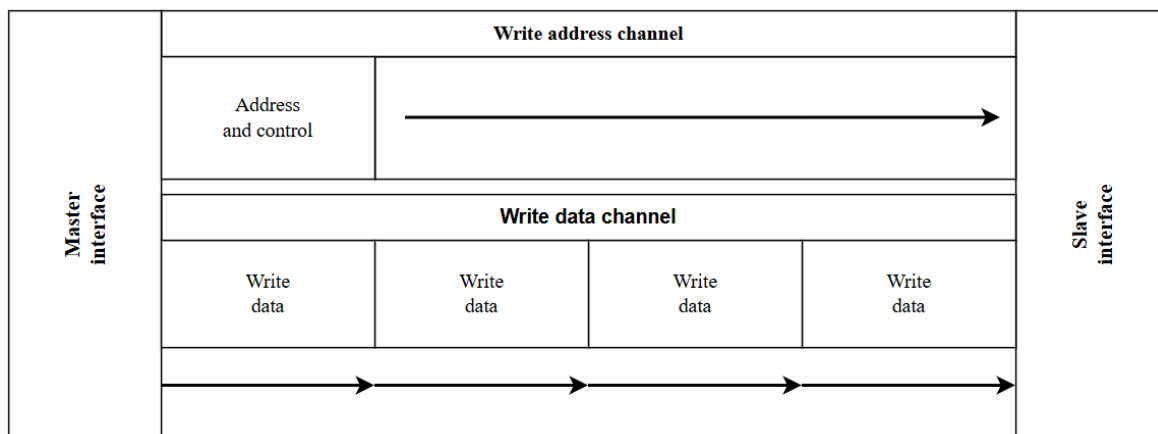
2.7.2. Kiến trúc giao thức AXI

Cơ chế truyền tải được thực hiện thông qua 5 kênh giao tiếp độc lập giữa Master và Slave [6], bao gồm: kênh địa chỉ đọc (tiền tố AR) và kênh địa chỉ ghi (AW) mang các tín hiệu điều khiển cùng địa chỉ tương ứng; kênh dữ liệu đọc (R) và kênh dữ liệu ghi (W) đảm nhận việc truyền tải dữ liệu hai chiều giữa Master và Slave; cùng kênh phản hồi ghi (B) để Slave trả về trạng thái xác nhận giao dịch ghi. Nhờ thiết kế này, kiến trúc sở hữu ba đặc tính cốt lõi giúp tối đa hóa hiệu suất Bus, bao gồm: khả năng phát hành thông tin địa chỉ trước dữ liệu, hỗ trợ đa giao dịch đang chờ (outstanding) và cho phép hoàn thành giao dịch không tuần tự (out-of-order) [6].

2.7.3. Mô tả các quá trình giao dịch

a. Quá trình Ghi (Write Transaction)

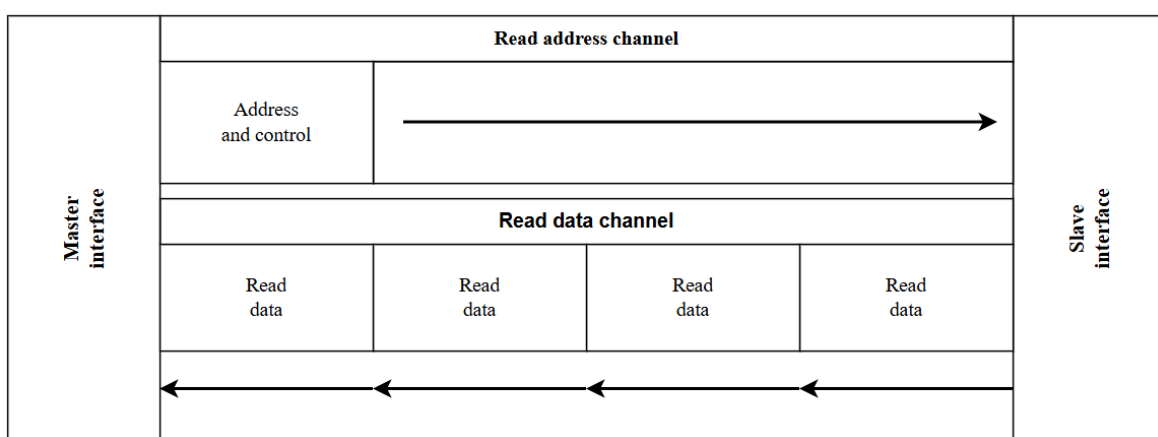
Giao dịch ghi trên chuẩn AXI vận hành qua 3 kênh chính: Master gửi địa chỉ và tín hiệu điều khiển (AW), truyền luồng dữ liệu (W), và Slave gửi tín hiệu xác nhận hoàn tất (B) [6]. Luồng tín hiệu của các giao dịch này được mô tả tại Hình 2.9.



Hình 2.9: Sơ đồ luồng tín hiệu giao dịch ghi trên chuẩn AXI

b. Quá trình Đọc (Read Transaction)

Giao dịch đọc trên chuẩn AXI vận hành qua 2 kênh chính: Master phát tín hiệu yêu cầu địa chỉ (AR), Slave thực hiện truy xuất và truyền trả luồng dữ liệu (R) cho Master [6]. Luồng tín hiệu chi tiết của các giao dịch đọc được mô tả tại Hình 2.10.

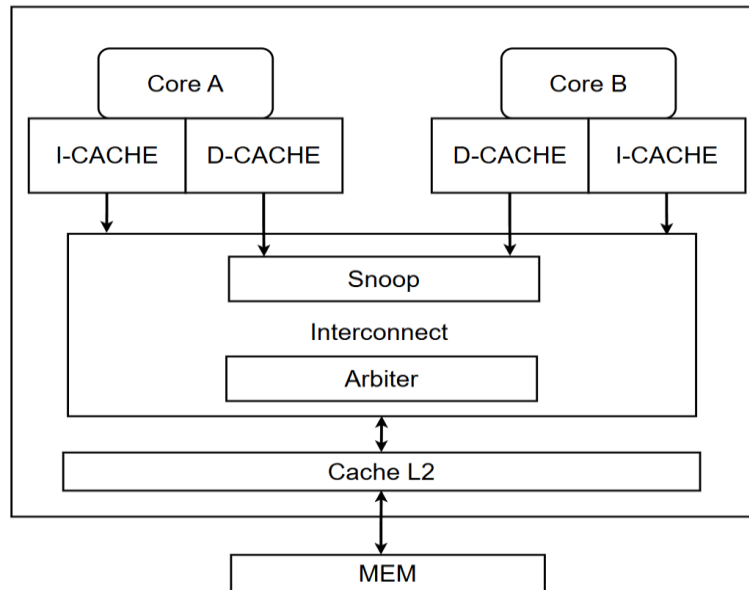


Hình 2.10: Sơ đồ luồng tín hiệu giao dịch đọc trên chuẩn AXI

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

3.1. Tổng quan hệ thống

Hệ thống được thiết kế là một vi xử lý lõi kép dựa trên kiến trúc tập lệnh RISC-V RV32IA. Toàn bộ hệ thống được tổ chức thành ba tầng phân cấp rõ ràng: tầng lõi xử lý, tầng bộ nhớ đệm phân cấp và tầng kết nối dữ liệu. Kiến trúc tổng thể của hệ thống lõi kép với cơ chế đồng bộ dữ liệu được mô tả chi tiết tại Hình 3.1.



Hình 3.1: Sơ đồ hệ thống 2 lõi sử dụng Cache Coherence

Tầng lõi xử lý: Gồm 2 lõi RV32IA độc lập, mỗi lõi được xây dựng theo kiến trúc pipeline 6 tầng với bộ dự đoán rẽ nhánh động và cơ chế xử lý xung đột dữ liệu. Tầng bộ nhớ đệm phân cấp: Mỗi lõi trang bị L1 I-Cache và L1 D-Cache độc lập (16-Set). Hai lõi chia sẻ chung một L2 Cache (64-Set). Tính nhất quán dữ liệu được đảm bảo bởi giao thức MOESI Snoopy tích hợp tại L1 D-Cache, cùng với khối AMO ALU xử lý trực tiếp các lệnh nguyên tử. Tầng kết nối dữ liệu: Giao tiếp qua chuẩn bus AXI4 Full sử dụng cơ chế phân xử phân cấp (Hierarchical Arbitration), điều phối luồng lệnh và dữ liệu từ hai lõi xuống L2 Cache và bộ nhớ chính.

Khi phát sinh yêu cầu truy cập bộ nhớ, bộ điều khiển D-Cache sẽ kiểm tra trạng thái nội bộ (Hit/Miss). Nếu là Cache Hit và dữ liệu hợp lệ, yêu cầu được xử lý ngay tại chỗ. Ngược lại, khi xảy ra Cache Miss hoặc cần chuyển trạng thái theo giao thức MOESI, bộ điều khiển sẽ phát tín hiệu snoop lên Bus để thông báo lõi đối diện, đồng

thời gửi yêu cầu lên L2 Cache hoặc bộ nhớ chính thông qua mạng phân xử. Các thông số kỹ thuật cho cấu hình phân cấp bộ nhớ đệm này được tổng hợp tại Bảng 3.1.

Bảng 3.1: Thông số cấu hình phân cấp bộ nhớ đệm

Thành phần	L1 I-Cache	L1 D-Cache	L2 Cache
Kiến trúc	4-Way Set Associative	4-Way Set Associative	4-Way Set Associative
Số Set	16-Set	16-Set	64-Set
Cache Line Size	64 byte	64 byte	64 byte
Dung lượng	4 KB	4 KB	16 KB
Giải thuật thay thế	PLRUt 3-bit	PLRUt 3-bit	PLRUt 3-bit
Chính sách ghi	Read-only (Fetch)	Write-back + R/W Allocate	Write-back + R/W Allocate
Cache Coherence	Không	MOESI Snoopy	Không
Hỗ trợ Atomic	Không	AMO ALU + LR/SC	Không

3.2. Thiết kế hệ thống phần cứng

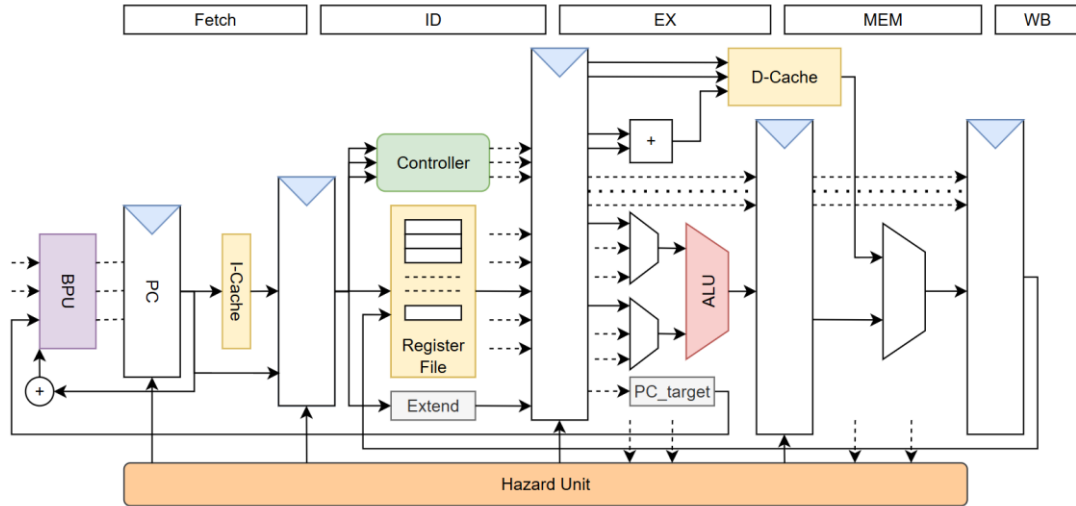
3.2.1. Thiết kế phần cứng lõi xử lý RISC-V RV32I

Trong vi xử lý RISC-V truyền thống, kiến trúc pipeline 5 tầng thường gặp phải điểm nghẽn về tần số hoạt động (Fmax) do đường tới hạn (critical path) nằm ở chu trình đọc mảng SRAM của bộ nhớ đệm lệnh (I-Cache). Để khắc phục hạn chế này, lõi xử lý RV32IA được thiết kế với kiến trúc pipeline 6 tầng phân cấp vật lý. Cải tiến cốt lõi của kiến trúc này là việc phân rã phân đoạn nạp lệnh (Fetch) thành hai tầng chuyên biệt (IF và S2) nhằm cô lập mảng SRAM lệnh, triệt tiêu thời gian trễ thiết lập (setup time) và tối ưu hóa tần số clock chung. Chức năng cụ thể của từng tầng được tóm tắt trong Bảng 3.2.

Bảng 3.2: Mô tả 6 tầng trong kiến trúc pipeline lõi RV32IA

Tầng	Tên tầng	Chức năng
1	IF (Instruction Fetch - S1)	Quản lý con trỏ, tích hợp bộ dự đoán rẽ nhánh động để cung cấp sớm địa chỉ mục tiêu, tối ưu luồng nạp chỉ thị và phát yêu cầu truy xuất sang L1 I-Cache
2	S2 (ICache Response)	Tiếp nhận mã máy phản hồi từ L1 I-Cache, giúp tăng đệm che giấu độ trễ đọc của mảng SRAM lệnh và thực hiện hủy luồng lệnh lỗi nếu phát hiện dự đoán sai rẽ nhánh
3	ID (Instruction Decode)	Giải mã chỉ thị để phát sinh hệ thống tín hiệu điều khiển (cho tập lệnh RV32IA), đọc toán hạng từ tập thanh ghi và phối hợp kiểm soát rủi ro dữ liệu (Data Hazard)
4	EX (Execute)	Thực thi các phép toán đại số/logic qua ALU, tính địa chỉ nhảy và đánh giá điều kiện rẽ nhánh thực tế. Khởi chuyển tiếp dữ liệu (Forwarding) xử lý nhanh toán hạng. Đồng thời phát toàn bộ thông tin truy xuất bộ nhớ sang L1 D-Cache (tương ứng giai đoạn Access của bộ đệm)
5	MEM (Memory Access)	Tiếp nhận dữ liệu phản hồi từ L1 D-Cache, kích hoạt dừng pipeline (stall) nếu xảy ra Cache Miss. Thực hiện mạch căn chỉnh/mở rộng định dạng dữ liệu đọc (byte/nửa từ mã), hoàn tất chu trình đọc-sửa-ghi nguyên tử và đối chiếu kết quả rẽ nhánh thực tế để sửa sai PC
6	WB (Write Back)	Lựa chọn nguồn dữ liệu đích cuối cùng (từ ALU hoặc bộ nhớ) để ghi trả kết quả về tập thanh ghi. Đồng thời cập nhật trạng thái thành công/thất bại của các tác vụ kiểm tra điều kiện ghi nguyên tử (Store - Conditional)

Dựa trên cấu trúc phân định chức năng nêu trên, sơ đồ khối thể hiện sự liên kết tín hiệu và đường đi của dữ liệu qua các tầng pipeline vật lý của lõi xử lý được mô tả chi tiết trong Hình 3.2.



Hình 3.2: Kiến trúc tổng quát lõi vi xử lý RISC-V RV32I

3.2.2. Thiết kế phần cứng bộ nhớ đệm

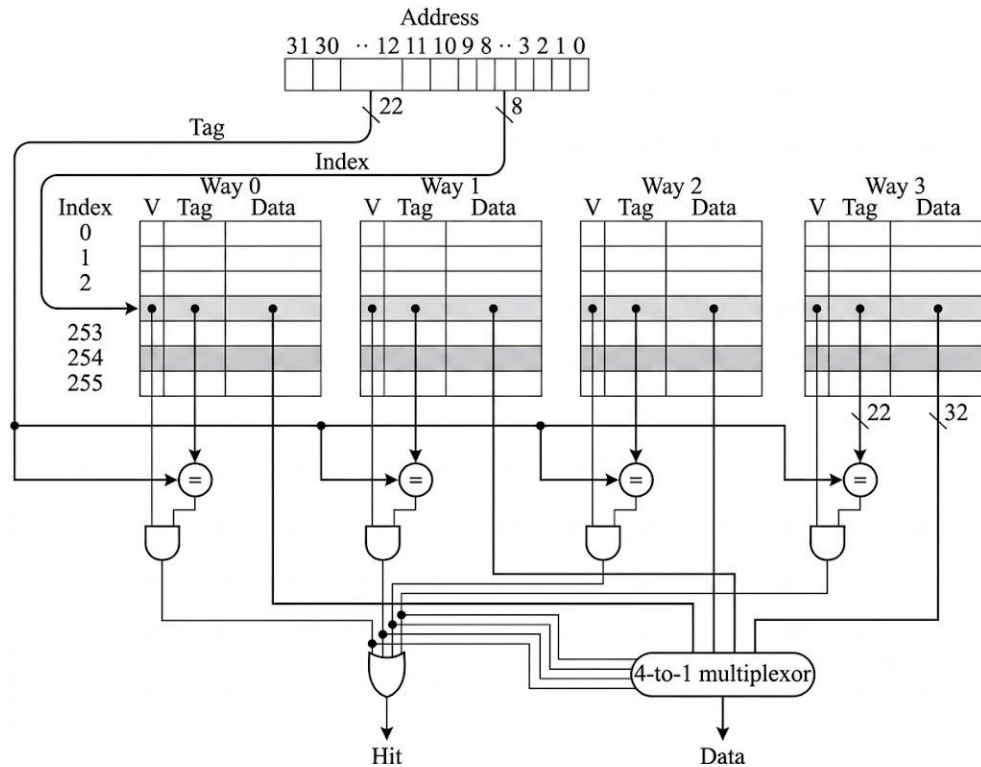
Hệ thống bộ nhớ đệm của vi xử lý đa lõi được thiết kế theo cấu trúc phân cấp hai tầng (L1 và L2) nhằm tối ưu hóa sự đánh đổi giữa độ trễ truy xuất (latency) và tỷ lệ đánh trúng (hit rate). Thông số kỹ thuật và cấu hình chi tiết của từng mô-đun trong hệ thống phân cấp được tổng hợp trong Bảng 3.3.

Bảng 3.3: Thông số cấu hình hệ thống phân cấp bộ nhớ đệm

Thành phần	Số Set	Số Way	Cache Line	Dung lượng	Ghi chú
L1 I-Cache	16	4	64 byte	4 KB × 2 lõi	Riêng từng lõi, chỉ đọc
L1 D-Cache	16	4	64 byte	4 KB × 2 lõi	Riêng từng lõi, MOESI Snoopy
L2 Cache	64	4	64 byte	16 KB	Dùng chung, không Coherence

a. Kiến trúc bộ nhớ đệm cơ sở

Mặc dù hệ thống phân cấp bao gồm ba mô-đun với chức năng đặc thù (L1 I-Cache, L1 D-Cache và L2 Cache), cả ba thành phần này đều được xây dựng trên một nền tảng vi kiến trúc cơ sở chung với cấu trúc liên kết tập hợp 4 đường (4-Way Set Associative). Sơ đồ khối của kiến trúc này được trình bày tại Hình 3.3.

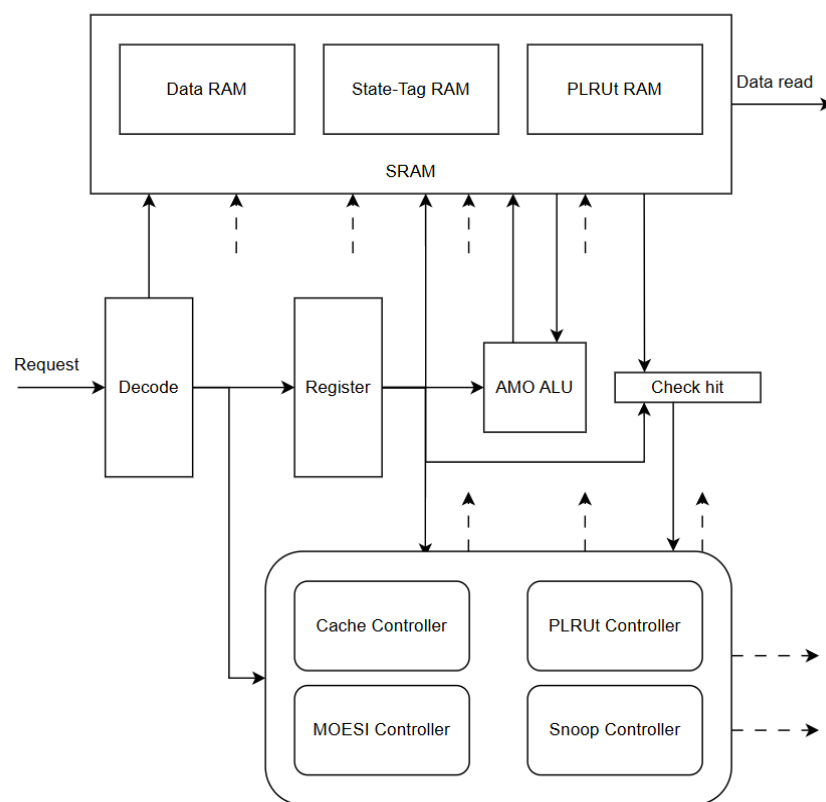


Hình 3.3: Sơ đồ khối kiến trúc Cache 4-Way Set Associative

Dựa trên cấu trúc này, nền tảng vận hành thông qua ba phân hệ chính: cơ chế giải mã phân rã địa chỉ vật lý 32-bit thành các trường Byte/Word Offset, Index và Tag; tổ chức các mảng nhớ SRAM độc lập (Data RAM, State-Tag RAM lưu trạng thái MOESI và mảng PIM) cho phép truy xuất song song; và thuật toán thay thế Pseudo-LRU (PLRUt). Sự kết hợp chặt chẽ giữa ba phân hệ này giúp tối ưu hóa đáng kể độ trễ truy xuất cho hệ thống. Bằng cách ứng dụng mô hình duyệt cây nhị phân, giải thuật PLRUt giúp tối ưu chi phí phần cứng so với LRU truyền thống, cho phép hệ thống xác định nhanh chóng khối dữ liệu ít được sử dụng nhất (victim block) để ghi đè khi cache bị đầy. Điều này giúp hệ thống luôn duy trì hiệu suất hoạt động ổn định trong mọi điều kiện xử lý.

b. Vi kiến trúc L1 D-Cache (Thành phần cốt lõi)

Kiến trúc bên trong của L1 D-Cache được cấu thành từ 3 mảng lưu trữ vật lý (SRAM) và 5 khối điều khiển logic hoạt động đồng bộ chặt chẽ nhằm nâng cao hiệu suất hoạt động của toàn bộ hệ thống. Điểm khác biệt cốt lõi so với các thiết kế bộ nhớ đệm đơn lõi truyền thống là sự hiện diện của khối điều khiển MOESI, khối giám sát Snoop và bộ tính toán AMO nhằm hỗ trợ tính nhất quán dữ liệu cho hệ thống đa lõi. Sự tương tác và luồng điều khiển giữa các thành phần này được mô hình hóa trực quan trong Hình 3.4.



Hình 3.4: Sơ đồ khối vi kiến trúc L1 D-Cache

Như minh họa trên sơ đồ tổng thể, thiết kế áp dụng nguyên lý phân tách kiến trúc rõ ràng: các đường truyền dữ liệu và địa chỉ được cô lập hoàn toàn với mảng lưu trữ tĩnh, điều này góp phần đảm bảo tính toàn vẹn tín hiệu trong quá trình truyền, trong khi toàn bộ logic quyết định điều phối được tập trung vào nhóm các bộ điều khiển trung tâm. Chức năng cụ thể của từng khối thành phần phần cứng này được đặc tả chi tiết tại Bảng 3.4.

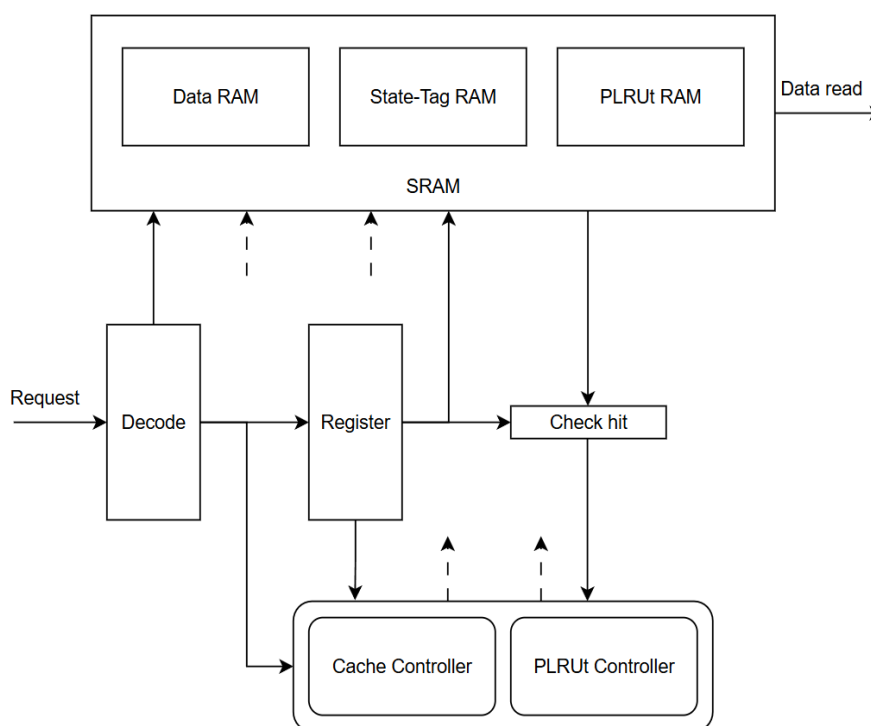
Bảng 3.4: Mô tả các khối chức năng vi kiến trúc L1 D-Cache

Khối chức năng	Mô tả chức năng
State-Tag RAM	Lưu trữ trường Tag địa chỉ vật lý và 3 bit mã hóa trạng thái MOESI cho mỗi Way trong mỗi Set
Data RAM	Mảng SRAM lưu trữ dữ liệu cache (64 byte mỗi Cache Line)
PLRUt RAM	Mảng nhớ PIM lưu trữ 3-bit lịch sử truy cập của mỗi Set, phục vụ trực tiếp cho thuật toán thay thế dòng bộ đệm
Cache Controller	Khối điều khiển trung tâm, điều phối toàn bộ hoạt động của D-Cache: xử lý hit/miss, stall pipeline, giao tiếp với khối phân xử hệ thống (Arbiter Interface), kiểm soát LR/SC (reservation tracker)
MOESI Controller	Thực thi logic chuyển trạng thái giao thức MOESI Snoopy. Tiếp nhận yêu cầu từ CPU cục bộ và tín hiệu snoop từ bus, phát tín hiệu điều khiển phù hợp (invalidate, supply, writeback)
Snoop Controller	Tiếp nhận và xử lý các yêu cầu giám sát từ mạng kết nối để theo dõi truy xuất từ lõi đối diện; phân tích địa chỉ và loại yêu cầu nhằm kích hoạt phản ứng snoop phù hợp (hit/miss, sinh tín hiệu invalidate)
PLRUt Controller	Thực thi giải thuật duyệt cây PLRUt 3-bit, xác định Way tối ưu để thay thế và tự động cập nhật vector lịch sử đối ngẫu sau mỗi lần truy cập

AMO ALU	Thực hiện chu trình đọc–sửa–ghi (read-modify-write) nội bộ cho các lệnh AMO (amoadd, amoswap, amoxor, amoand, amoor, amomin, amomax, amominu, amomaxu). Nhận toán hạng từ Cache Controller và ghi kết quả trực tiếp vào Data RAM
---------	--

c. Vi kiến trúc L1 I-Cache

Trái ngược với sự phức tạp của L1 D-Cache, L1 I-Cache được thiết kế tối giản nhằm mục đích cung cấp luồng chỉ thị liên tục và tốc độ cao cho tầng Nạp lệnh (Fetch). Do kiến trúc không hỗ trợ cơ chế tự sửa đổi mã (Self-modifying code), L1 I-Cache hoạt động ở chế độ chỉ đọc (Read-Only) đối với CPU. Sơ đồ cấu trúc khối của vi kiến trúc này được thể hiện tại Hình 3.5.



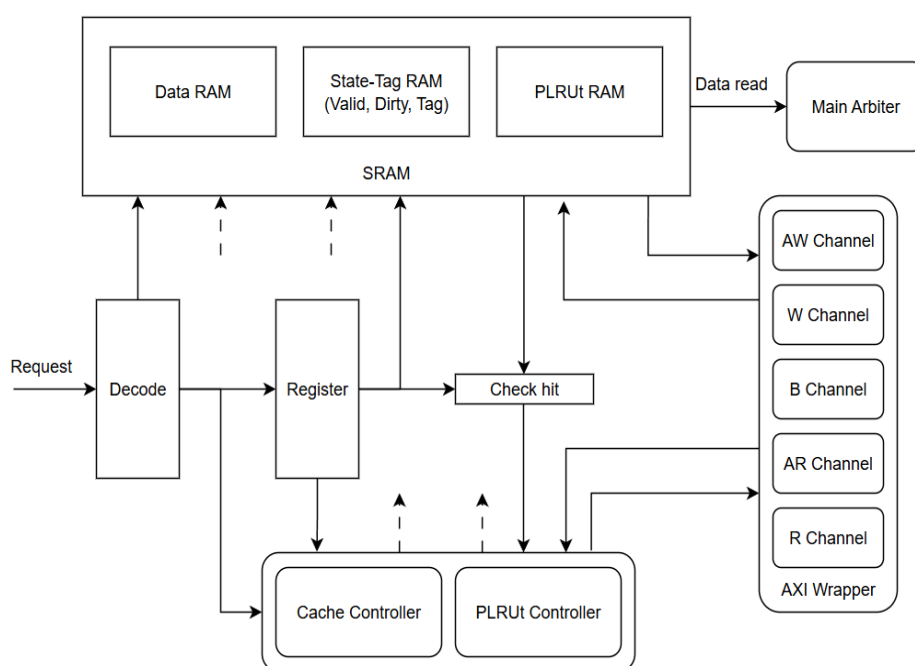
Hình 3.5: Sơ đồ khối vi kiến trúc L1 I-Cache

L1 I-Cache kế thừa nền tảng vi kiến trúc cơ sở (bao gồm tổ chức 4-Way Set Associative, cơ chế giải mã địa chỉ và thuật toán PLRUt) nhưng được tinh lược đáng kể các phân hệ không cần thiết. Cụ thể, thiết kế đã loại bỏ hoàn toàn logic đồng bộ

(gồm khối MOESI Controller, Snoop Controller và AMO ALU), khiến mảng trạng thái chỉ còn nhiệm vụ duy nhất là xác định tính hợp lệ của khối dữ liệu. Bên cạnh đó, luồng dữ liệu cũng được tối giản hóa khi mảng Data RAM chỉ thực hiện thao tác đọc; tác vụ ghi dữ liệu chỉ được kích hoạt trong chu trình nạp tràn (Refill 64 byte) từ L2 Cache khi xảy ra trượt bộ đệm (Cache Miss).

d. Vi kiến trúc L2 Cache và Giao tiếp AXI4 Full

Khác với các bộ nhớ đệm L1 phục vụ riêng rẽ cho từng lõi, L2 Cache đóng vai trò là điểm hội tụ dữ liệu dùng chung (Shared Cache) của toàn hệ thống trước khi kết nối trực tiếp với bộ nhớ chính. Kiến trúc của L2 Cache được tối ưu hóa theo hướng mở rộng dung lượng lưu trữ (16 KB) và tối đa hóa thông lượng truyền tải ngoại vi. Sơ đồ vi kiến trúc L2 Cache cùng cấu trúc các kênh truyền dữ liệu chuẩn AXI4 Full được mô tả chi tiết tại Hình 3.6.



Hình 3.6: Vi kiến trúc L2 Cache và giao tiếp AXI4 Full

Tối giản hóa mảng logic đồng bộ dữ liệu: Dựa trên nguyên tắc toàn bộ các vấn đề về nhất quán dữ liệu đều đã được xử lý triệt để tại bộ đệm L1, L2 Cache không tích hợp mạch giám sát bus và giao thức MOESI. Mảng lưu trữ trạng thái chỉ sử dụng các cờ cơ bản (Valid và Dirty) để quản lý tính hợp lệ của dữ liệu. Việc lược bỏ các logic

điều khiển phức tạp giúp tiết kiệm tài nguyên phần cứng, cho phép L2 Cache tập trung vào tốc độ phản hồi. Dù vậy, hệ thống vẫn tái sử dụng cấu trúc pipeline và thuật toán thay thế Pseudo-LRU để đảm bảo tính đồng nhất về thiết kế.

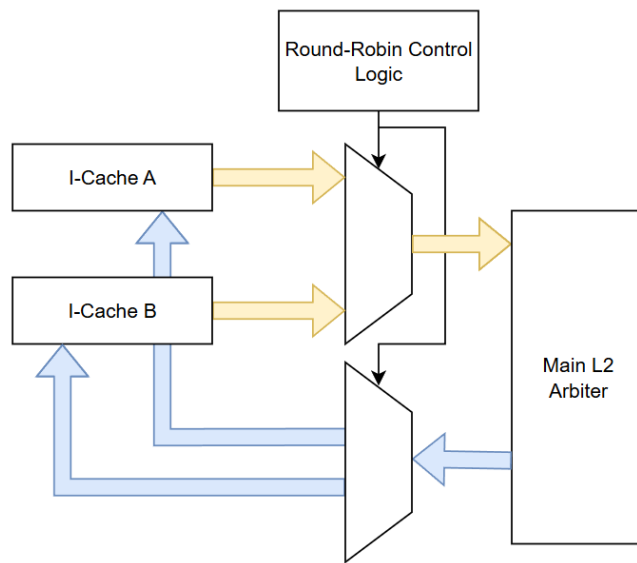
Giao tiếp bộ nhớ chính theo chuẩn AXI4 Full: Để đáp ứng yêu cầu truyền tải khối lượng dữ liệu lớn (Cache Line 64 byte) giữa L2 Cache và bộ nhớ chính, hệ thống tích hợp giao diện AXI4 Full. Nhằm giải quyết sự chênh lệch giữa kích thước Cache Line và độ rộng bus dữ liệu 32-bit mà không làm suy giảm hiệu năng, vi kiến trúc L2 Cache triển khai cơ chế truyền tải dạng Burst liên tục. Thay vì phải thiết lập lại địa chỉ liên tục cho từng từ mã, hệ thống tự động đóng gói các giao dịch đọc (Refill) hoặc ghi trả (Write-back) thành các chuỗi truyền tải liên tục, giúp tối ưu thông lượng và giảm thiểu tối đa độ trễ trên bus hệ thống.

3.2.3. Thiết kế mạng lưới phân xử và đồng bộ đa lõi

Trong kiến trúc đa lõi, hệ thống triển khai mạng lưới phân xử phân cấp (Hierarchical Arbitration) kết hợp cơ chế bộ lọc nội suy (Snoop Filter) nhằm giải quyết hiệu quả xung đột bus và duy trì tính nhất quán dữ liệu. Mạng lưới này đóng vai trò là trạm trung chuyển để điều phối toàn bộ luồng dữ liệu nội bộ, kết nối 4 bộ nhớ đệm L1 (2 I-Cache, 2 D-Cache) với L2 Cache dùng chung thông qua ba phân hệ chức năng độc lập.

a. Khối kết nối luồng lệnh (Instruction Interconnect)

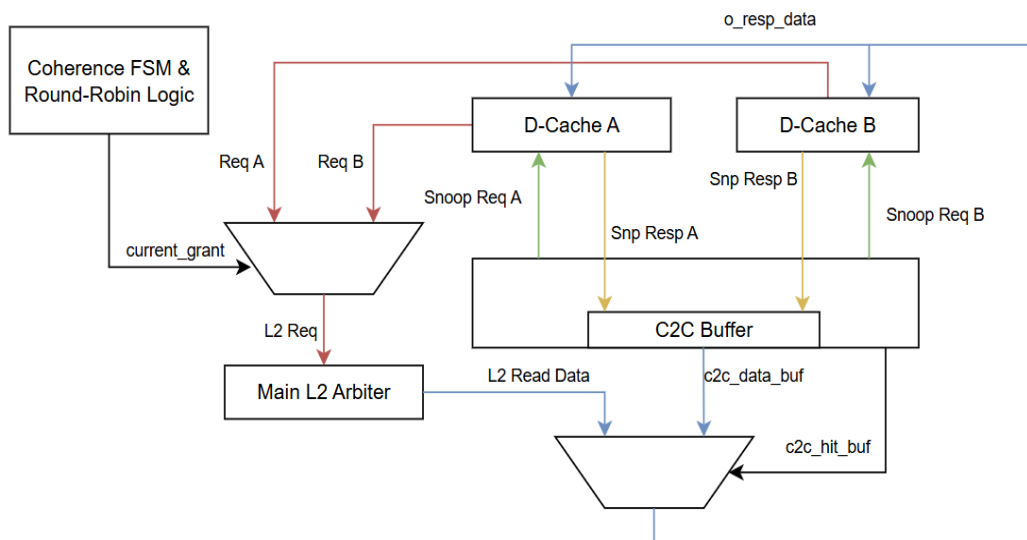
Khối này đóng vai trò giao tiếp, thu thập và điều phối các yêu cầu nạp lệnh (chỉ đọc) từ I-Cache của cả hai lõi xuống hệ thống bộ nhớ phía dưới, đồng thời thiết lập một kênh liên kết vững chắc nhằm tối ưu hóa hơn nữa độ trễ truyền tải các lệnh. Phân hệ được thiết kế như một bộ mạch chọn kênh dựa trên thuật toán phân xử luân phiên (Round-Robin), nhằm đảm bảo tính công bằng trong việc cấp phát băng thông bus và ngăn chặn triệt để hiện tượng "đói" tài nguyên (starvation) của các lõi xử lý. Sơ đồ chi tiết về cấu trúc vi kiến trúc và luồng định tuyến dữ liệu của khối kết nối này được mô tả tại Hình 3.7.



Hình 3.7: Vi kiến trúc khối phân xử lệnh

b. Khối phân xử và đồng bộ luồng dữ liệu

Đây là phân hệ điều khiển phức tạp nhất trong mạng lưới, đảm nhận đồng thời hai vai trò: phân xử truy cập dữ liệu (theo thuật toán Round-Robin) và điều phối các tín hiệu giao thức MOESI giữa hai lõi. Sơ đồ khối chi tiết và luồng định tuyến dữ liệu của bộ phân xử này được minh họa tại Hình 3.8.



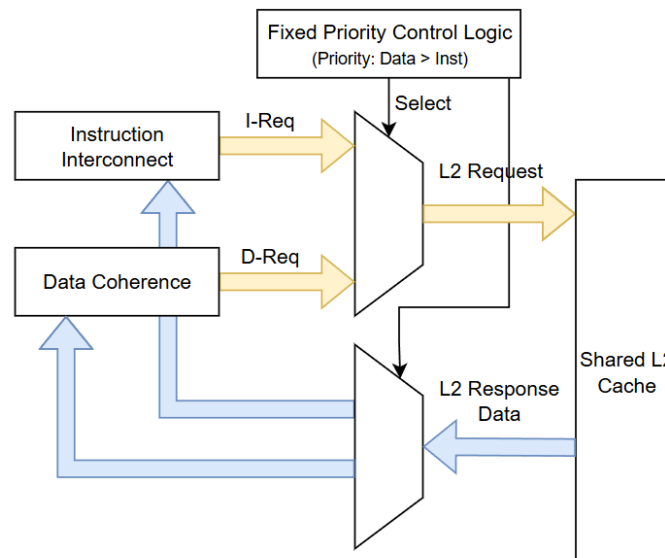
Hình 3.8: Sơ đồ khối và luồng dữ liệu của bộ phân xử luồng dữ liệu

Điểm đột phá của vi kiến trúc này là khả năng Truyền tải dữ liệu trực tiếp (Cache-to-Cache Transfer). Khi một lõi phát sinh yêu cầu truy xuất, khối điều khiển sẽ phát tín

hiệu giám sát (Snoop) sang lỗi đối diện để tìm kiếm bản sao dữ liệu. Nếu phát hiện dữ liệu hợp lệ (Snoop Hit), hệ thống lập tức trích xuất và chuyển tiếp trực tiếp dữ liệu đó về cho lõi yêu cầu, đồng thời cập nhật trạng thái MOESI tương ứng. Quá trình này cho phép bỏ qua hoàn toàn việc truy xuất xuống L2 Cache, giúp giảm thiểu đáng kể độ trễ và giải phóng băng thông bus hệ thống. Yêu cầu chỉ được chuyển tiếp xuống L2 Cache khi lỗi đối diện báo trượt (Snoop Miss).

c. Khối phân xử trung tâm (Main L2 Arbiter)

Khối phân xử trung tâm đóng vai trò là điểm hội tụ cuối cùng, đưa ra quyết định quyền truy xuất đối với luồng lệnh và luồng dữ liệu trước khi đi vào L2 Cache. Sơ đồ chi tiết về vi kiến trúc và luồng dữ liệu vận hành tại khối tại Hình 3.9.



Hình 3.9: Sơ đồ vi kiến trúc và luồng dữ liệu của Khối phân xử trung tâm

Thay vì sử dụng cơ chế luân phiên, khối này áp dụng Chính sách ưu tiên cố định (Fixed Priority): luôn ưu tiên luồng Dữ liệu trước luồng Lệnh. Thiết kế này xuất phát từ bản chất vận hành của pipeline: sự thiếu hụt dữ liệu (Data Hazard) sẽ gây đóng băng (Stall) nghiêm trọng tại các tầng thực thi sâu, trong khi độ trễ nạp lệnh chỉ gây đình trệ tại phần đầu pipeline (Front-end). Việc ưu tiên luồng dữ liệu giúp hệ thống nhanh chóng giải quyết xung đột, giải phóng các lệnh đang bị kẹt và tối ưu hóa chỉ số lệnh thực thi trên mỗi chu kỳ xung nhịp (IPC - Instructions Per Cycle).

CHƯƠNG 4. THIẾT KẾ CHI TIẾT

4.1. Thiết kế chi tiết phần cứng

Trước khi đi sâu vào hiện thực mạch logic cho bộ nhớ đệm D-Cache – thành phần phức tạp nhất, kiến trúc tổng thể của hệ thống vi xử lý đa lõi được tổ chức với luồng dữ liệu liền mạch từ vi xử lý xuống các tầng bộ nhớ. Cụ thể, khối điều khiển trung tâm là CPU RV32I lõi kép sở hữu pipeline 6 tầng, được tích hợp bộ dự đoán rẽ nhánh động, khối xử lý rủi ro và hỗ trợ tập lệnh nguyên tử. Gắn liền với mỗi lõi là hệ thống bộ nhớ đệm cấp 1 (kiến trúc 4-Way Set Associative, 16-Set), được chia tách thành hai khối độc lập: L1 I-Cache với thiết kế chỉ đọc, lược bỏ logic đồng bộ; và L1 D-Cache phức tạp hơn nhờ tích hợp phần cứng FSM giao thức MOESI Snoopy cùng ALU xử lý lệnh Atomic nội bộ. Cuối cùng, dữ liệu được đồng bộ và truyền tải ra ngoài thông qua L2 Cache dùng chung (64-Set, 4-Way), kết nối với tầng L1 qua mạng lưới phân xử phân cấp và giao tiếp với hệ thống ngoại vi bằng chuẩn bus AXI4 Full.

4.2. Thiết kế chi tiết lõi vi xử lý RV32IA

4.2.1. Vi kiến trúc Pipeline 6 tầng và Cơ chế đồng bộ

Vi xử lý sử dụng pipeline lệnh 6 tầng (IF, S2, ID, EX, MEM, WB) được đồng bộ bởi các thanh ghi trung gian nhằm tăng cường hiệu suất hoạt động. Cải tiến cốt lõi so với kiến trúc 5 tầng tiêu chuẩn nằm ở việc phân tách giai đoạn giao tiếp với bộ nhớ đệm (I-Cache và D-Cache), cho phép tối ưu hóa luồng dữ liệu thông qua cơ chế xử lý gói đầu.

a. Phân rã quá trình nạp lệnh tại tầng S1 và S2

Để khắc phục độ trễ truy xuất SRAM và nâng cao tần số xung nhịp, quá trình nạp lệnh được chia thành hai giai đoạn chuyên biệt trong pipeline. Tại tầng IF (S1), bộ đếm chương trình tính toán địa chỉ mục tiêu và gửi yêu cầu truy xuất trực tiếp tới I-Cache trong một chu kỳ xung nhịp. Tiếp theo, tại tầng S2 (Tiếp nhận lệnh), hệ thống chốt dữ liệu lệnh phản hồi từ I-Cache để chuyển tiếp tới giai đoạn giải mã. Nhằm tối ưu hóa hiệu suất rẽ nhánh, tầng S2 được tích hợp cờ trạng thái cho phép hủy bỏ (flush) tức thời các lệnh dự đoán sai, giúp duy trì tính toàn vẹn của pipeline mà không làm gián đoạn chu trình nạp lệnh liên tục từ bộ nhớ.

b. Cơ chế gởi đầu với D-Cache

Để tối ưu độ trễ, pipeline và D-Cache được thiết kế hoạt động gởi đầu qua hai tầng Thực thi (EX) và Truy cập bộ nhớ (MEM). Tầng EX (Chu kỳ N): Tín hiệu, địa chỉ từ ALU truyền thẳng đến D-Cache (không qua thanh ghi), kích hoạt ngay giai đoạn Truy cập (Access). Tầng MEM (Chu kỳ N+1): Khi D-Cache so sánh thẻ (Compare), CPU tiến hành truy cập, tự động căn chỉnh và mở rộng dữ liệu (8/16-bit lên 32-bit) vào pipeline.

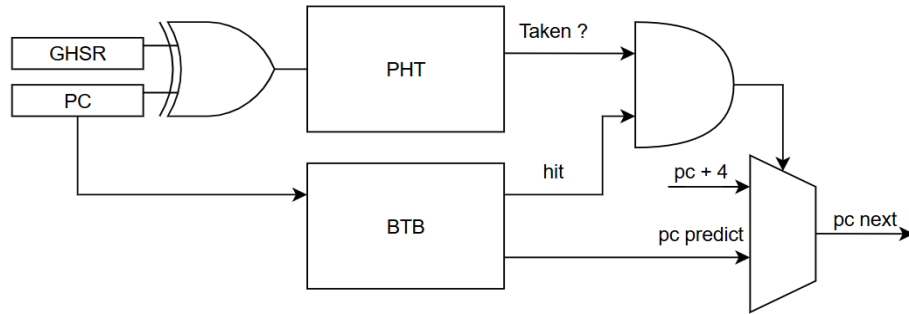
4.2.2. Hiện thực mạch logic Khối xử lý rủi ro (Hazard Unit)

Hazard Unit điều phối pipeline bằng cách tổng hợp tín hiệu từ các tầng (ID, EX, MEM, WB) và bộ nhớ đệm, từ đó xuất ra ba tín hiệu điều khiển: Chuyển tiếp (Forward), Đóng băng (Stall) và Làm sạch (Flush). Hệ thống điều khiển rủi ro pipeline được vận hành thông qua sự phối hợp của ba mạch chức năng chính nhằm đảm bảo tính toàn vẹn của dữ liệu và luồng thực thi. Đầu tiên, mạch chuyển tiếp dữ liệu (Forwarding) giải quyết xung đột dữ liệu đọc-sau-ghi bằng cách đối chiếu thanh ghi nguồn ở tầng EX với thanh ghi đích ở các tầng MEM và WB, trong đó mạch sẽ ưu tiên chọn dữ liệu mới nhất từ tầng gần nhất (MEM) nếu có nhiều bản cập nhật cho cùng một thanh ghi. Tiếp theo, mạch đóng băng (Stall) có nhiệm vụ khóa tạm thời các thanh ghi khi xảy ra hiện tượng Load-Use Hazard (lệnh tại tầng EX là lệnh đọc bộ nhớ và thanh ghi đích trùng với toán hạng cần dùng tại tầng ID) hoặc khi xảy ra lỗi trượt bộ đệm (Cache Miss) khiến bộ đệm dữ liệu hoặc lệnh báo bận và yêu cầu tạm dừng để đồng bộ. Cuối cùng, mạch làm sạch (Flush) chịu trách nhiệm kích hoạt chủ yếu khi dự đoán sai rẽ nhánh để dọn dẹp các lệnh nạp nhầm, hoặc chèn "bong bóng" (bubble) vào tầng thực thi nhằm giải quyết xung đột dữ liệu tải; đồng thời, để bảo toàn trạng thái hệ thống, mạch xóa này được thiết kế khóa chéo với mạch Stall, đảm bảo lệnh xóa chỉ được thực thi khi tầng pipeline không bị đóng băng.

4.2.3. Hiện thực khối dự đoán rẽ nhánh

Khối BPU (Branch Prediction Unit) sử dụng kiến trúc lai (hybrid) tích hợp ngay tại tầng Nạp lệnh (IF), vận hành dựa trên hai phân hệ chính theo mô hình GShare [9]: Bộ đệm địa chỉ đích (BTB) chịu trách nhiệm lưu trữ và xác định địa chỉ nhảy tiềm

năng [11], kết hợp với Bảng lịch sử mẫu (PHT) sử dụng phép toán logic XOR giữa thanh ghi lịch sử toàn cục (GHSR) và địa chỉ PC hiện tại để dự đoán hướng rẽ nhánh (Taken hoặc Not Taken) [9]. Sự phối hợp phân cứng chi tiết giữa GHSR, bảng PHT và BTB được trình bày tại Hình 4.1.

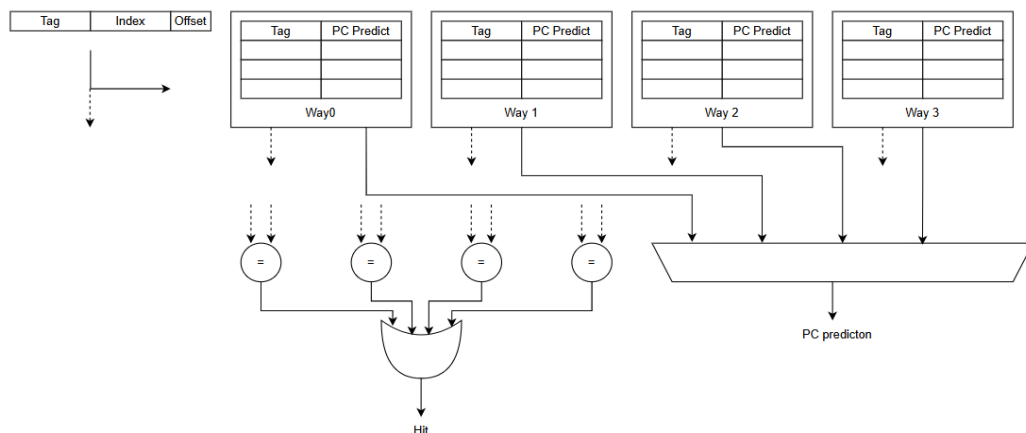


Hình 4.1: Sơ đồ vi kiến trúc tổng thể khối dự đoán rẽ nhánh động BPU

Việc cập nhật trạng thái lịch sử (PHT/GHSR) được dời về tầng Thực thi (EX). Cơ chế này đảm bảo chỉ những kết quả rẽ nhánh thực tế từ ALU mới được ghi nhận, giúp tối ưu độ chính xác của các lần dự đoán tiếp theo.

a. Vi kiến trúc Bộ đệm địa chỉ đích (BTB)

Bộ đệm địa chỉ đích (BTB) đóng vai trò cung cấp tức thì địa chỉ đích của lệnh nhảy ngay trong chu kỳ nạp lệnh mà không làm phát sinh trạng thái chờ (pipeline stall) [11]. Trong khóa luận này, BTB được thiết kế theo cấu trúc mảng liên kết tập hợp 4 đường (4-Way Set-Associative) với 8 tập hợp (8-Set). Sơ đồ mạch thiết kế được minh họa tại Hình 4.2.

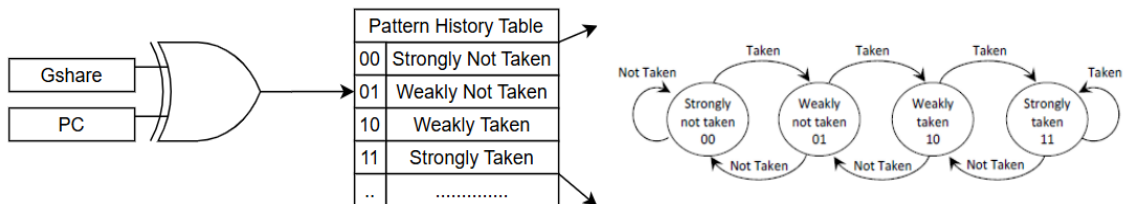


Hình 4.2: Sơ đồ thiết kế mạch phân cứng khối BTB

Luồng hoạt động của khối tuân theo nguyên lý truy xuất bộ nhớ đệm tiêu chuẩn [11]: mạch giải mã tách các bit Tag và Index từ địa chỉ để so sánh đồng thời trên 4 đường nhớ. Nếu xảy ra hiện tượng trúng bộ đệm (Hit), địa chỉ đích tương ứng được xuất qua bộ ghép kênh (MUX) để cập nhật trực tiếp cho bộ đếm chương trình (PC). Trường hợp xảy ra trượt bộ đệm (Miss) và mảng đã đầy, khối điều khiển sẽ kích hoạt mô-đun PLRUt (Mục 4.3) để xác định đường nhớ cần bị ghi đè.

b. Mạch băm GShare và Máy trạng thái dự đoán 2-bit

Khối PHT kết hợp logic băm GShare và máy trạng thái 2-bit để đưa ra quyết định dự đoán rẽ nhánh [9]. Vi kiến trúc tổng thể được minh họa tại Hình 4.3.



Hình 4.3: Sơ đồ mạch băm logic GShare và máy trạng thái bộ đếm 2-bit

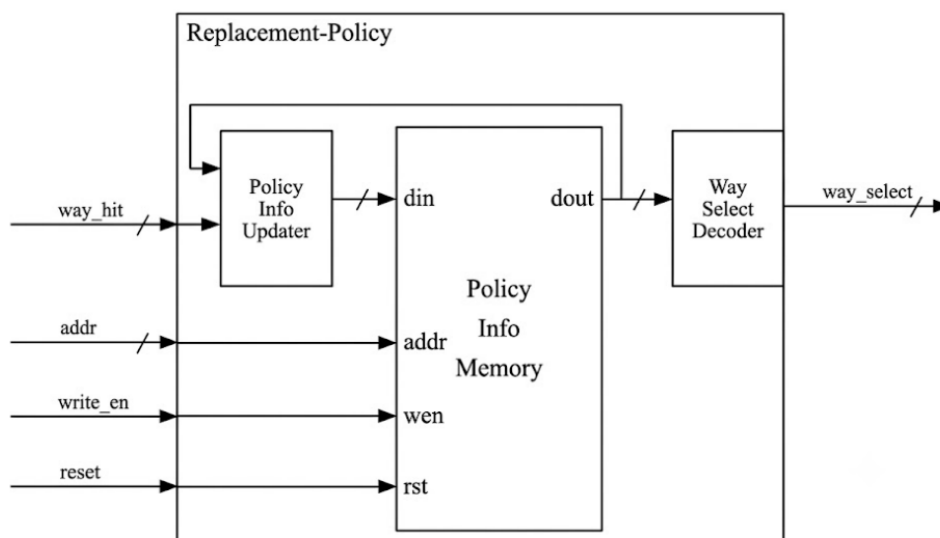
Mạch băm GShare sử dụng cổng XOR kết hợp địa chỉ PC với Thanh ghi lịch sử toàn cục (GHSR) để phân tán chỉ mục, giảm xung đột địa chỉ trong bảng PHT khi xử lý vòng lặp phức tạp [9]. Mỗi ô nhớ trong PHT được quản lý bởi một bộ đếm bão hòa 2-bit với 4 trạng thái từ Không rẽ nhánh mạnh (00) đến Rẽ nhánh mạnh (11) [11]. Các trạng thái này cập nhật theo kết quả thực tế từ ALU (tăng khi Taken, giảm khi Not Taken), tạo ra độ trễ từ tính giúp hệ thống lọc nhiễu và tránh thay đổi quyết định dự đoán đột ngột [11].

4.3. Hiện thực giải thuật thay thế PLRUt

Thay vì duy trì ma trận lịch sử phức tạp như LRU truyền thống, hệ thống sử dụng thuật toán Pseudo-LRU dạng cây (Tree-based PLRU) để tối ưu diện tích silicon và giảm đáng kể mức tiêu thụ năng lượng tĩnh [11]. Kiến trúc này có tính khái quát cao, được ứng dụng linh hoạt cho cả bộ đệm rẽ nhánh (BTB) và các tầng Cache (L1/L2) [14].

4.3.1. Vi kiến trúc tổng quát của mô-đun PLRUt

Mạch quản lý chính sách thay thế gồm ba phân hệ phối hợp như Hình 4.4.

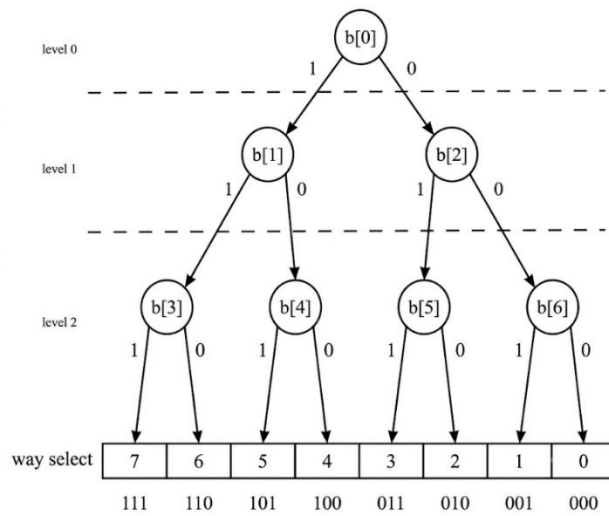


Hình 4.4: Sơ đồ khối phân hệ quản lý chính sách thay thế

Để tối ưu hóa tài nguyên phần cứng, khối điều khiển thay thế được cấu thành từ ba phân hệ chính liên kết chặt chẽ với nhau. Đầu tiên là Khối bộ nhớ trạng thái (PIM), nơi lưu trữ vector lịch sử thông qua mảng SRAM hoặc mảng thanh ghi tùy thuộc vào yêu cầu tối ưu dung lượng hay tốc độ truy xuất. Kế tiếp, Khối giải mã chọn đường (Way Select Decoder) sẽ đọc dữ liệu vector này để giải mã ra tín hiệu chỉ định đường cần thay thế (Victim Way). Cuối cùng, Khối cập nhật thông tin (Policy Info Updater) đảm nhận nhiệm vụ nhận tín hiệu `way_hit` từ hệ thống để tính toán và cập nhật lại trạng thái lịch sử chính xác cho các chu kỳ tiếp theo [14].

4.3.2. Cấu trúc định tuyến cây nhị phân (Binary Tree Routing)

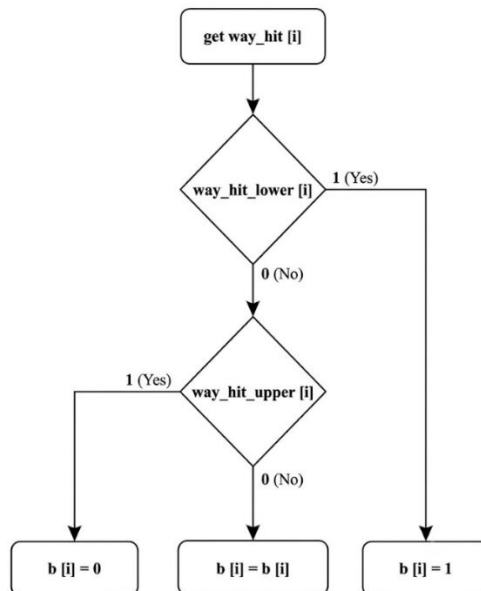
Thay vì quản lý mảng 2 chiều, PLRUt tổ chức các đường vật lý (Way) thành các nút lá của một cây nhị phân. Mỗi nút (`b[i]`) đóng vai trò là một bộ chuyển mạch (MUX) điều hướng luồng duyệt dựa trên giá trị bit trạng thái [14]. Cụ thể, giá trị bit logic 1 sẽ điều hướng luồng duyệt đi sang nhánh bên trái, còn giá trị bit logic 0 sẽ điều hướng luồng duyệt đi sang nhánh bên phải. Quá trình này giúp hệ thống xác định chính xác dòng dữ liệu cần bị ghi đè. Nhờ vậy, độ phức tạp của mạch phần cứng được tối ưu đáng kể, với toàn bộ luồng hoạt động được mô tả chi tiết tại Hình 4.5.



Hình 4.5: Cấu trúc cây nhị phân định tuyến của giải thuật PLRUt

4.3.3. Mạch tổ hợp phân rẽ tín hiệu và cập nhật trạng thái đối ngẫu

Để cây luôn trở về phía các khối dữ liệu "lạnh", mạch cập nhật sử dụng cơ chế phân rẽ bit tín hiệu way_hit thành các lát cắt (slices). Cơ chế này hoạt động song song với luồng truy xuất [14]. Tín hiệu way_hit được cắt làm hai nửa: upper (nhánh trái) và lower (nhánh phải). Dựa trên các lát cắt này, mạch tổ hợp thực hiện cập nhật trạng thái cho từng nút theo lưu đồ logic tại Hình 4.6 [14].



Hình 4.6: Lưu đồ logic mạch cập nhật trạng thái nhánh PLRUt

Nếu đường truy cập thuộc nửa dưới (lower = 1): Ép nút lên mức 1. Nếu đường truy cập thuộc nửa trên (upper = 1): Ép nút xuống mức 0. Nếu không có tác động: Tín hiệu ghi bị khóa, bảo toàn trạng thái hiện tại.

4.4. Thiết kế chi tiết bộ nhớ đệm phân cấp

4.4.1. Vi kiến trúc nền tảng (Common Cache Architecture)

Các bộ nhớ đệm trong hệ thống (I-Cache, D-Cache, L2 Cache) đều kế thừa một cấu trúc nền tảng chung nhằm tối ưu hóa việc tái sử dụng phần cứng. Nền tảng này tập trung xử lý ba cơ chế cốt lõi: phân rã địa chỉ, đồng bộ pipeline 2 tầng và mạch logic chống xung đột dữ liệu nội bộ. Riêng giải thuật thay thế PLRUt đã được trình bày chi tiết tại Mục 4.3.

a. Mạch tổ hợp định tuyến phân rã địa chỉ

Mạch giải mã sử dụng phần cứng tổ hợp để trích xuất tức thời địa chỉ vật lý 32-bit (dựa trên cấu hình Cache Line 64 byte) thành 3 trường chức năng có khả năng tham số hóa linh hoạt cho các cấp bộ nhớ đệm. Trong đó, trường Offset chiếm 6 bit thấp nhất (gồm 2 bit định vị byte cục bộ và 4 bit chọn word), trường Index dùng để định tuyến tra cứu mảng SRAM với độ rộng bit linh hoạt từ 4 bit cho L1 (16 lines) đến 6 bit cho L2 (64 lines), và trường Tag gồm các bit cao còn lại được chốt thẳng vào thanh ghi pipeline để phục vụ đối chiếu ở chu kỳ tiếp theo.

b. Đồng bộ hóa phần cứng Pipeline 2 giai đoạn

Quá trình truy xuất được đồng bộ qua 2 chu kỳ xung nhịp (pipeline) nhằm tối ưu tốc độ. Giai đoạn 1 (Truy cập): Địa chỉ sau khi giải mã sẽ lấy phần Chỉ mục (Index) để kích hoạt đọc mảng bộ nhớ (SRAM). Cùng lúc, phần Thẻ (Tag) được đưa vào chốt tại thanh ghi trung gian (REG). Giai đoạn 2 (So sánh): Mạch so sánh (Check hit) tiến hành đối chiếu Thẻ đọc ra từ SRAM với Thẻ đang được chốt tạm thời trong REG để xuất tín hiệu Đánh trúng (Hit). Thanh ghi REG này cũng đóng vai trò chốt giữ trạng thái (Stall) pipeline khi xảy ra trượt bộ đệm.

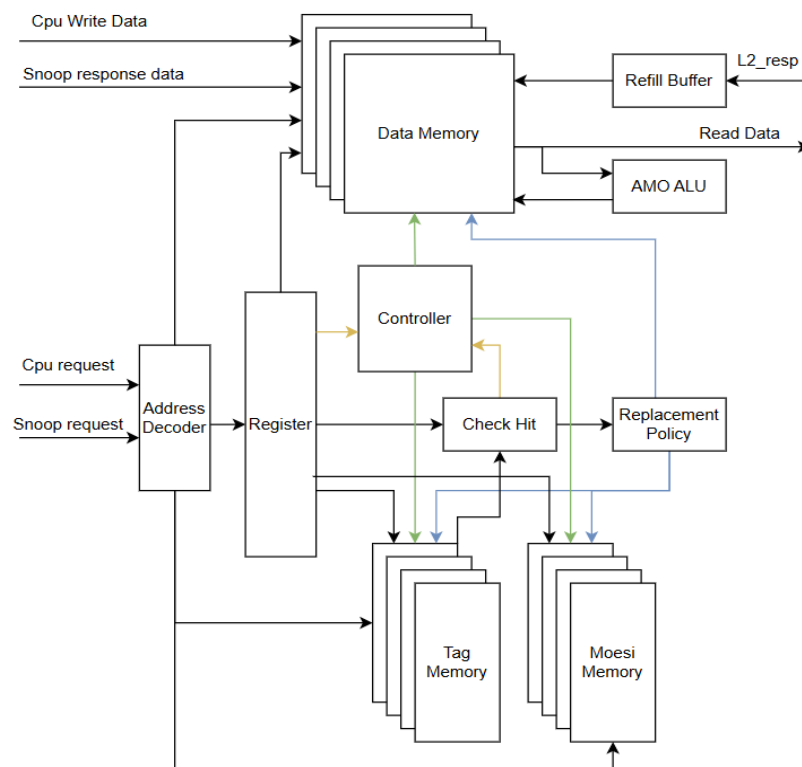
c. Cơ chế chuyển tiếp dữ liệu nội bộ

Để xử lý xung đột Đọc-sau-Ghi (Read-After-Write) khi truy xuất cùng một địa chỉ trong một chu kỳ, phần cứng tích hợp mạch vòng chuyển tiếp dữ liệu tự động.

Đường dẫn tín hiệu đặc biệt này hoạt động song song và hoàn toàn trong suốt đối với tập lệnh của phần mềm. Khi mạch logic phát hiện địa chỉ đọc và ghi trùng nhau đồng thời có lệnh ghi đang kích hoạt, hệ thống sẽ lập tức điều khiển bộ ghép kênh (MUX) để đưa thẳng dữ liệu mới tới ngõ ra đọc. Cơ chế này cho phép cập nhật dữ liệu tức thời mà không cần chờ chu trình ghi vào mảng nhớ hoàn tất, giúp pipeline vận hành liên tục không bị gián đoạn.

4.4.2. Kiến trúc D-Cache

L1 D-Cache kế thừa cấu trúc pipeline 2 giai đoạn nền tảng nhưng được mở rộng đáng kể để quản lý tiến trình ghi và đồng bộ đa lõi. Khác biệt cốt lõi nằm ở việc tích hợp ba phân hệ chuyên biệt: Bộ điều khiển giao thức MOESI, Mạch giám sát bus (Snoop) và Đơn vị tính toán nguyên tử nội bộ (AMO ALU). Sơ đồ liên kết tổng thể được thể hiện tại Hình 4.7.



Hình 4.7: Sơ đồ khối vi kiến trúc của bộ nhớ đệm L1 D-Cache

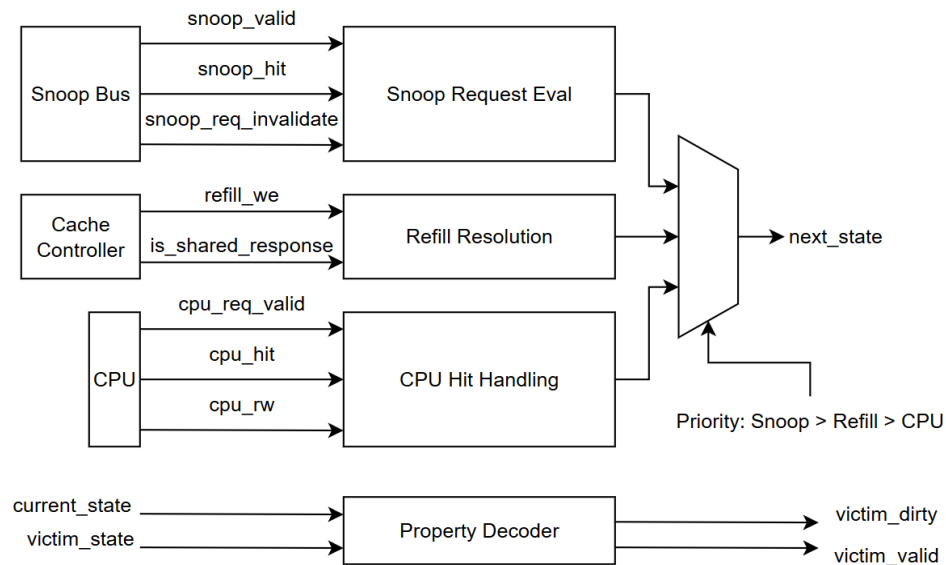
a. Hoạt Động Của Máy chuyển đổi trạng thái (FSM)

Hoạt động của toàn bộ hệ thống D-Cache được định tuyến bởi một máy trạng thái hữu hạn (FSM) trung tâm. Máy trạng thái này có nhiệm vụ phân xử các yêu cầu

từ lỗi xử lý, giao tiếp với bộ phân xử bus (Bus Arbiter) để nạp (Refill) hoặc ghi trả (Write-back) dữ liệu. FSM này bao gồm 9 trạng thái hoạt động nghiêm ngặt, được đặc tả chi tiết tại Bảng PIII.1.

b. Quản Lý Giao Thức Đồng Bộ MOESI

Để tối ưu hóa tốc độ phản hồi, khối điều khiển MOESI được thiết kế hoàn toàn bằng mạch tổ hợp (combinational logic). Mạch này hoạt động như một bộ giải mã ưu tiên, tính toán trạng thái kế tiếp của bộ đệm ngay trong một chu kỳ tại Hình 4.8.



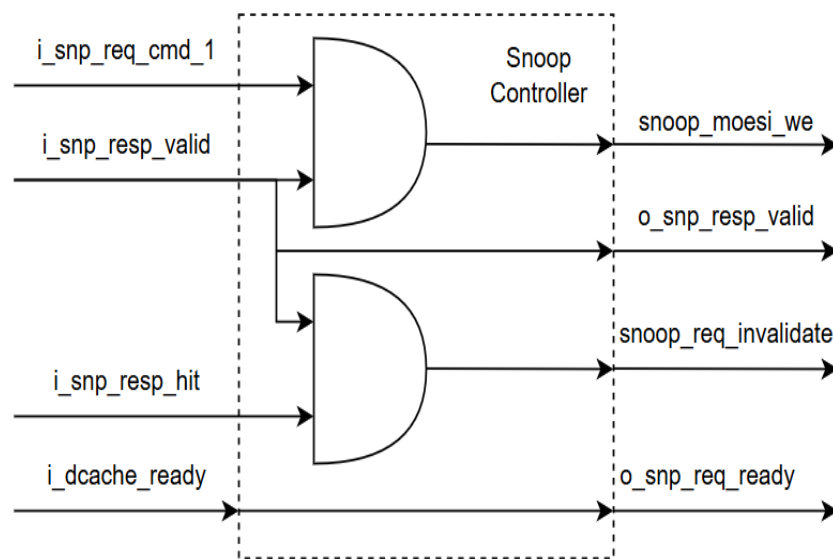
Hình 4.8: Sơ đồ vi kiến trúc mạch tổ hợp giải mã trạng thái MOESI

Mạch giải mã sử dụng một bộ ghép kênh ưu tiên (Priority MUX) để định tuyến và giải quyết xung đột trạng thái hệ thống theo thứ tự nghiêm ngặt từ cao xuống thấp. Đầu tiên, ở cấp Giám sát Bus (Ưu tiên 1), mạch duy trì tính nhất quán bằng cách hạ cấp trạng thái về Owned hoặc Shared khi lõi khác đọc, hoặc chuyển hẳn sang Invalid để vô hiệu hóa khi lõi khác ghi độc quyền. Tiếp theo, ở giai đoạn Nạp dữ liệu (Ưu tiên 2) khi xảy ra hiện tượng trượt bộ đệm (Miss), mạch sẽ cấp phát trạng thái khởi tạo như Modified, Exclusive, hoặc Shared dựa trên loại lệnh từ CPU và cờ chia sẻ của hệ thống. Cuối cùng, ở mức Truy xuất cục bộ (Ưu tiên 3) khi đánh trúng bộ đệm (Hit), trạng thái hiện tại được giữ nguyên và chỉ nâng cấp lên Modified nếu xuất hiện lệnh ghi từ CPU. Bên cạnh luồng ưu tiên này, khối giải mã thuộc tính hoạt động độc lập ở ngõ ra sẽ trích xuất trạng thái của dòng dữ liệu bị thay thế thành các cờ vật lý

như dữ liệu bản hay cờ hợp lệ, từ đó hỗ trợ bộ điều khiển nhận biết và kích hoạt chính xác chu trình ghi trả (Write-back) khi cần thiết.

c. Vi kiến trúc khối Giám sát Bus (Snoop Controller)

Khối giám sát kênh truyền (Snoop) đóng vai trò giao tiếp trực tiếp và lọc tín hiệu dữ liệu hai chiều giữa bus hệ thống ngoại vi và pipeline bộ nhớ đệm. Để đảm bảo khả năng phản hồi tức thời trong cùng một chu kỳ xung nhịp hệ thống, phân hệ này được thiết kế hoàn toàn bằng logic tổ hợp. Chi tiết vi kiến trúc mức cổng logic được minh họa tại Hình 4.9.

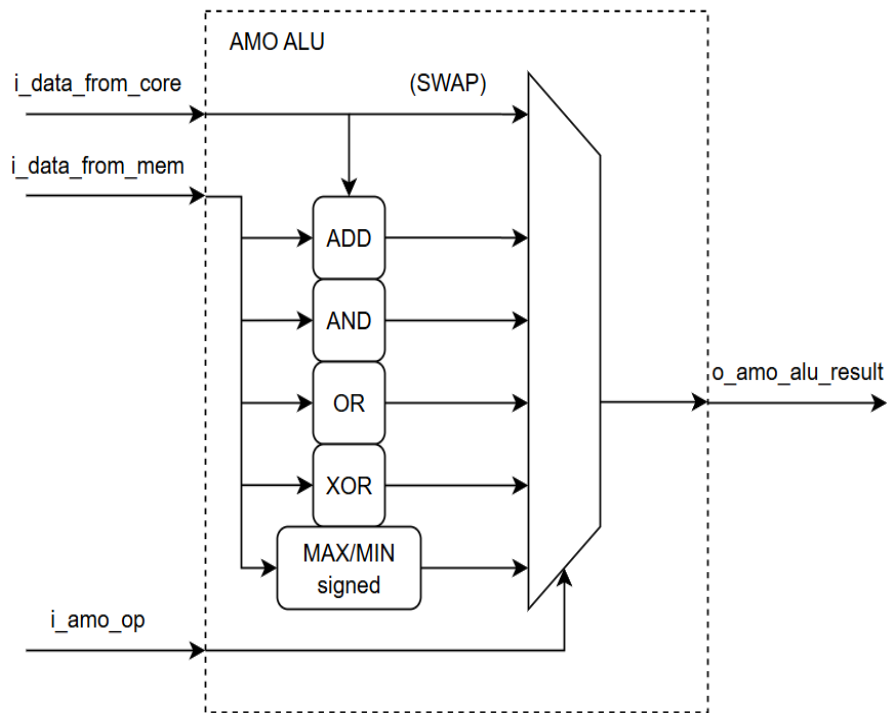


Hình 4.9: Sơ đồ vi kiến trúc mức cổng logic của Khối giám sát Bus

Cấu trúc logic cốt lõi của khối bao gồm ba đường dẫn tín hiệu chính nhằm tối ưu hóa quá trình đồng bộ toàn hệ thống. Đầu tiên, mạch đồng bộ trạng thái sẵn sàng chuyển tiếp trực tiếp cờ trạng thái của pipeline ra bus ngoại vi, đảm bảo hệ thống chỉ tiếp nhận yêu cầu giám sát khi bộ đệm không bị đóng băng (stall). Tiếp theo, mạch giải mã lệnh Vô hiệu hóa (Invalidation) sử dụng cổng AND để kết hợp lệnh ghi đọc quyền từ bus với cờ hợp lệ của pipeline, lập tức ép khối dữ liệu cục bộ về trạng thái Invalid khi được kích hoạt. Cuối cùng, mạch cho phép cập nhật MOESI dùng cổng AND ràng buộc tín hiệu ghi, chỉ cho phép kích hoạt khi quá trình giám sát hoàn tất và có đánh trúng thẻ định danh (Snoop Hit), giúp ngăn chặn triệt để việc ghi dữ liệu rác vào mảng SRAM trạng thái.

d. Khối tính toán nguyên tử nội bộ (AMO ALU)

Khối ALU hoạt động hoàn toàn bằng logic tổ hợp, tiếp nhận đồng thời hai toán hạng 32-bit đầu vào (từ mảng SRAM và từ lõi CPU) cùng mã lệnh điều khiển 3-bit. Sơ đồ vi kiến trúc và luồng định tuyến dữ liệu bên trong phân hệ này được minh họa chi tiết tại Hình 4.10.

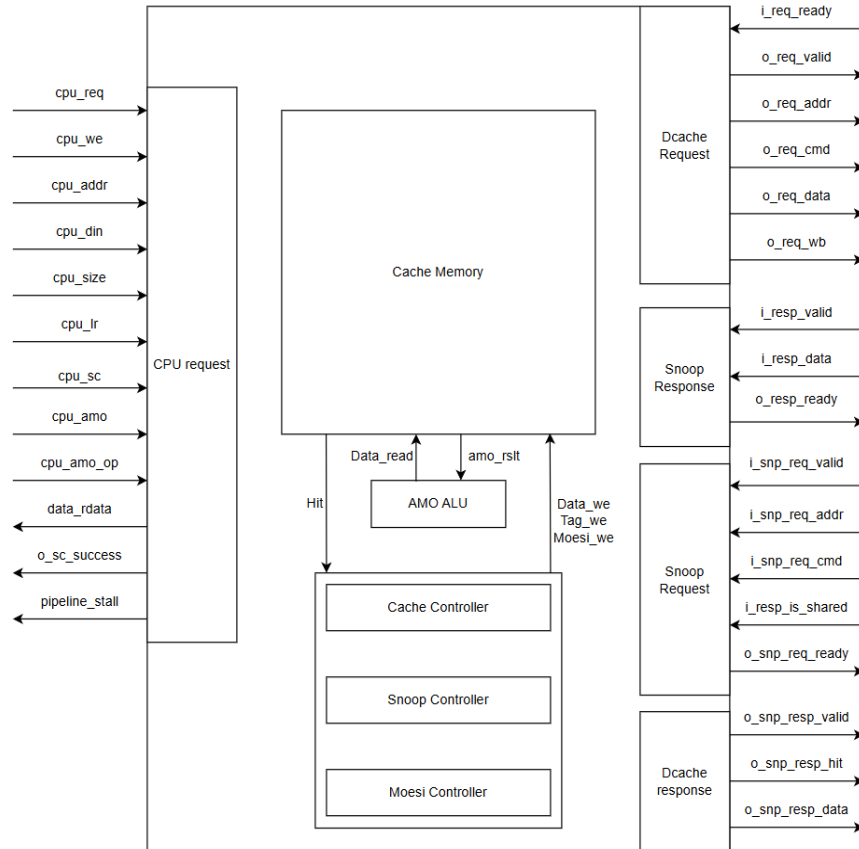


Hình 4.10: Sơ đồ vi kiến trúc mạch logic của Đơn vị tính toán nguyên tử

Dựa trên tín hiệu điều khiển ở ngõ vào (*i_amo_op*), mạch tổ hợp thực thi và xuất kết quả (*o_amo_alu_result*) thông qua bộ ghép kênh đa ngõ vào để ghi cập nhật xuống bộ nhớ trong cùng chu kỳ xung nhịp. Mạch phần cứng hỗ trợ 7 phép toán cơ bản bao gồm: phép Hoán đổi (SWAP) định tuyến trực tiếp dữ liệu từ CPU để ghi đè; phép toán học (ADD) đưa hai toán hạng qua bộ cộng số học 32-bit; các phép Logic bit (AND, OR, XOR) xử lý toán hạng qua các cổng logic tương ứng; và các phép So sánh (MAX, MIN) đưa dữ liệu qua bộ so sánh có dấu rồi dùng MUX để chọn ra giá trị lớn hoặc nhỏ nhất. Việc tích hợp đồng bộ các thành phần này giúp đảm bảo tính nguyên tử tuyệt đối cho các thao tác truy xuất dữ liệu, từ đó duy trì tính nhất quán cho toàn bộ hệ thống đa lõi.

e. Đặc Tả Tín Hiệu Giao Tiếp (Interface)

Giao diện tín hiệu mô tả các kết nối vật lý giữa L1 D-Cache với lõi xử lý (CPU) và mạng lưới phân xử hệ thống (Snoop Bus / L2 Arbiter). Sơ đồ tổng quan các cổng tín hiệu vào/ra (I/O ports) được thể hiện ở Hình 4.11.

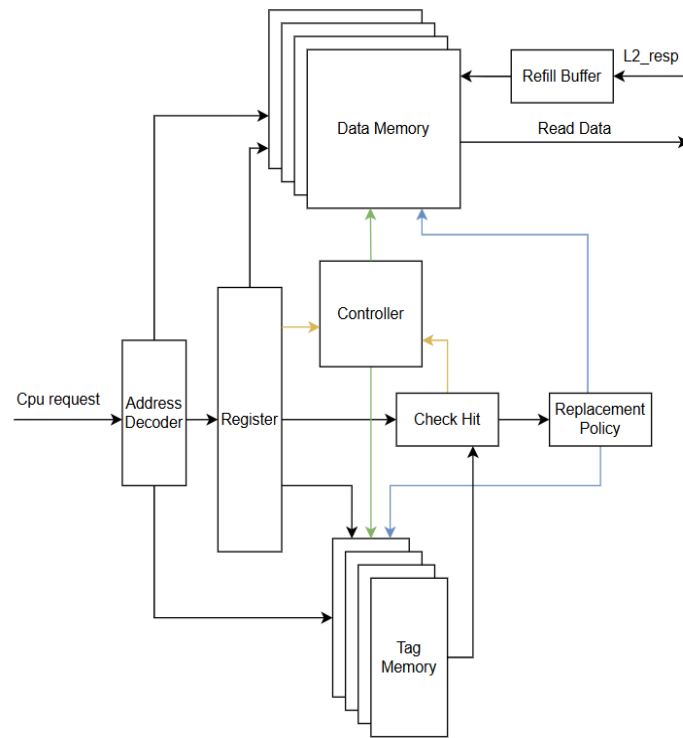


Hình 4.11: Tổng quan interface của D-cache

Dựa trên sơ đồ kết nối, các tín hiệu giao tiếp được chia thành các nhóm chức năng riêng biệt. Hướng tín hiệu (I/O), độ rộng bit và chức năng chi tiết của từng tín hiệu được liệt kê trong Bảng PIII.2.

4.4.3. Thiết kế phần cứng L1 I-Cache (Instruction Cache)

Kế thừa vi kiến trúc nền tảng (Mục 4.4.1), L1 I-Cache được tối ưu hóa chuyên biệt cho tầng nạp lệnh (Instruction Fetch). Nhờ đặc thù chỉ đọc (Read-Only), phân hệ này lược bỏ hoàn toàn các mạch logic phức tạp (MOESI, Snoop, AMO ALU), giúp tiết kiệm đáng kể diện tích silicon. Sơ đồ vi kiến trúc tổng thể tại Hình 4.12.



Hình 4.12: Sơ đồ khối vi kiến trúc của L1 I-Cache

a. Đặc thù chỉ đọc và Giảm lược phân cứng

Sự khác biệt về mặt chức năng dẫn đến hai tính chính lớn trong thiết kế Datapath của I-Cache so với vi kiến trúc nền tảng. Thứ nhất, hệ thống tối giản mảng bộ nhớ trạng thái khi mảng bộ nhớ thẻ định danh (Tag RAM) của I-Cache chỉ cần duy trì duy nhất một cờ Hợp lệ (Valid bit) cho mỗi khối dữ liệu, thay vì phải lưu trữ 3-bit trạng thái MOESI phức tạp. Thứ hai, thiết kế loại bỏ hoàn toàn mạch ghi trả (Write-back) vì luồng lệnh không bao giờ bị CPU sửa đổi nên không tồn tại khái niệm "dữ liệu bẩn". Do đó, khi xảy ra trượt bộ đệm (Cache Miss), khối điều khiển chỉ yêu cầu nạp dòng lệnh mới, hứng qua một thanh ghi đệm (Refill Buffer) rồi ghi đè thẳng lên đường dữ liệu cũ.

b. Cơ chế đồng bộ khóa pipeline toàn cục (Global Stall)

Để duy trì đồng bộ cho toàn hệ thống SoC, mạch tổ hợp của I-Cache sẽ tự động khóa pipeline (Stall) khi nhận được tín hiệu từ một trong ba nguồn độc lập. Đầu tiên là lệnh khóa cục bộ do chính bộ điều khiển I-Cache phát ra khi trượt bộ đệm (Cache Miss) để chờ nạp lệnh mới. Tiếp theo là tín hiệu khóa từ CPU, xuất hiện khi CPU chủ

động yêu cầu dừng nạp lệnh do gặp xung đột điều khiển (Control Hazard) hoặc đang bận giải mã. Cuối cùng là lệnh khóa từ D-Cache thông qua một liên kết cứng; khi D-Cache bận xử lý (như chờ bus để Write-back), I-Cache cũng buộc phải dừng để đảm bảo hai pipeline lệnh và dữ liệu luôn vận hành khớp nhịp với nhau.

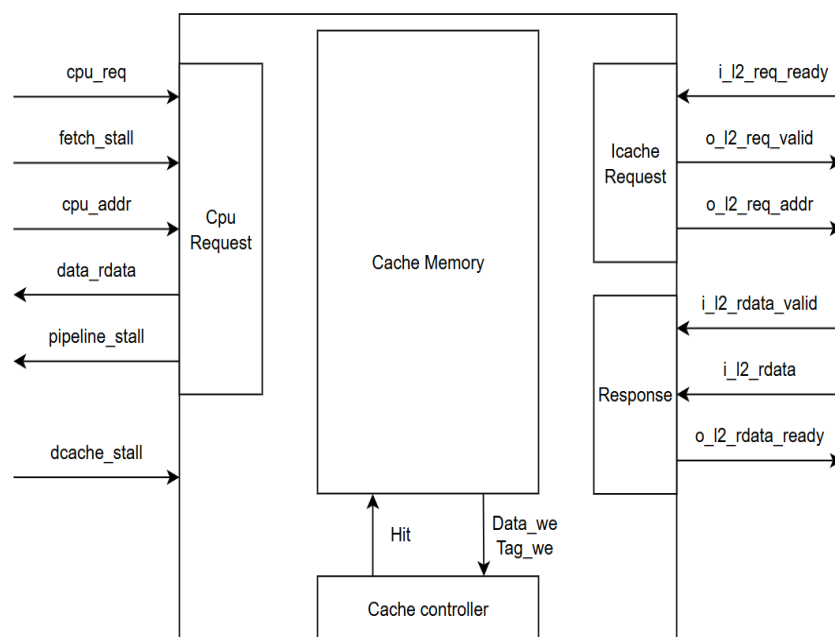
c. Hoạt Động Của Máy chuyển đổi trạng thái (FSM)

Khởi điều khiển trung tâm của I-Cache quản lý quá trình nạp lệnh thông qua một máy trạng thái (FSM) gồm 5 bước. Chức năng và điều kiện chuyển đổi của từng trạng thái được đặc tả chi tiết trong Bảng PIII.3.

d. Đặc Tả Tín Hiệu Giao Tiếp (Interface)

Giao diện tín hiệu mô tả các kết nối vật lý giữa L1 I-Cache với tầng nạp lệnh (Fetch) của lõi CPU, khối D-Cache (để đồng bộ pipeline) và bộ nhớ cấp dưới (L2 Cache / Bus Arbiter). Sơ đồ tổng quan các cổng tín hiệu vào/ra (I/O ports) của phân hệ được thể hiện ở

Hình 4.13.



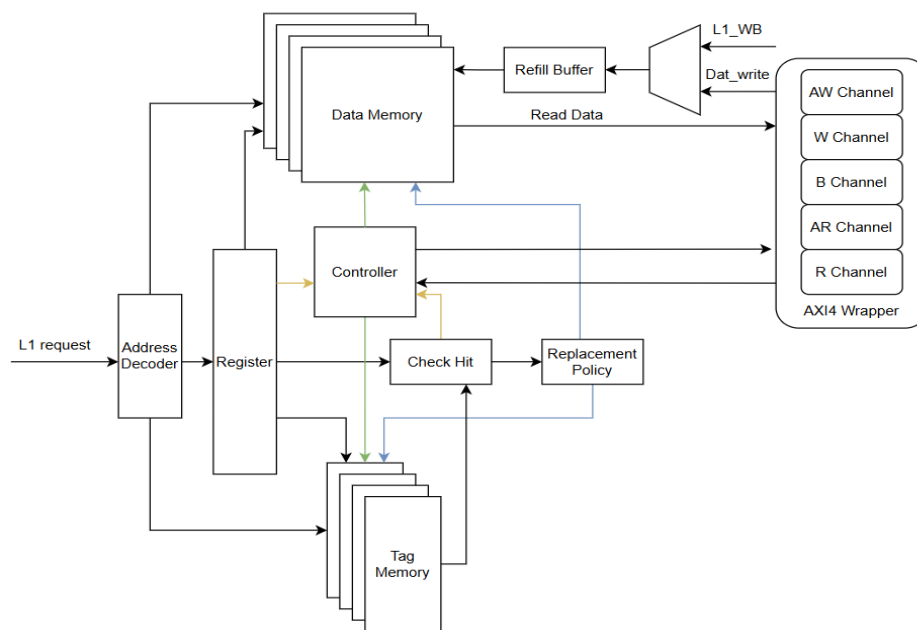
Hình 4.13: Tổng quan giao diện tín hiệu (Interface) của L1 I-Cache

Dựa trên cấu trúc kết nối vật lý đã minh họa, các tín hiệu giao tiếp được chia thành từng nhóm chức năng cụ thể nhằm phục vụ quá trình nạp lệnh và đồng bộ hệ thống.

Hướng di chuyển (I/O), độ rộng bit và chức năng chi tiết của từng tín hiệu được đặc tả trong Bảng PIII.4.

4.4.4. Thiết kế phần cứng L2 Cache và giao tiếp AXI4 Full

L2 Cache (16KB) đóng vai trò trung gian giữa L1 Cache và bộ nhớ chính, tối ưu hóa các tiến trình nạp/ghi trả thông qua chuẩn AXI4 Full. Sơ đồ kiến trúc được thể hiện tại Hình 4.14.



Hình 4.14: Sơ đồ khối vi kiến trúc L2 Cache

a. Tối ưu hóa cấu trúc

So với L1, L2 Cache tối ưu hóa việc quản lý cờ trạng thái (Valid/Dirty) bằng cách sử dụng mảng Flip-Flop độc lập thay vì SRAM. Cải tiến phần cứng này giúp giảm đáng kể độ trễ so sánh và tăng tốc độ truy xuất cho mức dung lượng lớn.

b. Cơ chế truyền tin AXI4

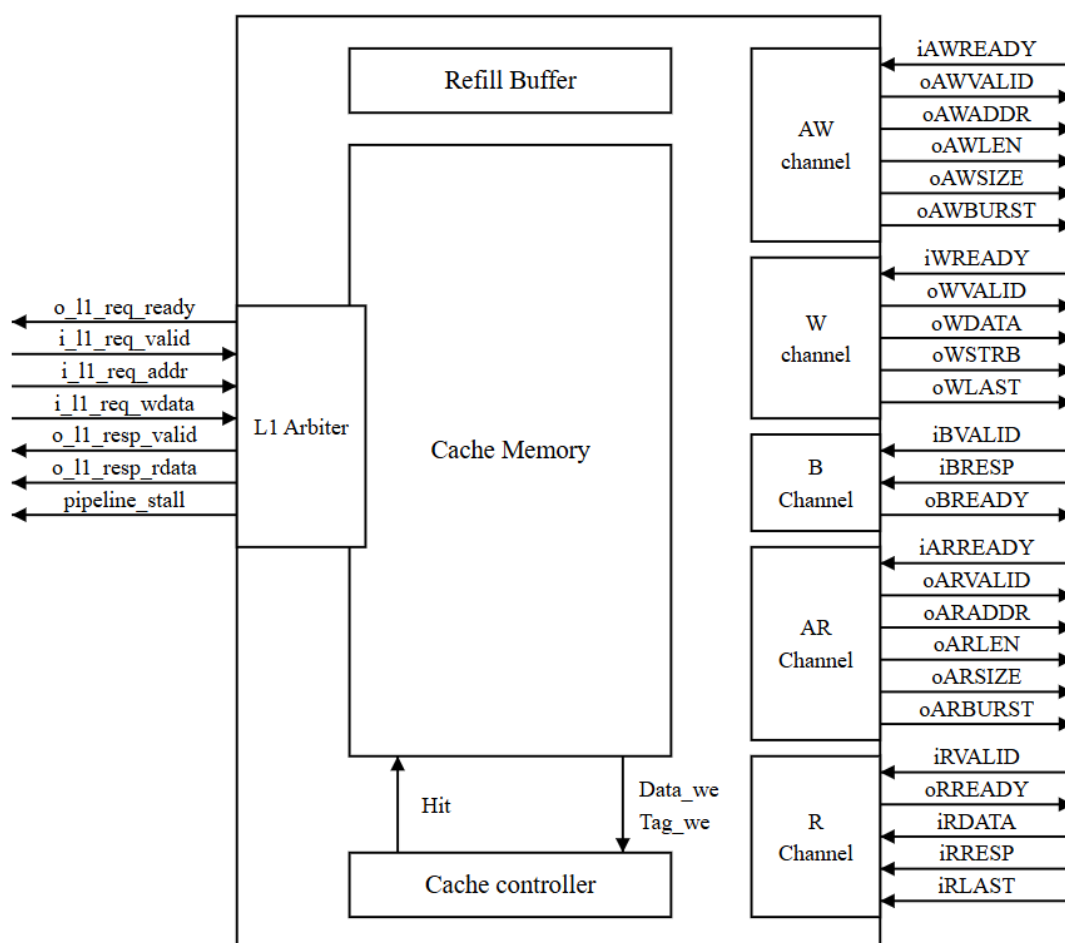
Bộ điều khiển truyền tin (Burst Controller) tại L2 hỗ trợ giao thức AXI4 giúp tối ưu hóa luồng dữ liệu thông qua ba cơ chế: tự động hóa Burst bằng bộ đếm nội bộ tự tính toán độ dài gói theo kích thước Cache Line và dừng khi nhận tín hiệu gói cuối; tổng hợp địa chỉ ghi trả trực tiếp từ thẻ định danh của dòng bị thay thế (Victim Way) để giảm truy xuất dư thừa; và đồng bộ Bus nhờ bộ đếm tự động tăng theo mỗi chu kỳ giao tiếp hợp lệ (handshake), đảm bảo dữ liệu truyền tải liên tục không gián đoạn.

c. Hoạt Động Của Máy chuyển đổi trạng thái (FSM)

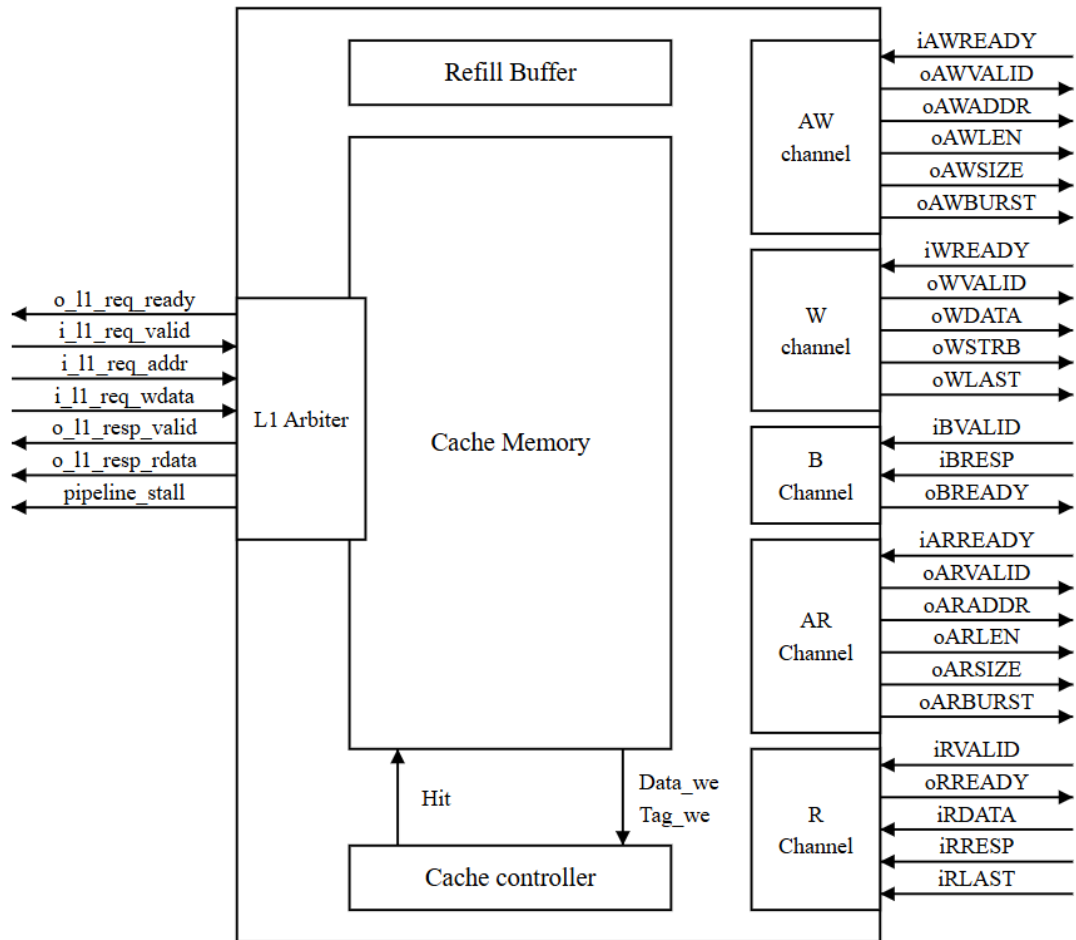
Khối điều khiển L2 vận hành các chu trình truy xuất và đồng bộ dữ liệu thông qua máy trạng thái (FSM) gồm 9 bước. Thiết kế này đảm bảo sự tuân thủ nghiêm ngặt các quy tắc bắt tay (handshake) của giao thức AXI4. Chức năng và điều kiện chuyển trạng thái được đặc tả chi tiết trong Bảng PIII.5.

d. Đặc Tả Giao Tiếp (Interface Specification)

Giao diện L2 Cache được chia thành hai nhóm độc lập: giao tiếp với L1 Arbiter (Upstream) và giao tiếp chuẩn AXI4 Full Master với bộ nhớ chính (Downstream). Sơ đồ kết nối vật lý được minh họa ở



Hình 4.15.

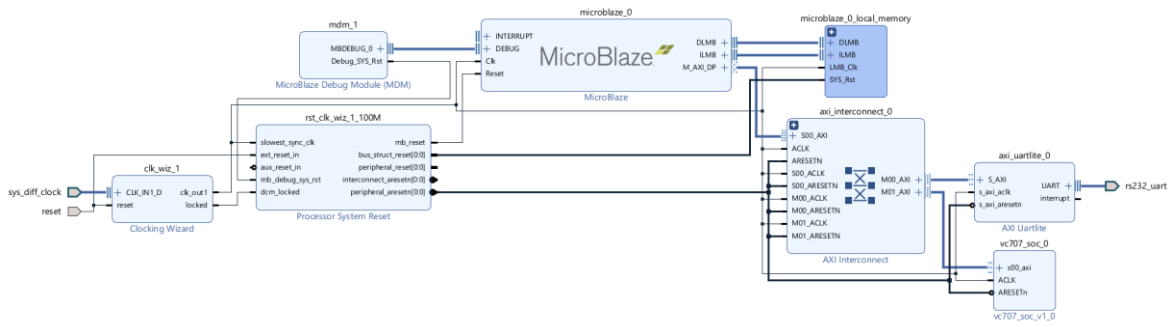


Hình 4.15: Tổng quan giao diện tín hiệu (Interface) của L2 Cache

Trong đó, giao tiếp giữa L2 Cache và L1 Cache sử dụng cơ chế bắt tay (Valid/Ready) nhằm tối ưu hóa độ trễ phản hồi, trong khi giao tiếp với hệ thống qua bus AXI4 sử dụng 5 kênh độc lập (AW, W, B, AR, R) để hỗ trợ truyền dữ liệu dạng khối (Burst Mode). Chi tiết về các trạng thái hoạt động và danh mục tín hiệu vào/ra (I/O) của các khối bộ đệm được mô tả cụ thể trong Bảng PIII.6.

4.5. Hiện thực SoC để nạp xuống kit Virtex 7

Quá trình hiện thực và đóng gói phần cứng được tiến hành trên phần mềm Vivado 2020.2 kết hợp Vitis 2022.2, nhằm tới nền tảng bo mạch FPGA Xilinx Virtex-7 VC707. Kiến trúc tổng thể của hệ thống (Block Design) sử dụng chuẩn giao tiếp AXI4 được minh họa chi tiết tại Hình 4.16.



Hình 4.16: Block design trên Vivado

Hệ thống được cấu thành từ ba phân hệ IP cốt lõi kết nối chặt chẽ với nhau. Tâm điểm của thiết kế là Khối lõi vi xử lý trung tâm (vc707_soc_0), một hệ thống vi xử lý đa lõi RV32IA tích hợp L2 Cache dùng chung và giao thức MOESI, được đóng gói hoàn chỉnh theo chuẩn AXI. Song song với đó, Khối xử lý phụ trợ (MicroBlaze & Local Memory) sử dụng lõi vi điều khiển mềm MicroBlaze và bộ nhớ BRAM để làm chip master phụ, đảm nhiệm vai trò nạp lệnh (bootloader) và giao tiếp với máy tính. Toàn bộ luồng dữ liệu giữa hai khối xử lý này với các thiết bị ngoại vi được điều phối và phân xử chính xác thông qua bộ chuyển mạch Khối định tuyến trung tâm (AXI Interconnect).

Bên cạnh các trực xử lý chính, hệ thống được hỗ trợ bởi các Khối ngoại vi và hạ tầng cơ sở (Infrastructure IPs) để vận hành ổn định. Trong đó, Clocking Wizard chịu trách nhiệm cung cấp xung nhịp 100MHz, còn Processor System Reset quản lý các tín hiệu reset đồng bộ an toàn cho toàn mạch. Cuối cùng, giao tiếp dữ liệu và gỡ lỗi phần cứng với máy tính được thực hiện trực tiếp thông qua khối truyền nhận nối tiếp AXI Uartlite và khối gỡ lỗi Debug Module (MDM).

CHƯƠNG 5. MÔ PHỎNG VÀ ĐÁNH GIÁ THIẾT KẾ

5.1. Kế hoạch kiểm thử tổng quan

5.1.1. Giai đoạn 1: Xác minh mức lỗi đơn (Single-Core Verification)

Quy trình xác minh gồm hai giai đoạn kiểm thử cốt lõi kết hợp cơ chế đánh giá tự động. Đầu tiên, lõi RV32IA được đánh giá mức tuân thủ tập lệnh cơ sở bằng bộ kiểm thử chuẩn riscv-tests, đồng thời xác minh pipeline và khả năng xử lý xung đột của Hazard Unit. Tiếp theo, bộ nhớ đệm độc lập được kiểm tra qua các giao dịch đọc/ghi cơ bản, giải thuật thay thế PLRU và các chính sách Write-back, Write Allocate. Toàn bộ quá trình dựa trên cơ chế Self-checking tích hợp trên Testbench: thay vì đối sánh với mô hình vàng bên ngoài, Testbench tự giám sát lệnh ecall và thanh ghi trạng thái CPU để trả kết quả nhanh.

5.1.2. Giai đoạn 2: Xác minh hệ thống đa lõi

Quy trình xác minh triển khai qua ba hạng mục chính. Trước hết, kiểm tra giao thức MOESI tập trung vào các kịch bản Đọc/Ghi trùng tại L1 để tránh giao dịch thừa trên Bus, đồng thời kiểm tra khả năng phản hồi và cập nhật trạng thái khi nhận yêu cầu snoop. Tiếp theo, hệ thống được đánh giá qua các bài toán mô phỏng hệ điều hành: Critical Section xác minh cơ chế khóa phần mềm; Producer-Consumer kiểm tra luồng dữ liệu qua L2 Cache dùng chung; và False Sharing khảo sát khả năng xử lý Ping-ponging dữ liệu giữa hai lõi. Cuối cùng, kiểm tra giao tiếp AXI4 Full đảm bảo chu kỳ bắt tay qua tín hiệu VALID/READY trên 5 kênh, cùng logic quản lý địa chỉ và cờ WLAST/RLAST trong các giao dịch Burst khi nạp hoặc ghi trả dòng đệm.

5.1.3. Hệ thống kiểm thử tự động tập lệnh RV32IA

Để xác minh Datapath và Control Unit, hệ thống chạy toàn bộ bài kiểm thử chuẩn RV32I và RV32A từ thư mục riscv-tests/isa, đòi hỏi môi trường mô phỏng linh hoạt cho phép tự động hóa hàng loạt hoặc gỡ lỗi chuyên sâu ở mức chu kỳ xung nhịp.

a. Công cụ và Môi trường Phối hợp

Hệ thống dùng luồng làm việc lai hiệu quả giữa Windows và WSL. Quy trình Auto-Testing dùng bộ công cụ Xilinx Vivado (xvlog, xelab, xsim) để biên dịch Verilog một lần và chạy kiểm thử hàng loạt ở tốc độ cao. Khi cần gỡ lỗi đơn lẻ, Icarus

Verilog (iverilog, vvp) kết hợp GTKWave được kích hoạt qua run_sim.ps1 để xuất .vcd phân tích tín hiệu chi tiết nhằm hỗ trợ người thiết kế quan sát rõ các thay đổi. Phân hệ Binary Translation trên WSL dùng Python 3 và RISC-V GNU Toolchain (elf2hex.py) biên dịch các file ELF của riscv-tests thành mã hex (.txt) để nạp vào bộ nhớ mô phỏng Verilog qua lệnh \$readmemh.

b. Cấu trúc Testbench và phân bổ bộ nhớ

Để cô lập Core 0 khi kiểm tra tập lệnh cơ sở trên kiến trúc lõi kép, Testbench tb_single_core phân vùng chặt chẽ không gian bộ nhớ hợp nhất: Core 0 (Core A) có vùng lệnh khởi tạo từ 32'h8000_0000, Core 1 (Core B) từ 32'h8000_4000. Để cô lập Core B, hệ thống xóa toàn bộ RAM về 32'h0 rồi nạp cứng mã 32'h0000006F (JAL x0, 0) vào địa chỉ bắt đầu của Core B, ép lõi này lặp vô hạn an toàn mà không phát sinh giao dịch AXI4. Cuối cùng, Testbench nạp mã Hex của bộ test vào vùng lệnh Core A để lõi sẵn sàng giải mã và thực thi độc lập trong môi trường mô phỏng có kiểm soát các điều kiện.

c. Cơ chế Can thiệp Pipeline và Đánh giá Trạng thái

Do CPU chưa hỗ trợ đầy đủ Exceptions và CSR, bộ giải mã xem lệnh ecall như NOP, buộc Testbench can thiệp trực tiếp vào Datapath để xác minh kết quả. Testbench theo dõi thanh ghi gp (x3), nơi riscv-tests ghi số thứ tự bài test, đồng thời giám sát tầng Decode để phát hiện lệnh ecall (opcode 32'h00000073) kết thúc mỗi bài test. Khi phát hiện opcode này cùng tín hiệu Flush không tích cực, hệ thống trễ 200 đơn vị thời gian để Writeback cập nhật xong thanh ghi, rồi trích xuất register_file.register[3]: giá trị 1 là PASS, ngược lại dịch bit (gp >> 1) để in số Test Case gây lỗi (FAIL). Một cơ chế Timeout an toàn cũng được thiết lập để tự động ép kết thúc mô phỏng và in cảnh báo kèm giá trị PC nếu quá 2.000.000 đơn vị thời gian không gặp ecall.

d. Quy trình tự động và xử lý lỗi

Luồng kiểm thử được tự động hóa qua kịch bản run_all_tests.ps1. Kịch bản duyệt thư mục kiểm thử để lọc các tệp thuộc rv32ui và rv32ua, loại trừ fence_i chưa được hỗ trợ. Mô phỏng dùng xsim ở chế độ batch để tiết kiệm thời gian, rồi dùng

regex match để tổng hợp kết quả Pass/Fail/Timeout vào test_results.log. Khi phát hiện lỗi, quy trình Traceback được kích hoạt để đối chiếu mã assembly qua file .dump tương ứng. Kết hợp run_sim.ps1 xuất dạng sóng trên GTKWave, người thiết kế có thể so sánh giá trị ALU thực tế với kết quả kỳ vọng để khoanh vùng thành phần lỗi.

5.1.4. Kế hoạch kiểm thử với lỗi kép

Để khẳng định tính ổn định và khả năng đồng bộ của hệ thống đa lõi, quy trình kiểm thử được chia thành ba nhóm bài toán trọng tâm:

a. Đánh giá qua bài toán mô phỏng hệ điều hành

Hạng mục này đánh giá hiệu năng và tính đúng đắn của hệ thống đa lõi qua ba kịch bản. Đầu tiên, bài toán Đồng bộ hóa (Critical Section) giả lập hai lõi cùng cập nhật một biến đếm toàn cục qua thuật toán khóa phần mềm như Peterson. Tiêu chí đánh giá là khả năng xử lý trạng thái M và E: khi Core 0 giữ khóa và cập nhật dữ liệu, dòng cache tại Core 1 phải bị ép về Invalid. Tiếp theo, mô hình Producer-Consumer cho Core 0 ghi dữ liệu vào bộ đệm chia sẻ, Core 1 đọc và xử lý, xác minh sự luân chuyển trạng thái từ M của Core 0 sang O hoặc S tại Core 1 qua tín hiệu Snoop và L2 dùng chung. Cuối cùng, kịch bản False Sharing kiểm tra hai lõi ghi vào các địa chỉ khác nhau nhưng cùng một Cache Line, qua đó kiểm chứng khả năng chịu tải của bộ điều khiển L2 khi liên tục phát và phản hồi các yêu cầu Invalidate/Snoop đan xen, nhằm kiểm soát hiện tượng Ping-ponging gây suy giảm hiệu năng.

b. Kiểm chứng chuyển trạng thái MOESI

Nhóm bài toán này kiểm tra bộ điều khiển Cache thực thi đúng sơ đồ máy trạng thái MOESI qua ba tình huống cốt lõi. Đầu tiên, ở Đọc trúng, khi một lõi yêu cầu đọc dữ liệu đã có trong L1 (ở M, O, E hoặc S), dữ liệu phải trả về ngay cho CPU mà không phát sinh giao dịch thừa trên Bus. Tiếp theo, ở Ghi trúng, nếu dòng cache đang ở Shared hoặc Owned, thao tác ghi từ CPU phải kích hoạt tín hiệu Invalidate đến các lõi còn lại để vô hiệu hóa bản sao cũ trước khi chuyển sang Modified. Cuối cùng, Snoop Request được xác minh bằng cách kích hoạt yêu cầu Read/Write từ Core 0 lên Bus, buộc Core 1 phản hồi đúng dữ liệu nếu đang giữ bản sao mới nhất ở M hoặc O, đồng thời tự động chuyển đổi trạng thái dòng cache theo đúng quy tắc MOESI.

c. Xác minh giao tiếp bộ nhớ qua chuẩn Bus AXI4 Full

Quy trình kiểm tra giao tiếp hệ thống xác minh tính ổn định của bus dữ liệu qua hai nội dung cốt lõi của AXI4 Full. Đầu tiên, kiểm tra chu kỳ bắt tay bằng cách giám sát các cặp tín hiệu VALID/READY trên 5 kênh AW, W, B, AR, R, đảm bảo bộ điều khiển L2 tuân thủ nghiêm ngặt giao thức, không mất dữ liệu hay treo hệ thống dù Master hoặc Slave chưa sẵn sàng, đồng thời tối ưu độ trễ qua quản lý linh hoạt các trạng thái chờ. Tiếp theo, hệ thống kiểm tra giao dịch Burst qua các yêu cầu nạp dòng (Cache Line Fill) và ghi xuất dòng (Write-back). Do một dòng Cache (512-bit) lớn hơn nhiều độ rộng Bus AXI (32-bit), bộ điều khiển phải cấp phát Burst với độ dài gói AxLEN tương ứng, đánh giá logic bộ đếm địa chỉ nội bộ và cơ chế quản lý cờ WLAST/RLAST, đảm bảo khối dữ liệu 512-bit được nạp hoặc ghi trọn vẹn trong một chu kỳ giao dịch, loại bỏ rủi ro tắc nghẽn băng thông và duy trì toàn vẹn dữ liệu khi tải cao nhất. Việc thực hiện các kiểm thử nghiêm ngặt này không chỉ giúp phát hiện sớm các lỗi tiềm ẩn trong logic phân xử bus mà còn đảm bảo hệ thống đạt được hiệu suất tối ưu khi vận hành ở tần số cao trên nền tảng phần cứng thực tế.

5.1.5. Kết quả kiểm thử lỗi đơn.

Bộ riscv-tests [19] được sử dụng nhằm xác minh tính đúng đắn chức năng và độ tuân thủ tập lệnh của lõi CPU, đảm bảo không xảy ra bất kỳ lỗi logic nào. Kết quả đạt chuẩn khẳng định pipeline hoạt động chính xác, xử lý an toàn xung đột (hazards) và giao tiếp bộ nhớ ổn định. Đây là bước tiền đề quan trọng, tạo nền tảng vững chắc để thực hiện các bước xác minh hệ thống đa lõi phức tạp hơn về sau.

a. Kết quả tổng quan

Mô phỏng được tự động hóa qua script biên dịch chéo (Windows/WSL). Hệ thống ghi nhận kết quả bằng cách can thiệp pipeline, đối soát trực tiếp thanh ghi gp (x3) khi Core 0 giải mã lệnh ecall. Các ngưỡng thời gian được thiết lập linh hoạt nhằm phát hiện và cô lập sớm những sai lệch logic phát sinh trong bước thực thi quá trình xử lý lệnh. Log hệ thống xác nhận Datapath và Control Unit hoạt động ổn định, vượt qua toàn bộ 51 bài kiểm tra cơ sở, tuân thủ tuyệt đối chuẩn RISC-V, được trích chi tiết tại Bảng 5.1.

Bảng 5.1: Kết quả thực thi bộ kiểm thử chuẩn RISC-V Test

Running rv32ui-p-slli ... Thành công: Đã tạo file Verilog/hexfile.txt từ riscv-
Running rv32ui-p-sub ... Thành công: Đã tạo file Verilog/hexfile.txt từ riscv-
tests/isa/rv32ui-p-sub!
PASS
Running rv32ui-p-sw ... Thành công: Đã tạo file Verilog/hexfile.txt từ riscv-
tests/isa/rv32ui-p-sw!
PASS
Running rv32ui-p-xor ... Thành công: Đã tạo file Verilog/hexfile.txt từ riscv-
tests/isa/rv32ui-p-xor!
PASS
Running rv32ui-p-xori ... Thành công: Đã tạo file Verilog/hexfile.txt từ riscv-
tests/isa/rv32ui-p-xori!
PASS

Summary:
Total: 51 Passed: 51 Failed: 0 Timeout: 0
Detailed log saved to: test_results.log

Tổng quan kết quả thực thi bộ kiểm thử chuẩn RISC-V (RV32I) được thống kê cụ thể như sau:

- Tổng số bài kiểm thử: **51**
- Số lượng đạt (Passed): **51**
- Số lượng lỗi (Failed): 0
- Số lượng quá thời gian (Timeout): 0
- Tỷ lệ thành công (Pass Rate): **100.0%**

b. Kết quả chi tiết

Các bài kiểm thử được phân loại theo nhóm lệnh nhằm minh họa rõ ràng khả năng xử lý của từng khối logic phần cứng, được thống kê chi tiết tại Bảng 5.2.

Bảng 5.2: Tổng hợp kết quả kiểm thử chức năng tập lệnh cơ sở (RV32I)

STT	Nhóm tập lệnh	Các bài test tiêu biểu	Lĩnh vực đánh giá	Trạng thái
1	Chỉ thị cơ bản	simple	Luồng khởi tạo và hoạt động cơ bản của Pipeline	PASS
2	Phép toán số học	add, addi, sub	Khối ALU cộng/trừ và xử lý tràn số	PASS
3	Phép toán Logic	and, andi, or, ori, xor, xori	Khối ALU thao tác bit logic	PASS
4	Phép dịch Bit	sll, slli, sra, srai, srl, srli	Khối Barrel Shifter và bảo toàn bit dấu	PASS
5	Phép so sánh	slt, slti, sltu, sltiu	So sánh có dấu và không dấu	PASS
6	Rẽ nhánh có điều kiện	beq, bne, bge, bgeu, blt, bltu	Khối dự đoán/tính toán địa chỉ nhảy và xử lý Hazard	PASS
7	Rẽ nhánh vô điều kiện	jal, jalr	Lưu địa chỉ quay về và chuyển hướng luồng PC	PASS
8	Truy xuất bộ nhớ	lw, lh, lhu, lb, lbu, sw, sh, sb	Giao tiếp Load/Store định tuyến và căn lề byte (Alignment)	PASS

5.1.6. Kết quả kiểm thử lỗi kép.

Để đánh giá cơ chế đồng bộ phần cứng trên hệ thống lỗi kép, năm kịch bản kiểm thử đã được triển khai dựa trên các tập lệnh nguyên tử RISC-V. Kịch bản đầu tiên (mã nguồn tại Phụ lục I) sử dụng cặp lệnh `lr.w` (Load-Reserved) và `sc.w` (Store-Conditional) để đồng bộ CPU A và CPU B nhằm bảo vệ biến đếm tại ô nhớ `0x204`. Với khóa Mutex khởi tạo bằng 0 tại ô nhớ `0x200`, mỗi CPU dùng lệnh `lr.w` để đọc và giám sát độc quyền, sau đó dùng `sc.w` cố gắng ghi giá trị 1 để chiếm khóa. Nếu ô nhớ chưa bị can thiệp, lệnh `sc.w` thành công (trả về 0); ngược lại, CPU rơi vào trạng thái chờ quay vòng (spin-lock). Khi chiếm khóa thành công, CPU bước vào vùng tranh chấp, tăng biến đếm lên 1, rồi ghi giá trị 0 (`sw x0`) để giải phóng khóa. Quy trình này lặp lại chính xác 10 lần cho mỗi CPU. Kết quả mô phỏng (Phụ lục II) cho thấy biến đếm đạt chính xác giá trị `0x14` (tương đương 20), chứng minh tính đúng đắn của giải thuật loại trừ tương hỗ trên hệ thống đa lõi.

Kịch bản thứ hai (Phụ lục I) áp dụng mô hình Producer - Consumer trên bộ đệm đơn `0x200`, dùng lệnh nguyên tử `amoswap.w` để điều khiển cờ `empty` (`0x240`, khởi tạo 1) và cờ `full` (`0x280`, khởi tạo 0). Mỗi vòng lặp, CPU A dùng `amoswap.w` trao đổi giá trị 0 vào cờ `empty`; nếu cờ cũ là 1, CPU A ghi dữ liệu vào `0x200` rồi đặt cờ `full` lên 1. Song song đó, CPU B liên tục kiểm tra bằng `amoswap.w`; khi cờ `full` lên 1, CPU B lấy dữ liệu xử lý cộng dồn, xóa cờ `full` và dựng lại cờ `empty` lên 1. Quá trình luân phiên này lặp lại 10 lần, đảm bảo truyền dữ liệu toàn vẹn, không ghi đè hay đọc trùng, được xác thực qua biểu đồ xung tại Phụ lục II.

Kịch bản thứ ba kế thừa mô hình trên nhưng dùng các lệnh nguyên tử `amoand.w` và `amoor.w` để quản lý cờ (Phụ lục I). Các cờ `empty` (`0x240`) và `full` (`0x280`) được kiểm tra bằng `amoand.w` với thanh ghi `x0`, giúp hệ thống vừa đọc giá trị cũ rẽ nhánh, vừa đồng thời xóa cờ về 0 để khóa tài nguyên. Khi `empty` bằng 1, CPU A ghi dữ liệu mới vào `0x200` rồi dùng `amoor.w` dựng cờ `full` lên 1. Ngược lại, CPU B đợi cờ `full` lên 1 qua lệnh `amoand.w`, trích xuất dữ liệu xử lý, rồi dùng lệnh `amoor.w` dựng lại cờ `empty` lên 1. Chu kỳ phối hợp logic luân phiên 10 lần này đảm bảo cờ trạng thái cập nhật chính xác, nhất quán theo kết quả mô phỏng ở Phụ lục II.

Kịch bản thứ tư ứng dụng các lệnh so sánh nguyên tử amomin.w và amomax.w để quản lý cờ trạng thái (Phụ lục I). Cờ empty (0x240) và full (0x280) được kiểm tra bằng amomin.w với thanh ghi x0, vừa đọc giá trị cũ vừa tự động ghi đè giá trị nhỏ nhất (đưa cờ về 0) để khóa trạng thái. Khi empty bằng 1, CPU A ghi dữ liệu vào 0x200, rồi dùng amomax.w (với thanh ghi giá trị 1) dựng cờ full lên 1. Song song, CPU B chờ cờ full lên 1 qua amomin.w, lấy dữ liệu cộng dồn tích lũy, rồi dùng amomax.w đặt lại cờ empty về 1 cho chu kỳ kế tiếp của CPU A. Sự phối hợp luân phiên này hoạt động chuẩn xác, minh chứng qua kết quả tại Phụ lục II.

Cuối cùng, kịch bản thứ năm sử dụng tổ hợp lệnh toán học và logic amoadd.w và amoxor.w để điều khiển đồng bộ (Phụ lục I). Các cờ empty và full được kiểm tra bằng lệnh amoadd.w (cộng giá trị 0) để đọc trạng thái cũ mà không làm thay đổi giá trị gốc. Khi empty bằng 1, CPU A dùng amoxor.w đảo empty về 0 để khóa tài nguyên, ghi dữ liệu mới vào 0x200, rồi tiếp tục dùng amoxor.w đảo cờ full lên 1. Ngược lại, CPU B chờ cờ full lên 1 qua amoadd.w, dùng amoxor.w đảo cờ này về 0 để đọc quyền xử lý, và cuối cùng khôi phục cờ empty lên 1. Chu kỳ 10 vòng lặp này hoạt động ổn định, đạt đầu ra đúng dự đoán toán học (Phụ lục II).

5.1.7. Kết quả tổng hợp và thực thi thiết kế

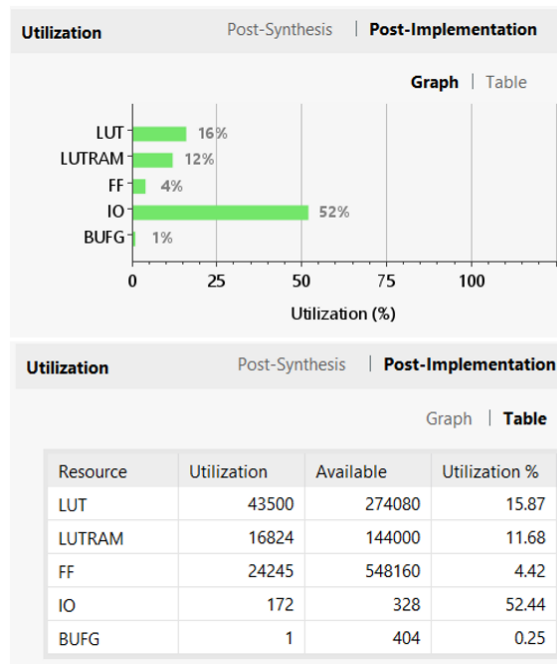
Quá trình tổng hợp và thực thi phần cứng trên phần mềm Vivado bắt đầu với việc thiết lập chu kỳ xung nhịp 10ns (tương đương tần số 100 MHz). Kết quả tổng hợp timing (Hình 5.1) khẳng định mạch vận hành an toàn, ổn định ở tần số hoạt động thiết kế 100 MHz, với WNS đạt dương 1,428 ns và hoàn toàn không có đường dẫn vi phạm (0 Failing Endpoints). Slack dương cho thấy thiết kế còn dư địa thời gian, tương ứng với tần số tối đa lý thuyết khoảng 116 MHz theo ước tính của công cụ tổng hợp.

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.428 ns	Worst Hold Slack (WHS): 0.010 ns	Worst Pulse Width Slack (WPWS): 4.468 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 171687	Total Number of Endpoints: 171687	Total Number of Endpoints: 45450

All user specified timing constraints are met.

Hình 5.1: Kết quả tổng hợp timing toàn hệ thống

Theo Hình 5.2, hệ thống tiêu thụ tài nguyên FPGA ở mức thấp với 43.500 LUT (15,87%) và 24.245 FF (4,42%). Tỷ lệ chân I/O chiếm 52,44% do yêu cầu kết nối AXI4 và các ngoại vi. Nhìn chung, mức sử dụng này rất tối ưu, đảm bảo không gian rộng rãi để nâng cấp và mở rộng hệ thống sau này.



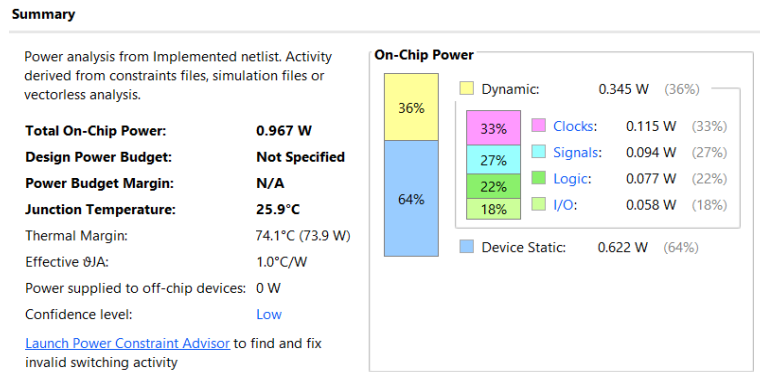
Hình 5.2: Tổng hợp tài nguyên sử dụng của toàn hệ thống

Bảng chi tiết tài nguyên (Hình 5.3) khẳng định tính đối xứng trong thiết kế khi hai lõi xử lý chiếm lượng LUT tương đương (xấp xỉ 16.000 LUT mỗi lõi). Ngoài ra, tỷ trọng tài nguyên thấp của khối L2 Cache (9.183 LUT) và bộ điều phối Coherence (1.363 LUT) cho thấy hiệu quả tối ưu hóa cao trong thiết kế phần cứng của hệ thống.

Name	CLB LUTs (274080)	CLB Registers (548160)	CARRY8 (34260)	F7 Muxes (137040)	F8 Muxes (68520)	CLB (34260)	LUT as Logic (274080)	LUT as Memory (144000)	Bonded IOB (328)	HPIOB_M (96)	HPIOB_S (96)	HPIOB_SINGL (16)	GLOBAL CLOCK BUFFERS (404)
dual_core	43500	24245	116	2518	576	9298	26676	16824	172	82	82	8	1
> core_0 (single_core)	16082	9630	56	1099	224	5024	9768	6314	0	0	0	0	0
> core_1 (single_core_parameterized0)	16675	9636	56	1099	224	5387	10561	6314	0	0	0	0	0
> u_cache_l2 (cache_L2)	9183	3358	4	320	128	2306	4987	4196	0	0	0	0	0
> u_coherence (cache_coherence)	1363	1621	0	0	0	1388	1363	0	0	0	0	0	0

Hình 5.3: Phân tách tài nguyên theo khối thành phần

Về đặc tính năng lượng, kết quả phân tích tại Hình 5.4 cho thấy hệ thống vận hành cực kỳ hiệu quả với tổng công suất tiêu thụ trên chip chỉ đạt 0,967 W. Mức tiêu thụ thấp này giúp duy trì nhiệt độ vận hành của lõi ở ngưỡng an toàn (25,9 °C), đảm bảo độ ổn định nhiệt mà không yêu cầu các giải pháp tản nhiệt phức tạp.



Hình 5.4: Báo cáo tổng hợp năng lượng tiêu thụ toàn hệ thống

Sau khi chạy tất cả Testcase, nhóm đã tính được các thông số trong Bảng 5.3.

Bảng 5.3: Tổng hợp thông số hiệu năng xử lý và bộ nhớ đệm L1, L2

Thành phần	Thông số	Core A / Cache L1 (Core A)	Core B / Cache L1 (Core B)	Cache L2 (Shared)
Hiệu năng Xử lý	Số Lệnh (Instructions)	1377	2127	—
	Tổng số chu kỳ (Total Cycles)	7487	8147	—
	CPI	5.44	3.52	—
Hiệu năng Bộ đệm	Lượt truy cập (Access)	2136	2395	252
	Lượt trượt (Miss)	147	173	88
	Lượt trúng (Hit)	1989	2222	164
	Tỷ lệ trúng (Hit Rate)	93.12%	92.78%	65.08%

Dựa trên Bảng 5.3, bộ nhớ đệm L1 của cả hai lõi đạt hiệu suất trúng đích rất cao (trên 92%), trong khi tầng L2 chia sẻ đạt 65,08%, phản ánh đúng tính chất phân cấp của

cấu trúc phần cứng. Về hiệu năng xử lý, Core B thực thi khối lượng lệnh lớn hơn (2127 lệnh) nhưng tối ưu hơn với chỉ số CPI đạt 3,52, thấp hơn so với 5,44 của Core A. Sự chênh lệch về tổng số chu kỳ (7487 so với 8147) cũng chỉ ra sự phân phối khối lượng công việc chưa đồng đều giữa hai lõi, khiến thời gian hoàn thành của toàn hệ thống bị giới hạn bởi lõi thực thi lâu hơn là Core B.

Bảng 5.4 tổng hợp các tiêu chí kỹ thuật cốt lõi nhằm đánh giá khách quan hiệu quả thiết kế của khóa luận so với các công trình nghiên cứu trong và ngoài nước hiện có. Nhìn chung, khóa luận kế thừa và phát triển thành công nhiều đặc tả vi kiến trúc phức tạp.

Bảng 5.4: So sánh kết quả đạt được với các công trình khác

Các yếu tố so sánh	Khóa luận này	Trong nước		Quốc tế	
		Tác giả	Tác giả	Tác giả	Tác giả
		N. Q. T. An D. H. Tuấn [1]	T. T. H. Nam N. Đ. D. Khang [3]	E. Humblet [11]	X. Guo [21]
Kiến trúc tập lệnh	RV32IA	RV32I	RV32IMF	RV64V	RV64GC
Cơ chế xử lý rẽ nhánh	Dự đoán động	N/A	Dự đoán động	Dự đoán động	Dự đoán động
Kiến trúc	4-Way Set Associative	4-Way Set Associative	4-Way Set Associative	4-Way Set Associative	4-Way Set Associative

Chính sách thay thế	PLRUt	PLRUt	LRU	N/A	LRU
Đồng bộ hóa	MOESI	MOESI	Không	MESI	MESI
Số lượng vi xử lý	2-Core	2-Core	8-Core	4-Core	8-Core
Tần số hệ thống	116MHz	124MHz	106MHz	75MHz	89 MHz
LUTs	43500	8598	190775	1400560	17420
LUTRAM	16824	390	6192	4629	N/A
FF	24245	6732	204124	409619	6683
BRAM	2	108.5	0	294	N/A
Power	0.967 (W)	0.625 (W)	3.85 (W)	1.782(W)	N/A
Board	Virtex-7 VC707	Virtex-7 VC707	Virtex-7 VC707	Xilinx Alveo U280	Xilinx Kintex 7

Về vi kiến trúc lõi, hệ thống đề xuất hỗ trợ tập lệnh RV32IA tích hợp khối xử lý lệnh nguyên tử (Atomic), hoàn thiện hơn kiến trúc RV32I cơ bản của N. Q. T. An [1]. Cơ chế dự đoán rẽ nhánh động giúp pipeline giảm thiểu chu kỳ ngưng đọng (stall clock), mang lại hiệu năng linh hoạt tiệm cận các nghiên cứu quốc tế của E. Humblet [11] và

X. Guo [21], trong khi thiết kế của N. Q. T. An [1] chưa có tính năng này. Đối với cấu trúc bộ nhớ đệm, hệ thống triển khai mô hình cache 4-Way Set Associative tương tự các tác giả trong nước và quốc tế. Tuy nhiên, điểm cải tiến về phần cứng nằm ở việc kết hợp giải thuật thay thế PLRUt. So với giải thuật LRU truyền thống trong nghiên cứu của T. T. H. Nam [3] và X. Guo [21], giải thuật PLRUt giúp tiết kiệm tài nguyên mạch logic và rút ngắn đường trễ định thời tới hạn (critical path delay), từ đó tăng tốc độ truy xuất dữ liệu từ các tầng cache. Về cơ chế đồng bộ đa lỗi (Cache Coherence), hệ thống lỗi kép đề xuất ứng dụng giao thức MOESI Snoopy, giải quyết triệt để bài toán nhất quán dữ liệu phần cứng mà kiến trúc 8 lỗi của T. T. H. Nam [3] chưa hỗ trợ. So với giao thức MESI trong nghiên cứu của E. Humblet [11] và X. Guo [21], việc lựa chọn giao thức MOESI giúp các lỗi chia sẻ trực tiếp dữ liệu ở trạng thái chỉnh sửa (Owner) mà không cần ghi ngược (write-back) về bộ nhớ chính quá sớm, từ đó giảm lưu lượng và tối ưu hóa băng thông truyền thông nội chip. Cuối cùng, kết quả tổng hợp trên FPGA Virtex-7 VC707 cho thấy hệ thống đề xuất đạt tần số tối đa lý thuyết khoảng 116 MHz (tính từ slack dương khi tổng hợp với constraint 100 MHz), cạnh tranh với nghiên cứu của N. Q. T. An [1] (124 MHz) và cao hơn cấu hình của E. Humblet [11] (75 MHz) hay X. Guo [21] (89 MHz). Về tài nguyên, để vận hành giao thức MOESI, hệ thống tiêu tốn 43.500 LUTs, 24.245 FFs với mức công suất 0.967 W. Các chỉ số này chứng minh tính hiệu quả của đề tài trong việc cân bằng giữa hiệu năng xung nhịp, công suất tiêu thụ và chi phí phần cứng cho các hệ thống SoC nhúng.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Sau quá trình nghiên cứu và thực hiện đồ án, nhóm đã đạt được các kết quả thiết kế và hiện thực phần cứng toàn diện cho hệ thống SoC đa lõi. Về mặt kiến trúc, nhóm đã hiện thực thành công lõi vi xử lý RISC-V RV32IA cấu trúc pipeline 6 tầng, tích hợp bộ dự đoán rẽ nhánh động và cơ chế xử lý xung đột dữ liệu, đồng thời hỗ trợ phần cứng cho tập lệnh Atomic (RV32A) thông qua khối AMO ALU và cơ chế đặt trước cho cặp lệnh LR/SC. Hệ thống sở hữu cấu trúc bộ nhớ đệm phân cấp 4-Way Set Associative bao gồm L1 I/D-Cache (16-Set) độc lập trên từng lõi và L2 Cache (64-Set) dùng chung, vận hành dựa trên giải thuật thay thế PLRUt 3-bit cùng chính sách Write-back và cấp phát Read/Write Allocate. Đặc biệt, tính nhất quán dữ liệu giữa hai lõi được duy trì nghiêm ngặt nhờ giao thức đồng bộ MOESI Snoopy tích hợp tại L1 D-Cache.

Toàn bộ hệ thống từ các lõi, bộ nhớ đệm đến bộ điều phối coherence đều được kết nối chặt chẽ thông qua chuẩn giao tiếp AXI4 Full. Quy trình xác minh đồng bộ (co-verification) với các kịch bản testcase phức tạp đã chứng minh hệ thống hoạt động đúng chức năng, hoàn toàn không xảy ra hiện tượng Race Condition hay Deadlock. Cuối cùng, thiết kế đã được tổng hợp thành công trên nền tảng AMD/Xilinx Vivado đạt mốc thời gian tối ưu với WNS = 1,428 ns (Failing Endpoint = 0), tiêu tốn tổng tài nguyên gồm 43.500 LUT (15,87%), 16.824 LUTRAM (11,68%), 24.245 FF (4,42%), và vận hành ổn định ở mức công suất tiêu thụ 0,967 W tại nhiệt độ Junction 25,9 °C.

6.2. Ưu điểm và hạn chế của thiết kế

6.2.1. Ưu điểm

Hệ thống đã đạt được các mục tiêu thiết kế cốt lõi nhờ vào việc tối ưu hóa cả về mặt kiến trúc lẫn vi kiến trúc phần cứng. Trước hết, bài toán đồng bộ hóa dữ liệu trong môi trường đa luồng được giải quyết triệt để thông qua việc hỗ trợ tập lệnh RV32A kết hợp cùng giao thức MOESI ở mức phần cứng, giúp duy trì tính nhất quán dữ liệu nghiêm ngặt giữa các lõi xử lý. Bên cạnh đó, hiệu năng tổng thể của hệ thống

được cải thiện đáng kể nhờ vào vi kiến trúc pipeline 6 tầng tiên tiến, hoạt động phối hợp nhịp nhàng với hệ thống phân cấp bộ nhớ đệm (L1/L2) để giảm thiểu độ trễ truy xuất. Cuối cùng, toàn bộ hệ thống đã được tổng hợp và xác minh ổn định trên bo mạch, đáp ứng hoàn toàn các chỉ tiêu kỹ thuật và tiêu chí thiết kế đặt ra ban đầu.

6.2.2. Hạn chế

Hệ thống hiện tại vẫn tồn tại một số hạn chế về mặt vi kiến trúc và tính năng cần được cải tiến. Đầu tiên là hiện tượng nghẽn cổ chai tại L2 Cache do bộ phân xử (Arbiter) chỉ cho phép một L1 truy cập L2 tại một thời điểm, làm giảm đáng kể băng thông khi cả hai lõi cùng gặp lỗi trượt bộ đệm (Cache Miss). Tiếp theo, cơ chế đồng bộ và vi kiến trúc này bị giới hạn số lõi, mới chỉ hỗ trợ cố định cấu hình tối đa là hệ thống 2 lõi (Dual-core). Về mặt xử lý lệnh, lõi vi xử lý vẫn thiếu tập lệnh hệ thống, chưa hỗ trợ các lệnh đặc quyền (ecall, ebreak) cũng như các thanh ghi trạng thái (CSR). Cuối cùng, do thiếu các khối chức năng ngoại vi quan trọng như Bộ quản lý bộ nhớ (MMU) và Truy cập bộ nhớ trực tiếp (DMA), hệ thống hiện vẫn chưa có khả năng triển khai bộ nhớ ảo hay vận hành các hệ điều hành nhúng phức tạp. Hệ quả là, kiến trúc hiện tại chưa thể hỗ trợ môi trường thực thi đa nhiệm, đặc biệt là khả năng chuyển ngữ cảnh (context switching) và luân chuyển tiến trình (process migration) giữa các lõi để khai thác triệt để năng lực xử lý song song.

6.3. Hướng phát triển

Để khắc phục các hạn chế hiện tại và mở rộng khả năng ứng dụng, định hướng phát triển tiếp theo của đề tài sẽ tập trung vào việc nâng cấp toàn diện vi kiến trúc và bổ sung các tính năng cốt lõi. Trước hết, để giải quyết triệt để điểm nghẽn tại bộ đệm, hệ thống cần cải tiến L1 Arbiter hoặc thiết kế L2 Cache dưới dạng đa cổng (Multi-port) nhằm cho phép các lõi truy xuất đồng thời. Đồng thời, khi mở rộng quy mô lên N lõi ($N > 2$), giao thức đồng bộ sẽ được chuyển đổi từ dạng Snooping sang kiến trúc Directory-based. Song song với đó, đề tài hướng tới việc nghiên cứu và tham khảo các chuẩn giao tiếp đồng nhất bộ nhớ chuyên dụng trong công nghiệp, tiêu biểu như kiến trúc AMBA ACE (AXI Coherency Extensions) hoặc AMBA CHI (Coherent Hub Interface) của ARM. Việc áp dụng các chuẩn này không chỉ tối ưu hiệu năng

giao tiếp trên bus mà còn giúp vi xử lý dễ dàng tương thích với các IP lõi khác. Về năng lực tính toán, thiết kế dự kiến tích hợp thêm các tập lệnh mở rộng chuẩn M (Nhân-chia), F/D (Dấu phẩy động) và hoàn thiện hệ thống thanh ghi trạng thái (CSR) nhằm hỗ trợ cơ chế xử lý ngắt và ngoại lệ. Bên cạnh đó, kiến trúc SoC cũng sẽ được hoàn thiện thông qua việc tích hợp Bộ quản lý bộ nhớ (MMU), Truy cập bộ nhớ trực tiếp (DMA) cùng các chuẩn giao tiếp ngoại vi (UART, SPI), tạo tiền đề phần cứng cần thiết để triển khai bộ nhớ ảo và vận hành các hệ điều hành nhúng phức tạp. Cuối cùng, hệ thống sẽ được đánh giá thông qua các kịch bản kiểm thử luân chuyển tiến trình; tiêu biểu là việc mô phỏng một tiến trình đang thực thi ở lõi 0 bị ngắt và được hệ điều hành điều phối sang lõi 1. Quá trình này đòi hỏi lõi 1 phải truy xuất và xử lý tron tru các dữ liệu chưa cập nhật xuống bộ nhớ chính (dirty data) nằm trong cache của lõi 0, qua đó chứng minh năng lực duy trì tính nhất quán dữ liệu (Cache Coherence) toàn vẹn ở mức hệ thống.

TÀI LIỆU THAM KHẢO

Tiếng Việt

- [1] N. Q. T. An và D. H. Tuấn, "Thiết kế bộ điều khiển đồng bộ cache cho hệ thống đa vi xử lý dựa trên lõi CPU RISC-V RV32I," 2025.
- [2] C. T. Bình và Đ. P. Hùng, "Thiết kế và hiện thực lõi vi xử lý RISC-V RV32IF hỗ trợ 4-Way Set Associative Cache và bộ tạo số ngẫu nhiên thực trên FPGA," 2023.
- [3] T. T. H. Nam và N. Đ. D. Khang, "Hiện thực một NoC (Network on Chip) sử dụng lõi vi xử lý RISC-V RV32IMF thông qua giao thức Tilelink trên FPGA," 2024.
- [4] K. T. Tài và P. T. Đức, "Thiết kế bộ vi xử lý 2 lõi dựa trên kiến trúc tập lệnh RISC-V RV32ICMFA," 2025.
- [5] T. Q. Trường và L. P. N. Nam, "Thiết kế và hiện thực bộ vi xử lý RISC-V 32 bit sử dụng kiến trúc Superscalar hỗ trợ bộ điều khiển Cache Associative 4-way và đơn vị quản lý bộ nhớ trên FPGA," 2022.

Tiếng Anh

- [6] Arm Limited, AMBA AXI and ACE Protocol Specification AXI3, AXI4, AXI5, ACE and ACE5, Tài liệu số ARM IHI 0022F.b, Cambridge, UK, 2017.
- [7] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition. Cambridge, MA, USA: Morgan Kaufmann, 2018.
- [8] D. Emil, M. Hamdy, và G. Nagib, "Development an efficient AXI-interconnect unit between set of customized peripheral devices and an implemented dual-core RISC-V processor," The Journal of Supercomputing, vol. 79, no. 15, pp. 17000–17019, 2023.

- [9] D. J. Sorin, M. D. Hill, and D. A. Wood, A Primer on Memory Consistency and Cache Coherence. San Rafael, CA, USA: Morgan & Claypool Publishers, 2011.
- [10] Daman Preet Kaur, V. Sulochana, “Design and Implementation of Cache Coherence Protocol for High-Speed Multiprocessor System,” 2nd IEEE International Conference on Power Electronics, Intelligent Control and Energy Systems (ICPEICES), India, 2018.
- [11] E. Humblet, "Verification and Characterization of Multicore Vector Processors Enhanced for Low-precision Convolutional Layers Through FPGA Emulation," Master's thesis, Dept. Elect. Eng., Polytechnique Montréal, Montreal, QC, Canada, 2024. [Online]. Available: <https://publications.polymtl.ca/58766/>.
- [12] J. Archibald and J. L. Baer, "An evaluation of cache coherence protocols for shared-bus multiprocessors," ACM Transactions on Computer Systems (TOCS), vol. 4, no. 4, pp. 273–298, 1986.
- [13] J. L. Hennessy và D. A. Patterson, Computer Architecture: A Quantitative Approach, 5th ed. Waltham, MA, USA: Morgan Kaufmann, 2011.
- [14] J. V. R. de A. Roque, “Development Environment for a RISC-V processor: Cache,” M.S. thesis, Dept. Elect. Comput. Eng., Instituto Superior Técnico, Universidade de Lisboa, Lisboa, Portugal, 2021.
- [15] M. Majiid, "RISC-V RV64IMAC RTL Design using SystemVerilog HDL," Báo cáo kỹ thuật, 2022.
- [16] M. S. Papamarcos và J. H. Patel, "A low-overhead coherence solution for multiprocessors with private cache memories," trong Proceedings of the 11th Annual International Symposium on Computer Architecture (ISCA), 1984, pp. 348–354.
- [17] P. Sweazey and A. J. Smith, "A class of compatible cache consistency protocols and their support by the IEEE Futurebus," in Proceedings of the

- 13th Annual International Symposium on Computer Architecture (ISCA), 1986, pp. 414–423.
- [18] RISC-V International, The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture, Version 20250508, May 2025.
 - [19] RISC-V Software Source, "riscv-tests," GitHub repository. [Online]. Available: <https://github.com/riscv-software-src/riscv-tests>.
 - [20] S. L. Harris và D. Harris, Digital Design and Computer Architecture, RISC-V Edition. Cambridge, MA, USA: Morgan Kaufmann, 2022.
 - [21] X. Guo, "Efficient virtual cache coherency for multicore systems and accelerators," University of Cambridge, Computer Laboratory, Cambridge, UK, Tech. Rep. UCAM-CL-TR-979, Feb. 2023. Available: <https://www.cl.cam.ac.uk/techreports/>.

PHỤ LỤC I. NGUỒN KỊCH BẢN KIỂM THỬ

Testcase 1: Đồng bộ Mutex (LR/SC)

CPU A	CPU B
<pre> .text .globl _start _start: addi x5, x0, 0x200 addi x6, x0, 0x204 addi x7, x0, 10 addi x28, x0, 1 loop: beq x7, x0, done acquire_lock: lr.w x29, (x5) bne x29, x0, acquire_lock sc.w x30, x28, (x5) bne x30, x0, acquire_lock critical_section: lw x31, 0(x6) addi x31, x31, 1 sw x31, 0(x6) release_lock: sw x0, 0(x5) addi x7, x7, -1 jal x0, loop done: addi x26, x0, 1 lw x11, 0(x6) end_spin: jal x0, end_spin </pre>	<pre> .text .globl _start _start: addi x5, x0, 0x200 addi x6, x0, 0x204 addi x7, x0, 10 addi x28, x0, 1 loop: beq x7, x0, done acquire_lock: lr.w x29, (x5) bne x29, x0, acquire_lock sc.w x30, x28, (x5) bne x30, x0, acquire_lock critical_section: lw x31, 0(x6) addi x31, x31, 1 sw x31, 0(x6) release_lock: sw x0, 0(x5) addi x7, x7, -1 jal x0, loop done: addi x26, x0, 1 lw x11, 0(x6) end_spin: jal x0, end_spin </pre>

Testcase 2: AMOSWAP (Producer - Consumer)

CPU A	CPU B
<pre> .section .text.init .globl _start _start: j main .text main: addi x10, x0, 0x200 addi x17, x10, 0x40 addi x18, x10, 0x80 addi x15, x0, 1 sw x15, 0(x17) sw x0, 0(x18) addi x11, x0, 0 addi x12, x0, 10 addi x16, x0, 1 producer_loop: beq x11, x12, end_test wait_empty: amoswap.w x14, x0, (x17) beq x14, x0, wait_empty addi x13, x11, 100 sw x13, 0(x10) amoswap.w x0, x16, (x18) addi x11, x11, 1 j producer_loop end_test: addi x26, x0, 1 spin: j spin </pre>	<pre> .section .text.init .globl _start _start: j main .text main: addi x10, x0, 0x200 addi x17, x10, 0x40 addi x18, x10, 0x80 addi x11, x0, 0 addi x12, x0, 10 addi x16, x0, 1 addi x20, x0, 0 consumer_loop: beq x11, x12, end_test wait_full: amoswap.w x14, x0, (x18) beq x14, x0, wait_full lw x13, 0(x10) add x20, x20, x13 amoswap.w x0, x16, (x17) addi x11, x11, 1 j consumer_loop end_test: addi x26, x0, 1 spin: j spin </pre>

Testcase 3: AMOAND/AMOOR (Producer - Consumer)

CPU A	CPU B
<pre> .section .text.init .globl _start _start: j main .text main: addi x10, x0, 0x200 addi x17, x10, 0x40 addi x18, x10, 0x80 addi x15, x0, 1 sw x15, 0(x17) sw x0, 0(x18) addi x11, x0, 0 addi x12, x0, 10 addi x16, x0, 1 producer_loop: beq x11, x12, end_test wait_empty: amoand.w x14, x0, (x17) beq x14, x0, wait_empty addi x13, x11, 100 sw x13, 0(x10) amoor.w x0, x16, (x18) addi x11, x11, 1 j producer_loop end_test: addi x26, x0, 1 spin: j spin </pre>	<pre> .section .text.init .globl _start _start: j main .text main: addi x10, x0, 0x200 addi x17, x10, 0x40 addi x18, x10, 0x80 addi x11, x0, 0 addi x12, x0, 10 addi x16, x0, 1 addi x20, x0, 0 consumer_loop: beq x11, x12, end_test wait_full: amoand.w x14, x0, (x18) beq x14, x0, wait_full lw x13, 0(x10) add x20, x20, x13 amoor.w x0, x16, (x17) addi x11, x11, 1 j consumer_loop end_test: addi x26, x0, 1 # x26 = 1 spin: j spin </pre>

Testcase 4: AMOMIN/AMOMAX (Producer - Consumer)

CPU A	CPU B
<pre> .section .text.init .globl _start _start: j main .text main: addi x10, x0, 0x200 addi x17, x10, 0x40 addi x18, x10, 0x80 addi x15, x0, 1 sw x15, 0(x17) sw x0, 0(x18) addi x11, x0, 0 addi x12, x0, 10 addi x16, x0, 1 producer_loop: beq x11, x12, end_test wait_empty: amomin.w x14, x0, (x17) beq x14, x0, wait_empty addi x13, x11, 100 sw x13, 0(x10) amomax.w x0, x16, (x18) addi x11, x11, 1 j producer_loop end_test: addi x26, x0, 1 spin: j spin </pre>	<pre> .section .text.init .globl _start _start: j main .text main: addi x10, x0, 0x200 addi x17, x10, 0x40 addi x18, x10, 0x80 addi x11, x0, 0 addi x12, x0, 10 addi x16, x0, 1 addi x20, x0, 0 consumer_loop: beq x11, x12, end_test wait_full: amomin.w x14, x0, (x18) beq x14, x0, wait_full lw x13, 0(x10) add x20, x20, x13 amomax.w x0, x16, (x17) addi x11, x11, 1 j consumer_loop end_test: addi x26, x0, 1 spin: j spin </pre>

Testcase 5: AMOADD/AMOXOR (Producer - Consumer)

CPU A	CPU B
<pre> .section .text.init .globl _start _start: j main .text main: addi x10, x0, 0x200 addi x17, x10, 0x40 addi x18, x10, 0x80 addi x15, x0, 1 sw x15, 0(x17) sw x0, 0(x18) addi x11, x0, 0 addi x12, x0, 10 addi x16, x0, 1 producer_loop: beq x11, x12, end_test wait_empty: amoadd.w x14, x0, (x17) beq x14, x0, wait_empty amoxor.w x0, x16, (x17) addi x13, x11, 100 sw x13, 0(x10) amoxor.w x0, x16, (x18) addi x11, x11, 1 j producer_loop end_test: addi x26, x0, 1 spin: j spin </pre>	<pre> .section .text.init .globl _start _start: j main .text main: addi x10, x0, 0x200 addi x17, x10, 0x40 addi x18, x10, 0x80 addi x11, x0, 0 addi x12, x0, 10 addi x16, x0, 1 addi x20, x0, 0 consumer_loop: beq x11, x12, end_test wait_full: amoadd.w x14, x0, (x18) beq x14, x0, wait_full amoxor.w x0, x16, (x18) lw x13, 0(x10) add x20, x20, x13 amoxor.w x0, x16, (x17) addi x11, x11, 1 j consumer_loop end_test: addi x26, x0, 1 spin: j spin </pre>

PHỤ LỤC II. KẾT QUẢ MÔ PHÒNG

Testcase 1: Kết quả mô phỏng và dự đoán

> [31][31:0]	0000000d	0000000a	> [31][31:0]	00000014	...	00000014
> [30][31:0]	00000000	00000000	> [30][31:0]	00000000		00000000
> [29][31:0]	00000000	00000000	> [29][31:0]	00000000		00000000
> [28][31:0]	00000001	00000001	> [28][31:0]	00000001		00000001
> [27][31:0]	00000000	00000000	> [27][31:0]	00000000		00000000
> [26][31:0]	00000001	00000001	> [26][31:0]	00000001	00000000	00000001
> [25][31:0]	00000000	00000000	> [25][31:0]	00000000		00000000
> [24][31:0]	00000000	00000000	> [24][31:0]	00000000		00000000
> [23][31:0]	00000000	00000000	> [23][31:0]	00000000		00000000
> [22][31:0]	00000000	00000000	> [22][31:0]	00000000		00000000
> [21][31:0]	00000000	00000000	> [21][31:0]	00000000		00000000
> [20][31:0]	00000000	00000000	> [20][31:0]	00000000		00000000
> [19][31:0]	00000000	00000000	> [19][31:0]	00000000		00000000
> [18][31:0]	00000000	00000000	> [18][31:0]	00000000		00000000
> [17][31:0]	00000000	00000000	> [17][31:0]	00000000		00000000
> [16][31:0]	00000000	00000000	> [16][31:0]	00000000		00000000
> [15][31:0]	00000000	00000000	> [15][31:0]	00000000		00000000
> [14][31:0]	00000000	00000000	> [14][31:0]	00000000		00000000
> [13][31:0]	00000000	00000000	> [13][31:0]	00000000		00000000
> [12][31:0]	00000000	00000000	> [12][31:0]	00000000		00000000
> [11][31:0]	0000000d	0000000a	> [11][31:0]	00000014	00000000	00000014
> [10][31:0]	00000000	00000000	> [10][31:0]	00000000		00000000
> [9][31:0]	00000000	00000000	> [9][31:0]	00000000		00000000
> [8][31:0]	00000000	00000000	> [8][31:0]	00000000		00000000
> [7][31:0]	00000000	00000000	> [7][31:0]	00000000	00...	00000000
> [6][31:0]	00000204	00000204	> [6][31:0]	00000204		00000204
> [5][31:0]	00000200	00000200	> [5][31:0]	00000200		00000200
> [4][31:0]	00000000	00000000	> [4][31:0]	00000000		00000000
> [3][31:0]	00000000	00000000	> [3][31:0]	00000000		00000000
> [2][31:0]	00000000	00000000	> [2][31:0]	00000000		00000000
> [1][31:0]	00000000	00000000	> [1][31:0]	00000000		00000000
> [0][31:0]	00000000	00000000	> [0][31:0]	00000000		00000000

expected							
CPU A				CPU B			
x0	0x00000000	x16	0x00000000	x0	0x00000000	x16	0x00000000
X1	0x00000000	x17	0x00000000	X1	0x00000000	x17	0x00000000
x2	0x00000000	x18	0x00000000	x2	0x00000000	x18	0x00000000
x3	0x00000000	x19	0x00000000	x3	0x00000000	x19	0x00000000
x4	0x00000000	x20	0x00000000	x4	0x00000000	x20	0x00000000
x5	0x00000200	x21	0x00000000	x5	0x00000200	x21	0x00000000
x6	0x00000204	x22	0x00000000	x6	0x00000204	x22	0x00000000
x7	0x00000000	x23	0x00000000	x7	0x00000000	x23	0x00000000
x8	0x00000001	x24	0x00000000	x8	0x00000000	x24	0x00000000
x9	0x00000002	x25	0x00000000	x9	0x00000000	x25	0x00000000
x10	0x00000003	x26	0x00000001	x10	0x00000000	x26	0x00000001
x11	0x0000000d	x27	0x00000000	x11	0x00000014	x27	0x00000000
x12	0x00000000	x28	0x00000001	x12	0x00000000	x28	0x00000001
x13	0x00000001	x29	0x00000000	x13	0x00000000	x29	0x00000000
x14	0x00000002	x30	0x00000000	x14	0x00000000	x30	0x00000000
x15	0x00000003	x31	0x0000000d	x15	0x00000000	x31	0x00000014

Testcase 2: Kết quả mô phỏng và dự đoán

> [31][31:0]	00000000	00000000	> [31][31:0]	00000000	00000000
> [30][31:0]	00000000	00000000	> [30][31:0]	00000000	00000000
> [29][31:0]	00000000	00000000	> [29][31:0]	00000000	00000000
> [28][31:0]	00000000	00000000	> [28][31:0]	00000000	00000000
> [27][31:0]	00000000	00000000	> [27][31:0]	00000000	00000000
> [26][31:0]	00000001	00000001	> [26][31:0]	00000001	00000000 00000001
> [25][31:0]	00000000	00000000	> [25][31:0]	00000000	00000000
> [24][31:0]	00000000	00000000	> [24][31:0]	00000000	00000000
> [23][31:0]	00000000	00000000	> [23][31:0]	00000000	00000000
> [22][31:0]	00000000	00000000	> [22][31:0]	00000000	00000000
> [21][31:0]	00000000	00000000	> [21][31:0]	00000000	00000000
> [20][31:0]	00000000	00000000	> [20][31:0]	00000415	00000415
> [19][31:0]	00000000	00000000	> [19][31:0]	00000000	00000000
> [18][31:0]	00000280	00000280	> [18][31:0]	00000280	00000280
> [17][31:0]	00000240	00000240	> [17][31:0]	00000240	00000240
> [16][31:0]	00000001	00000001	> [16][31:0]	00000001	00000001
> [15][31:0]	00000001	00000001	> [15][31:0]	00000000	00000000
> [14][31:0]	00000001	00000001	> [14][31:0]	00000001	00000001
> [13][31:0]	0000006d	0000006d	> [13][31:0]	0000006d	0000006d
> [12][31:0]	0000000a	0000000a	> [12][31:0]	0000000a	0000000a
> [11][31:0]	0000000a	0000000a	> [11][31:0]	0000000a	00... 0000000a
> [10][31:0]	00000200	00000200	> [10][31:0]	00000200	00000200
> [9][31:0]	00000000	00000000	> [9][31:0]	00000000	00000000
> [8][31:0]	00000000	00000000	> [8][31:0]	00000000	00000000
> [7][31:0]	00000000	00000000	> [7][31:0]	00000000	00000000
> [6][31:0]	00000000	00000000	> [6][31:0]	00000000	00000000
> [5][31:0]	00000000	00000000	> [5][31:0]	00000000	00000000
> [4][31:0]	00000000	00000000	> [4][31:0]	00000000	00000000
> [3][31:0]	00000000	00000000	> [3][31:0]	00000000	00000000
> [2][31:0]	00000000	00000000	> [2][31:0]	00000000	00000000
> [1][31:0]	00000000	00000000	> [1][31:0]	00000000	00000000
> [0][31:0]	00000000	00000000	> [0][31:0]	00000000	00000000

CPU A				CPU B			
x0	0x00000000	x16	0x00000001	x0	0x00000000	x16	0x00000001
X1	0x00000000	x17	0x00000240	X1	0x00000000	x17	0x00000240
x2	0x00000000	x18	0x00000280	x2	0x00000000	x18	0x00000280
x3	0x00000000	x19	0x00000000	x3	0x00000000	x19	0x00000000
x4	0x00000000	x20	0x00000000	x4	0x00000000	x20	0x00000415
x5	0x00000000	x21	0x00000000	x5	0x00000000	x21	0x00000000
x6	0x00000000	x22	0x00000000	x6	0x00000000	x22	0x00000000
x7	0x00000000	x23	0x00000000	x7	0x00000000	x23	0x00000000
x8	0x00000000	x24	0x00000000	x8	0x00000000	x24	0x00000000
x9	0x00000000	x25	0x00000000	x9	0x00000000	x25	0x00000000
x10	0x00000200	x26	0x00000001	x10	0x00000200	x26	0x00000000
x11	0x0000000a	x27	0x00000000	x11	0x0000000a	x27	0x00000000
x12	0x0000000a	x28	0x00000000	x12	0x0000000a	x28	0x00000001
x13	0x0000006d	x29	0x00000000	x13	0x0000006d	x29	0x00000000
x14	0x00000001	x30	0x00000000	x14	0x00000001	x30	0x00000000
x15	0x00000001	x31	0x00000000	x15	0x00000000	x31	0x00000000

Testcase 3: Kết quả mô phỏng và dự đoán

> [31][31:0]	00000000	00000000	> [31][31:0]	00000000	00000000
> [30][31:0]	00000000	00000000	> [30][31:0]	00000000	00000000
> [29][31:0]	00000000	00000000	> [29][31:0]	00000000	00000000
> [28][31:0]	00000000	00000000	> [28][31:0]	00000000	00000000
> [27][31:0]	00000000	00000000	> [27][31:0]	00000000	00000000
> [26][31:0]	00000001	00000001	> [26][31:0]	00000001	00000000 X 00000001
> [25][31:0]	00000000	00000000	> [25][31:0]	00000000	00000000
> [24][31:0]	00000000	00000000	> [24][31:0]	00000000	00000000
> [23][31:0]	00000000	00000000	> [23][31:0]	00000000	00000000
> [22][31:0]	00000000	00000000	> [22][31:0]	00000000	00000000
> [21][31:0]	00000000	00000000	> [21][31:0]	00000000	00000000
> [20][31:0]	00000000	00000000	> [20][31:0]	00000415	00000415 X
> [19][31:0]	00000000	00000000	> [19][31:0]	00000000	00000000
> [18][31:0]	00000280	00000280	> [18][31:0]	00000280	00000280
> [17][31:0]	00000240	00000240	> [17][31:0]	00000240	00000240
> [16][31:0]	00000001	00000001	> [16][31:0]	00000001	00000001
> [15][31:0]	00000001	00000001	> [15][31:0]	00000000	00000000
> [14][31:0]	00000001	00000001	> [14][31:0]	00000001	00000001
> [13][31:0]	0000006d	0000006d	> [13][31:0]	0000006d	0000006d
> [12][31:0]	0000000a	0000000a	> [12][31:0]	0000000a	0000000a
> [11][31:0]	0000000a	0000000a	> [11][31:0]	0000000a	00... X 0000000a
> [10][31:0]	00000200	00000200	> [10][31:0]	00000200	00000200
> [9][31:0]	00000000	00000000	> [9][31:0]	00000000	00000000
> [8][31:0]	00000000	00000000	> [8][31:0]	00000000	00000000
> [7][31:0]	00000000	00000000	> [7][31:0]	00000000	00000000
> [6][31:0]	00000000	00000000	> [6][31:0]	00000000	00000000
> [5][31:0]	00000000	00000000	> [5][31:0]	00000000	00000000
> [4][31:0]	00000000	00000000	> [4][31:0]	00000000	00000000
> [3][31:0]	00000000	00000000	> [3][31:0]	00000000	00000000
> [2][31:0]	00000000	00000000	> [2][31:0]	00000000	00000000
> [1][31:0]	00000000	00000000	> [1][31:0]	00000000	00000000
> [0][31:0]	00000000	00000000	> [0][31:0]	00000000	00000000

CPU A				CPU B			
x0	0x00000000	x16	0x00000001	x0	0x00000000	x16	0x00000001
x1	0x00000000	x17	0x00000240	x1	0x00000000	x17	0x00000240
x2	0x00000000	x18	0x00000280	x2	0x00000000	x18	0x00000280
x3	0x00000000	x19	0x00000000	x3	0x00000000	x19	0x00000000
x4	0x00000000	x20	0x00000000	x4	0x00000000	x20	0x00000415
x5	0x00000000	x21	0x00000000	x5	0x00000000	x21	0x00000000
x6	0x00000000	x22	0x00000000	x6	0x00000000	x22	0x00000000
x7	0x00000000	x23	0x00000000	x7	0x00000000	x23	0x00000000
x8	0x00000000	x24	0x00000000	x8	0x00000000	x24	0x00000000
x9	0x00000000	x25	0x00000000	x9	0x00000000	x25	0x00000000
x10	0x00000200	x26	0x00000001	x10	0x00000200	x26	0x00000001
x11	0x0000000a	x27	0x00000000	x11	0x0000000a	x27	0x00000000
x12	0x0000000a	x28	0x00000000	x12	0x0000000a	x28	0x00000000
x13	0x0000006d	x29	0x00000000	x13	0x0000006d	x29	0x00000000
x14	0x00000001	x30	0x00000000	x14	0x00000001	x30	0x00000000
x15	0x00000001	x31	0x00000000	x15	0x00000000	x31	0x00000000

Testcase 4: Kết quả mô phỏng và dự đoán

> [31][31:0]	00000000	00000000	> [31][31:0]	00000000	00000000
> [30][31:0]	00000000	00000000	> [30][31:0]	00000000	00000000
> [29][31:0]	00000000	00000000	> [29][31:0]	00000000	00000000
> [28][31:0]	00000000	00000000	> [28][31:0]	00000000	00000000
> [27][31:0]	00000000	00000000	> [27][31:0]	00000000	00000000
> [26][31:0]	00000001	00000001	> [26][31:0]	00000001	00000001
> [25][31:0]	00000000	00000000	> [25][31:0]	00000000	00000000
> [24][31:0]	00000000	00000000	> [24][31:0]	00000000	00000000
> [23][31:0]	00000000	00000000	> [23][31:0]	00000000	00000000
> [22][31:0]	00000000	00000000	> [22][31:0]	00000000	00000000
> [21][31:0]	00000000	00000000	> [21][31:0]	00000000	00000000
> [20][31:0]	00000000	00000000	> [20][31:0]	00000415	00000415
> [19][31:0]	00000000	00000000	> [19][31:0]	00000000	00000000
> [18][31:0]	00000280	00000280	> [18][31:0]	00000280	00000280
> [17][31:0]	00000240	00000240	> [17][31:0]	00000240	00000240
> [16][31:0]	00000001	00000001	> [16][31:0]	00000001	00000001
> [15][31:0]	00000001	00000001	> [15][31:0]	00000000	00000000
> [14][31:0]	00000001	00000001	> [14][31:0]	00000001	00000001
> [13][31:0]	0000006d	0000006d	> [13][31:0]	0000006d	0000006d
> [12][31:0]	0000000a	0000000a	> [12][31:0]	0000000a	0000000a
> [11][31:0]	0000000a	0000000a	> [11][31:0]	0000000a	0000000a
> [10][31:0]	00000200	00000200	> [10][31:0]	00000200	00000200
> [9][31:0]	00000000	00000000	> [9][31:0]	00000000	00000000
> [8][31:0]	00000000	00000000	> [8][31:0]	00000000	00000000
> [7][31:0]	00000000	00000000	> [7][31:0]	00000000	00000000
> [6][31:0]	00000000	00000000	> [6][31:0]	00000000	00000000
> [5][31:0]	00000000	00000000	> [5][31:0]	00000000	00000000
> [4][31:0]	00000000	00000000	> [4][31:0]	00000000	00000000
> [3][31:0]	00000000	00000000	> [3][31:0]	00000000	00000000
> [2][31:0]	00000000	00000000	> [2][31:0]	00000000	00000000
> [1][31:0]	00000000	00000000	> [1][31:0]	00000000	00000000
> [0][31:0]	00000000	00000000	> [0][31:0]	00000000	00000000

CPU A				CPU B			
x0	0x00000000	x16	0x00000001	x0	0x00000000	x16	0x00000001
x1	0x00000000	x17	0x00000240	x1	0x00000000	x17	0x00000240
x2	0x00000000	x18	0x00000280	x2	0x00000000	x18	0x00000280
x3	0x00000000	x19	0x00000000	x3	0x00000000	x19	0x00000000
x4	0x00000000	x20	0x00000000	x4	0x00000000	x20	0x00000415
x5	0x00000000	x21	0x00000000	x5	0x00000000	x21	0x00000000
x6	0x00000000	x22	0x00000000	x6	0x00000000	x22	0x00000000
x7	0x00000000	x23	0x00000000	x7	0x00000000	x23	0x00000000
x8	0x00000000	x24	0x00000000	x8	0x00000000	x24	0x00000000
x9	0x00000000	x25	0x00000000	x9	0x00000000	x25	0x00000000
x10	0x00000200	x26	0x00000001	x10	0x00000200	x26	0x00000001
x11	0x0000000a	x27	0x00000000	x11	0x0000000a	x27	0x00000000
x12	0x0000000a	x28	0x00000000	x12	0x0000000a	x28	0x00000000
x13	0x0000006d	x29	0x00000000	x13	0x0000006d	x29	0x00000000
x14	0x00000001	x30	0x00000000	x14	0x00000001	x30	0x00000000
x15	0x00000001	x31	0x00000000	x15	0x00000000	x31	0x00000000

Testcase 5: Kết quả mô phỏng và dự đoán

> [31][31:0]	00000000	00000000	> [31][31:0]	00000000	00000000
> [30][31:0]	00000000	00000000	> [30][31:0]	00000000	00000000
> [29][31:0]	00000000	00000000	> [29][31:0]	00000000	00000000
> [28][31:0]	00000000	00000000	> [28][31:0]	00000000	00000000
> [27][31:0]	00000000	00000000	> [27][31:0]	00000000	00000000
> [26][31:0]	00000001	00000001	> [26][31:0]	00000001	00000001
> [25][31:0]	00000000	00000000	> [25][31:0]	00000000	00000000
> [24][31:0]	00000000	00000000	> [24][31:0]	00000000	00000000
> [23][31:0]	00000000	00000000	> [23][31:0]	00000000	00000000
> [22][31:0]	00000000	00000000	> [22][31:0]	00000000	00000000
> [21][31:0]	00000000	00000000	> [21][31:0]	00000000	00000000
> [20][31:0]	00000000	00000000	> [20][31:0]	00000415	00000415
> [19][31:0]	00000000	00000000	> [19][31:0]	00000000	00000000
> [18][31:0]	00000280	00000280	> [18][31:0]	00000280	00000280
> [17][31:0]	00000240	00000240	> [17][31:0]	00000240	00000240
> [16][31:0]	00000001	00000001	> [16][31:0]	00000001	00000001
> [15][31:0]	00000001	00000001	> [15][31:0]	00000000	00000000
> [14][31:0]	00000001	00000001	> [14][31:0]	00000001	00000001
> [13][31:0]	0000006d	0000006d	> [13][31:0]	0000006d	0000006d
> [12][31:0]	0000000a	0000000a	> [12][31:0]	0000000a	0000000a
> [11][31:0]	0000000a	0000000a	> [11][31:0]	0000000a	0000000a
> [10][31:0]	00000200	00000200	> [10][31:0]	00000200	00000200
> [9][31:0]	00000000	00000000	> [9][31:0]	00000000	00000000
> [8][31:0]	00000000	00000000	> [8][31:0]	00000000	00000000
> [7][31:0]	00000000	00000000	> [7][31:0]	00000000	00000000
> [6][31:0]	00000000	00000000	> [6][31:0]	00000000	00000000
> [5][31:0]	00000000	00000000	> [5][31:0]	00000000	00000000
> [4][31:0]	00000000	00000000	> [4][31:0]	00000000	00000000
> [3][31:0]	00000000	00000000	> [3][31:0]	00000000	00000000
> [2][31:0]	00000000	00000000	> [2][31:0]	00000000	00000000
> [1][31:0]	00000000	00000000	> [1][31:0]	00000000	00000000
> [0][31:0]	00000000	00000000	> [0][31:0]	00000000	00000000

CPU A				CPU B			
x0	0x00000000	x16	0x00000001	x0	0x00000000	x16	0x00000001
x1	0x00000000	x17	0x00000240	x1	0x00000000	x17	0x00000240
x2	0x00000000	x18	0x00000280	x2	0x00000000	x18	0x00000280
x3	0x00000000	x19	0x00000000	x3	0x00000000	x19	0x00000000
x4	0x00000000	x20	0x00000000	x4	0x00000000	x20	0x00000415
x5	0x00000000	x21	0x00000000	x5	0x00000000	x21	0x00000000
x6	0x00000000	x22	0x00000000	x6	0x00000000	x22	0x00000000
x7	0x00000000	x23	0x00000000	x7	0x00000000	x23	0x00000000
x8	0x00000000	x24	0x00000000	x8	0x00000000	x24	0x00000000
x9	0x00000000	x25	0x00000000	x9	0x00000000	x25	0x00000000
x10	0x00000200	x26	0x00000001	x10	0x00000200	x26	0x00000001
x11	0x0000000a	x27	0x00000000	x11	0x0000000a	x27	0x00000000
x12	0x0000000a	x28	0x00000000	x12	0x0000000a	x28	0x00000000
x13	0x0000006d	x29	0x00000000	x13	0x0000006d	x29	0x00000000
x14	0x00000001	x30	0x00000000	x14	0x00000001	x30	0x00000000
x15	0x00000001	x31	0x00000000	x15	0x00000000	x31	0x00000000

PHỤ LỤC III. BẢNG MÔ TẢ TÍN HIỆU CÁC MODULE

Bảng PIII.1: Đặc tả máy trạng thái điều khiển D-Cache Controller

Trạng thái hiện tại	Chức năng	Điều kiện kích hoạt	Trạng thái kế tiếp
TAG_CHECK (Mặc định)	So sánh Tag, kiểm tra quyền MOESI, xử lý Hit/Miss và chặn ngắt Snoop	Bộ dò Snoop bận (snoop_busy = 1) HOẶC Hit (đủ quyền thao tác)	TAG_CHECK
		Ghi đánh trúng - Write Hit (thiếu quyền) HOẶC Trượt Cache - Miss (Victim Clean)	BUS_REQ
		Trượt Cache - Miss (Victim Dirty)	WB_SEND
		Đánh trúng Cache - Hit VÀ Lệnh xử lý nguyên tử (i_atomic_amo = 1)	AMO_EXEC
WB_SEND	Gửi yêu cầu xin Bus để đẩy dữ liệu bản (Write-back) ra bộ nhớ chính	Có tín hiệu Snoop ngắt ngang (snoop_busy = 1)	TAG_CHECK
		Kênh Bus chấp nhận (i_req_ready = 1)	WB_WAIT
WB_WAIT	Chờ xác nhận ghi trả hoàn tất từ Bus	Xác nhận hoàn tất hợp lệ (i_resp_valid = 1)	BUS_REQ
BUS_REQ	Xin Bus để lấy dữ liệu mới hoặc nâng cấp quyền ghi (Upgrade)	Có tín hiệu Snoop ngắt ngang (snoop_busy = 1)	TAG_CHECK
		Kênh Bus chấp nhận (i_req_ready = 1)	BUS_WAIT
BUS_WAIT	Treo Pipeline, chờ dữ liệu trả về từ L2	Dữ liệu trả về hợp lệ (i_resp_valid = 1)	UPDATE

	Cache hoặc Bus Arbiter		
UPDATE	Phát tín hiệu Write Enable để nạp khối dữ liệu mới, cập nhật Tag và trạng thái MOESI vào SRAM	Vô điều kiện	WAIT_RAM
AMO_EXEC	Thực thi phép toán nguyên tử và ghi trực tiếp kết quả vào bộ nhớ SRAM	Vô điều kiện	TAG_CHECK
WAIT_RAM	Tạo độ trễ 1 chu kỳ để phần cứng mảng nhớ vật lý hoàn tất lưu trữ	Vô điều kiện	FINISH_REQ
FINISH_REQ	Hoàn tất giao dịch ghi dữ liệu cục bộ, đảm bảo quyền Bus không bị cướp và giải phóng Pipeline	Vô điều kiện	TAG_CHECK

Bảng PIII.2: Mô tả tín hiệu I/O của D-Cache

Tín hiệu	I/O	Độ rộng	Mô tả
clk	I	1	Xung nhịp clock
rst_n	I	1	Reset đồng bộ tích cực mức thấp
CPU Interface (Giao tiếp với CPU Pipeline)			
cpu_req	I	1	Tín hiệu báo có yêu cầu truy xuất bộ nhớ từ CPU
cpu_we	I	1	Tín hiệu cho phép ghi từ CPU (Write Enable: 0 = Đọc, 1 = Ghi)

cpu_addr	I	ADDR_W	Địa chỉ bộ nhớ do CPU gửi tới để truy xuất
cpu_din	I	DATA_W	Dữ liệu CPU gửi đến muốn ghi vào bộ nhớ
cpu_size	I	2	Kích thước dữ liệu cần truy xuất (00 = Byte, 01 = Halfword, 10 = Word)
data_rdata	O	DATA_W	Dữ liệu từ Cache trả về cho CPU (Load)
pipeline_stall	O	1	Tín hiệu yêu cầu CPU tạm dừng pipeline (khi Cache Miss hoặc đang bận xử lý Snoop)
Atomic Interface (Giao tiếp cho các tập lệnh Atomic)			
cpu_lr	I	1	Tín hiệu yêu cầu thực thi Load-Reserved (LR)
cpu_sc	I	1	Tín hiệu yêu cầu thực thi Store-Conditional (SC)
cpu_amo	I	1	Tín hiệu yêu cầu thực thi lệnh Atomic Memory Operation (AMO)
cpu_amo_op	I	3	Mã định tuyến loại phép toán AMO (Add, Swap, Max, Min, v.v.)
o_sc_success	O	1	Kết quả của lệnh SC trả về cho CPU (0 = Thành công, 1 = Thất bại)
Request Channel (D-Cache gửi yêu cầu ra Bus/Snoop Filter)			
i_req_ready	I	1	Bus/Arbiter báo hiệu đã sẵn sàng nhận yêu cầu từ Cache
o_req_valid	O	1	D-Cache báo hiệu có yêu cầu xin mượn Bus hợp lệ
o_req_addr	O	ADDR_W	Địa chỉ bộ nhớ mà D-Cache muốn thao tác qua hệ thống Bus
o_req_cmd	O	2	Lệnh gửi lên Bus (00=Read Shared, 01=Write Back, 10=Upgrade/Invalidate Others, 11=Read Unique)
o_req_data	O	LINE_W	Dữ liệu nguyên dòng đẩy lên Bus

o_req_wb	O	1	Báo hiệu cho Arbiter đây là tiến trình Writeback (Victim dirty) để ưu tiên cấp Bus sớm
Snoop Response D-Cache (D-Cache nhận phản hồi từ Bus)			
i_resp_valid	I	1	RAM/Bus báo hiệu đã trả về dữ liệu hoặc cờ hiệu hợp lệ
i_resp_data	I	LINE_W	Dữ liệu nguyên dòng từ RAM/Snoop Filter trả về để nạp vào bộ đệm (Refill)
o_resp_ready	O	1	D-Cache báo hiệu đã sẵn sàng nhận phản hồi từ Bus
Snoop Request D-Cache (Snoop Filter gửi yêu cầu vào D-Cache)			
i_snp_req_valid	I	1	Có yêu cầu Snoop từ Core/Cache khác gửi đến cần xử lý
i_snp_req_addr	I	ADDR_W	Địa chỉ bộ nhớ cần dò (Snoop) bên trong D-Cache
i_snp_req_cmd	I	2	Lệnh Snoop (Ví dụ: kiểm tra dữ liệu, Invalidate xoá dữ liệu)
i_resp_is_shared	I	1	Cờ báo hiệu dữ liệu nạp về từ Bus có đang tồn tại ở Core khác không (Quyết định trạng thái Shared (S) hay Exclusive (E))
o_snp_req_ready	O	1	D-Cache sẵn sàng tiếp nhận yêu cầu Snoop (Pipeline không bị khóa)
D-Cache Response Snoop (D-Cache trả phản hồi Snoop ra ngoài)			
o_snp_resp_valid	O	1	Tín hiệu báo hiệu kết quả trả về từ quá trình Snoop là hợp lệ
o_snp_resp_hit	O	1	Báo hiệu kết quả dò Snoop là Cache Hit (địa chỉ yêu cầu đang tồn tại trong cache)
o_snp_resp_data	O	LINE_W	Dữ liệu nguyên dòng (Line) trả về cho Core khác hoặc Main Memory nếu có Snoop Hit (và dữ liệu cần chia sẻ/Writeback)

Bảng PIII.3: Đặc tả máy trạng thái điều khiển I-Cache

Trạng thái hiện tại	Chức năng	Điều kiện kích hoạt	Trạng thái kế tiếp
TAG_CHECK (Mặc định)	Đánh giá thẻ định danh để xác định trạng thái đánh trúng/trượt. Nếu không có yêu cầu đọc hoặc đánh trúng, giải phóng tín hiệu đóng băng cho phép pipeline hoạt động. Nếu trượt, phát tín hiệu khóa pipeline	Không có yêu cầu đọc ($\text{cpu_req} = 0$) HOẶC Đánh trúng bộ đệm ($\text{hit} = 1$)	TAG_CHECK
		Có yêu cầu đọc ($\text{cpu_req} = 1$) VÀ Trượt bộ đệm ($\text{hit} = 0$)	ALLOC_REQ
ALLOC_REQ	Kích hoạt cờ yêu cầu nạp lệnh, gửi tín hiệu xin cấp phát dữ liệu xuống kiến trúc bộ nhớ cấp dưới (L2 Cache hoặc bộ phân xử hệ thống)	Nhận tín hiệu chấp nhận yêu cầu từ bộ nhớ cấp dưới ($\text{i_mem_req_ready} = 1$)	ALLOC_WAIT
ALLOC_WAIT	Trạng thái treo. Duy trì cờ yêu cầu và bật tín hiệu sẵn sàng tiếp nhận để chờ dữ liệu nguyên dòng được trả về từ bộ nhớ cấp dưới	Nhận được cờ hiệu dữ liệu trả về đã hợp lệ ($\text{imem_rdata_valid} = 1$)	UPDATE
UPDATE	Phát các tín hiệu cho phép ghi để nạp khối dữ liệu lệnh mới và cập nhật thẻ định danh tương ứng vào SRAM nội bộ	Vô điều kiện	WAIT_RAM
WAIT_RAM	Tạo độ trễ cứng 1 chu kỳ để phần cứng mảng nhớ hoàn tất quá trình ghi. Điều hướng mạch ghép kênh sử dụng chỉ mục gốc để sẵn sàng truy xuất lại lệnh vừa bị trượt	Vô điều kiện	TAG_CHECK

Bảng PIII.4: Mô tả tín hiệu I/O của L1 I-Cache

Tín hiệu	I/O	Độ rộng	Mô tả
clk	I	1	Xung nhịp clock
rst_n	I	1	Reset đồng bộ tích cực mức thấp
CPU Interface (Giao tiếp với tầng Fetch của CPU)			
cpu_req	I	1	Tín hiệu yêu cầu nạp lệnh (Instruction Fetch request) từ CPU
fetch_stall	I	1	Tín hiệu báo hiệu tầng Fetch của CPU đang bị tạm dừng, I-Cache cần giữ nguyên trạng thái đọc hiện tại
cpu_addr	I	ADDR_W	Địa chỉ (PC - Program Counter) do CPU gửi tới để yêu cầu lấy lệnh
data_rdata	O	DATA_W	Lệnh (Instruction) đọc được từ I-Cache trả về cho CPU
pipeline_stall	O	1	Tín hiệu từ I-Cache yêu cầu CPU tạm dừng pipeline (thường do I-Cache Miss đang phải đợi nạp dữ liệu)
D-Cache Interface (Giao tiếp đồng bộ với D-Cache)			
dcache_stall	I	1	Tín hiệu tạm dừng từ Data Cache. Được dùng để đồng bộ stall toàn hệ thống (khi D-Cache đang bận thì I-Cache cũng giữ nguyên trạng thái, tránh lệch nhịp)
L2 / Bus Request (I-Cache gửi yêu cầu lấy lệnh)			
i_l2_req_ready	I	1	Bộ nhớ cấp dưới (L2 Cache hoặc Bus Arbiter) báo hiệu đã sẵn sàng nhận yêu cầu từ I-Cache
o_l2_req_valid	O	1	I-Cache báo hiệu có yêu cầu đọc bộ nhớ hợp lệ (xảy ra khi I-Cache Miss)

o_l2_req_addr	O	ADDR_W	Địa chỉ gốc của khối lệnh (block/line) mà I-Cache muốn lấy từ L2/RAM.
L2 / Bus Response (Nhận lệnh nạp về từ L2)			
i_l2_rdata_valid	I	1	Tín hiệu từ L2/Bus báo dữ liệu trả về đã sẵn sàng và hợp lệ
i_l2_rdata	I	LINE_W	Dữ liệu nguyên dòng lệnh (Cache Line - chứa nhiều word lệnh) từ L2/Bus trả về để nạp (refill) vào I-Cache
o_l2_rdata_ready	O	1	I-Cache báo hiệu đã sẵn sàng nhận và ghi dữ liệu nạp lại (Refill) từ L2/Bus

Bảng PIII.5: Đặc tả máy trạng thái điều khiển L2 Cache

Trạng thái hiện tại	Chức năng	Điều kiện kích hoạt	Trạng thái kế tiếp
TAG_CHECK (Mặc định)	Kiểm tra thẻ định danh để xác định Hit/Miss. Nếu Hit, xử lý và trả kết quả. Nếu Miss, rẽ nhánh dựa trên trạng thái của khối dữ liệu bị thay thế (Victim)	Hit hoặc không có yêu cầu	TAG_CHECK
		Trượt + Victim có dữ liệu bản	AW_REQ
		Trượt + Victim sạch + Lệnh Đọc	AR_REQ
		Trượt + Victim sạch + Lệnh Ghi	UPDATE_WM
AW_REQ	Phát bản tin yêu cầu ghi địa chỉ lên kênh AXI Write Address (AW)	Kênh AW chấp nhận (iAWREADY = 1)	W_DATA
W_DATA	Đẩy tuần tự các khối dữ liệu (Burst) ra kênh AXI Write Data (W)	Bộ nhớ đã nhận (iWREADY = 1) VÀ là gói dữ liệu cuối cùng (oWLAST = 1)	B_WAIT

		Ngược lại, duy trì trạng thái	W_DATA
B_WAIT	Chờ phản hồi xác nhận quá trình ghi hoàn tất từ kênh AXI Write Response (B). Bỏ qua kiểm tra mã lỗi để tránh treo hệ thống	Có phản hồi hợp lệ (iBVALID = 1) + Lệnh Đọc	AR_REQ
		Có phản hồi hợp lệ (iBVALID = 1) + Lệnh Ghi	UPDATE_WM
AR_REQ	Phát bản tin yêu cầu đọc địa chỉ lên kênh AXI Read Address (AR) và giải phóng pipeline cho L1	Kênh AR chấp nhận (iARREADY = 1)	R_WAIT
R_WAIT	Hứng các gói dữ liệu từ kênh AXI Read Data (R) vào bộ đệm nạp (Refill Buffer)	Có dữ liệu hợp lệ (iRVALID = 1) VÀ là gói dữ liệu cuối cùng (iRLAST = 1)	REFILL_EXEC
UPDATE_WM	Trạng thái rẽ nhánh chuyên biệt cho Write Miss. Kích hoạt bộ đệm nạp để hứng dữ liệu ghi trả từ L1 thay vì đọc từ bộ nhớ chính	Vô điều kiện	REFILL_EXEC
REFILL_EXEC	Phát tín hiệu cho phép ghi (Write Enable) để đồng loạt nạp dữ liệu từ bộ đệm và thẻ định danh vào mảng SRAM	Vô điều kiện	WAIT_RAM
WAIT_RAM	Tạo độ trễ cứng 1 chu kỳ để phần cứng SRAM lưu trữ an toàn. Tạm thời chặn mọi yêu cầu mới từ L1	Vô điều kiện	TAG_CHECK

Bảng PIII.6: Mô tả tín hiệu I/O của L2 Cache

Tín hiệu (Signal)	I/O	Độ rộng	Mô tả
clk	I	1	Xung nhịp hệ thống
rst_n	I	1	Tín hiệu reset đồng bộ tích cực mức thấp
Giao tiếp Upstream (Với L1 Arbiter / L1 Cache)			
o_l1_req_ready	O	1	L2 Cache báo hiệu đã sẵn sàng tiếp nhận yêu cầu từ tầng L1
i_l1_req_valid	I	1	Tín hiệu từ L1 Arbiter báo hiệu có yêu cầu truy xuất bộ nhớ hợp lệ
i_l1_req_addr	I	ADDR_W	Địa chỉ truy xuất do L1 Arbiter gửi tới
i_l1_req_rw	I	1	Cờ chỉ định loại giao dịch (0 = Đọc lệnh/dữ liệu mới, 1 = Ghi Writeback từ L1)
i_l1_req_wdata	I	LINE_W	Khối dữ liệu nguyên dòng (Cache Line) từ L1 đẩy xuống để ghi đè (Writeback)
o_l1_resp_valid	O	1	L2 Cache báo hiệu dữ liệu trả về đã sẵn sàng và hợp lệ
o_l1_resp_rdata	O	LINE_W	Dữ liệu nguyên dòng từ L2 Cache xuất trả về cho tầng L1
pipeline_stall	O	1	Tín hiệu từ L2 yêu cầu tạm dừng pipeline khi đang bận xử lý trượt bộ đệm (Miss)
Giao tiếp Downstream (AXI4 Master Interface - Kênh AW: Write Address)			
iAWREADY	I	1	Slave (Bộ nhớ chính) báo hiệu sẵn sàng nhận địa chỉ ghi
oAWVALID	O	1	L2 Cache báo hiệu địa chỉ ghi trên kênh AW là hợp lệ

oAWADDR	O	ADDR_W	Địa chỉ bắt đầu của khối dữ liệu cần ghi trả (Writeback) ra bộ nhớ
oAWLEN	O	8	Độ dài gói truyền Burst (Số lượng word trong một Cache Line)
oAWSIZE	O	3	Kích thước của mỗi nhịp truyền dữ liệu (thường là 4 byte cho hệ 32-bit)
oAWBURST	O	2	Kiểu truyền Burst (mặc định cấu hình INCR - Tăng dần)
Giao tiếp Downstream (AXI4 Master Interface - Kênh W: Write Data)			
iWREADY	I	1	Slave báo hiệu sẵn sàng nhận gói dữ liệu ghi tiếp theo
oWVALID	O	1	L2 Cache báo hiệu dữ liệu trên kênh W đang hợp lệ
oWDATA	O	DATA_W	Gói dữ liệu đang được đẩy ra bộ nhớ
oWSTRB	O	STRB_W	Cho phép ghi byte độc lập (Byte strobe)
oWLAST	O	1	Cờ báo hiệu đây là gói dữ liệu cuối cùng trong chuỗi truyền Burst
Giao tiếp Downstream (AXI4 Master Interface - Kênh B: Write Response)			
iBVALID	I	1	Slave báo hiệu quá trình ghi toàn bộ khối Burst đã hoàn tất
iBRESP	I	2	Trạng thái mã lỗi phản hồi từ Slave
oBREADY	O	1	L2 Cache sẵn sàng nhận tín hiệu phản hồi
Giao tiếp Downstream (AXI4 Master Interface - Kênh AR: Read Address)			
iARREADY	I	1	Slave báo hiệu sẵn sàng nhận địa chỉ đọc

oARVALID	O	1	L2 Cache báo hiệu địa chỉ đọc trên kênh AR là hợp lệ
oARADDR	O	ADDR_W	Địa chỉ bộ nhớ cần nạp lệnh/dữ liệu
oARLEN	O	8	Độ dài gói truyền Burst cần lấy về
oARSIZE	O	3	Kích thước của mỗi nhịp truyền dữ liệu
oARBURST	O	2	Kiểu truyền Burst
Giao tiếp Downstream (AXI4 Master Interface - Kênh R: Read Data)			
iRVALID	I	1	Slave báo hiệu dữ liệu đọc gửi lên hợp lệ
oRREADY	O	1	L2 Cache sẵn sàng hứng dữ liệu tiếp theo
iRDATA	I	DATA_W	Gói dữ liệu đọc được trả về từ bộ nhớ
iRRESP	I	2	Trạng thái mã lỗi phản hồi từ Slave
iRLAST	I	1	Cờ từ Slave báo hiệu đây là gói dữ liệu đọc cuối cùng của chuỗi Burst

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

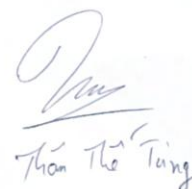
TP. Hồ Chí Minh, ngày 17 tháng 07 năm 2026

DANH MỤC KIỂM TRA BÁO CÁO KLTN SAU BẢO VỆ

STT	Nội dung kiểm tra	Tình trạng kiểm tra	
		Đã thực hiện	Chưa thực hiện
1	Bìa báo cáo theo mẫu, hướng dẫn	☒	☐
2	Kiểm tra mục lục, danh mục hình ảnh, danh mục bảng biểu, ký hiệu viết tắt	☒	☐
3	Có tóm tắt bằng tiếng Việt và tiếng Anh	☒	☐
4	Hình ảnh và bảng biểu có đầy đủ tên, đánh số thứ tự, phân tích đầy đủ	☒	☐
5	Báo cáo có đánh số trang theo quy định	☒	☐
6	Các chương được bắt đầu bằng một trang mới	☒	☐
7	Kiểm tra, chỉnh sửa lỗi chính tả, kiểu chữ và font chữ	☒	☐
8	Tài liệu tham khảo được trích dẫn đầy đủ theo quy định	☒	☐
9	Tài liệu tham khảo được trích dẫn đầy đủ theo quy định	☒	☐
10	Có giải trình chỉnh sửa theo góp ý của hội đồng	☒	☐

Xác nhận của GVHD

(Ký và ghi rõ họ tên)


Thion The Tung

TP. Hồ Chí Minh, ngày 10 tháng 07 năm 2026

**BẢNG GIẢI TRÌNH CHỈNH SỬA
BÁO CÁO KHOA LUẬN TỐT NGHIỆP SAU BẢO VỆ**

Số quyết định Hội đồng: Quyết định số 710/QĐ-ĐHCNTT

Ngày bảo vệ: 07/07/2026

Tên đề tài: Thiết kế bộ điều khiển đồng bộ cache cho các hệ thống 2 lõi dựa trên kiến trúc tập lệnh RISC-V RV321A

Danh sách sinh viên

Sinh viên 1: Văn Công Gia Luật MSSV: 22520830

Sinh viên 2: Lê Minh Hùng MSSV: 22520506

Giảng viên hướng dẫn: ThS. Thân Thế Tùng, KS. Trần Đại Dương

Căn cứ trên nội dung Biên bản Hội đồng khóa luận tốt nghiệp ngày 07/07/2026 và các góp ý của các thành viên Hội đồng, nhóm sinh viên thực hiện khóa luận xin phép được giải trình nội dung cập nhật báo cáo chi tiết như sau:

STT	Nội dung góp ý của Hội đồng	Nội dung giải trình cập nhật báo cáo	Trang
1	Điều chỉnh kết luận về tần số hoạt động và tần số tối đa	Đã bổ sung và làm rõ đánh giá hiệu năng: Hệ thống được tổng hợp với constraint 100 MHz, WNS +1,428 ns, tần số hoạt động lý thuyết đạt 116 MHz	57, 63
2	Kiểm tra, thống kê tổng hợp về các trường hợp Race Condition và deadlock	Đã bổ sung các kịch bản kiểm thử đồng bộ tiến trình (Đồng bộ hóa Mutex, Producer-Consumer) và kết quả kiểm thử chứng minh hệ thống không xảy ra hiện tượng Race Condition hay Deadlock	52, 56, 63
3	Điều chỉnh và cập nhật các sơ đồ thiết kế cho đúng với chức năng và logic xử lý	Đã rà soát, điều chỉnh và cập nhật lại toàn bộ các sơ đồ thiết kế (Hình 2.2 đến 2.10; Hình 3.3, 3.7, 3.9; Hình 4.4, 4.5, 4.6) nhằm đảm bảo phản ánh chính xác chức năng và luồng logic	8, 11, 13, 14, 15, 17, 18, 20, 25, 31, 32, 37, 38
4	Tìm hiểu và tham khảo thêm về cache coherence	Đã tìm hiểu và bổ sung định hướng nghiên cứu mở rộng với các chuẩn Cache Coherence chuyên dụng trong công nghiệp (AMBA ACE và AMBA CHI của ARM) vào mục Hướng phát triển	64, 65

Xác nhận của GVHD
(Ký và ghi rõ họ tên)


Thân Thế Tùng

Sinh viên thực hiện
(Ký và ghi rõ họ tên)


Lê Minh Hùng

Văn Công Gia Luật